

BRICKSTransaction Server

Application Programming Reference

Order number: BX26-1500-01 Edition: First Edition Date: 2026 Applies to: Bricks Transaction Server, version 1.5 and later

Document

Bricks Application Programming Reference

Audience

Application programmers writing REXX or COBOL transactions for Bricks.

Companion

README.md — installation, configuration, and operations.

Operator manual

ISPF_editor.md — the in-3270 source editor reference.

About this publication

This reference describes the application-programming interface of the **Bricks Transaction Server**: the COBOL and REXX languages it accepts, the EXEC CICS command set those programs may issue, the EXEC SQL embedded database surface, the EXEC CICS WEB command family for HTTP service, the BMS-flavoured map DSL used to build 3270 panels, the VSAM and temporary-storage queue surfaces, and the catalogue of sample programs shipped under runtime/.

Operational and installation topics — running bricks, editing bricks.cnf, signing on through CSSN, the CEMT master terminal, the bricksload stress tester, and the /metrics endpoint — are documented in the companion **README.md**.

Who should read this publication

This manual is for application programmers who write transactions that run under Bricks. Familiarity with the IBM CICS programming model (pseudo-conversational dispatch, the **EIB**, **COM-MAREA**, BMS maps, KSDS files, temporary-storage queues, the SQLCA, and embedded SQL) is assumed. Where Bricks deviates from the IBM behaviour the difference is documented explicitly.

Conventions used in this publication

The following notational conventions appear throughout the manual.

Symbol	Meaning
UPPERCASE	A keyword. Coded literally.
lowercase	A value supplied by the programmer (a name or expression).
[]	An optional clause.

Symbol	Meaning
{ a b }	A required choice between alternatives.
...	The preceding clause may repeat.

Command syntax is shown in the EXEC CICS ... END-EXEC form. The bare-string form ("VERB OPTIONS" under ADDRESS CICS, REXX only) is described in *Chapter 2. The EXEC CICS command environment*; both forms dispatch identically.

Reference layout

Each command described in this manual follows the same five-section layout, in this order:

1. **Format** — the command syntax as a code block.
2. **Description** — what the command does and any important runtime behaviour.
3. **Options** — every keyword that follows the verb, with its meaning and any value constraints.
4. **Conditions** — the EIBRESP values (or SQLCODE values, for EXEC SQL) the command may return, together with the cause of each.
5. **Example** — a short, runnable fragment illustrating typical use.

Programming Note. Where the same verb behaves differently in COBOL and in REXX, the difference is called out in a *Programming Note* like this one. Where both languages share identical behaviour, the example uses whichever form reads most clearly.

Contents

This publication is organised into nine parts, an appendix series, and a quick-reference card. Read **Part 1** first; the remaining parts are reference material consulted as needed.

Part 1. Bricks programming principles

- Chapter 1. Overview
- Chapter 2. The EXEC CICS command environment
- Chapter 3. The map DSL

Part 2. The COBOL language

- Chapter 20. COBOL source format
- Chapter 21. DATA DIVISION
- Chapter 22. PROCEDURE DIVISION
- Chapter 23. The EIB block in COBOL
- Chapter 24. EXEC CICS in COBOL
- Chapter 25. Copybooks
- Chapter 27. Restrictions and deferred features

Part 3. The REXX language

- Chapter 14. REXX program structure

- Chapter 15. Variables and stems
- Chapter 16. Control flow
- Chapter 17. PARSE templates
- Chapter 18. Conditions and SIGNAL ON
- Chapter 19. Built-in functions

Part 4. EXEC CICS command reference

- Chapter 4. Terminal I/O commands — SEND MAP, RECEIVE MAP, CONVERSE, SEND TEXT, RECEIVE
- Chapter 5. Program control commands — RETURN, XCTL, LINK, ABEND, START, RETRIEVE
- Chapter 6. System services — ASSIGN, ASKTIME, FORMATTIME
- Chapter 10. Recovery and condition handling — SYNCPOINT, SYNCPOINT ROLLBACK, HANDLE CONDITION, IGNORE CONDITION, HANDLE AID, HANDLE ABEND
- Chapter 11. The Execute Interface Block (EIB)
- Chapter 12. Response codes
- Chapter 13. Commands not implemented

Part 5. EXEC CICS file and queue commands

VSAM-style KSDS files and temporary-storage / transient-data queues.

- Chapter 7. KSDS file commands — READ, WRITE, REWRITE, DELETE
- Chapter 8. KSDS browse commands — STARTBR, READNEXT, READPREV, RESETBR, ENDBR
- Chapter 9. Temporary storage and transient data commands — READQ TS/TD, WRITEQ TS/TD, DELETEQ TS/TD, the tmp_dir sandbox

Part 6. EXEC SQL command reference

- Chapter 26. Embedded SQL (COBOL and REXX) — SELECT INTO, INSERT, UPDATE, DELETE, COMMIT, ROLLBACK, CONNECT TO, cursors (DECLARE / OPEN / FETCH / CLOSE), WHENEVER, null indicators, the SQLCA, the SQLCODE catalogue, SYNCPOINT integration.

Part 7. EXEC CICS WEB command reference

- EXEC CICS WEB — server side
- EXEC CICS WEB — client side
- EXEC CICS DOCUMENT — chunked body builder
- EXEC CICS WEB — Phase 3b additions

Part 8. Sample programs

- Chapter 28. Pre-installed sample transactions
- Chapter 29. Worked examples

Appendixes

- Appendix A. Adapting to terminal size (mod 2 vs mod 4)
- Appendix B. Pitfalls and idioms
- Appendix C. Quick command reference card

Note on physical order. The chapter numbers in this contents list are the canonical numbers used throughout the publication. Because some parts have been re-ordered into the logical grouping above, a chapter you find on Part 4 of the contents may sit physically later in the file than a chapter listed in Part 5. Cross-references everywhere in the manual use the chapter number, not the file position.

Part 1. The bricks programming model

Chapter 1. Overview

A bricks **transaction** is a 4-character TRANSID listed in `runtime/transactions.conf`. Each transaction names a **program** — either a REXX source file in `runtime/rexx/` or a COBOL source file in `runtime/cobol/` — and an optional access-control list:

```
HEL0:rexx:hello.rexx
HELP:rexx:help.rexx:public
QAGE:rexx:qage.rexx:public,users,admin
HELC:cobol:hello.cob:public
GUST:cobol:gust.cob:public
```

A program that runs EXEC SQL against something other than the **first** database in `runtime/databases.conf` adds an explicit binding as the 5th colon-separated field:

```
BANK:cobol:bank.cob:public,users,admin:bank
```

See *Database binding* in Chapter 26 for the full contract; the short version is that an absent 5th field defaults to the first row of `databases.conf` and a misbinding manifests as `SQLCODE = -204` (relation does not exist) on every dispatch.

Organising programs into sub-directories

Once a deployment grows past a handful of programs, the flat `runtime/rexx/` / `runtime/cobol/` layout becomes hard to scan. You can group programs into sub-directories at any depth and reference them with a relative path from the language root:

```
runtime/cobol/billing/invoice.cob
runtime/cobol/billing/statement.cob
runtime/cobol/cards/visa/authz.cob
runtime/rexx/admin/userprov.rexx

INVC:cobol:billing/invoice.cob:public
STMT:cobol:billing/statement.cob:public
AUTH:cobol:cards/visa/authz.cob:admin
USRP:rexx:admin/userprov.rexx:admin
```

Forward slashes work on both POSIX and Windows. CEDA PROGRAM walks the language roots recursively and shows each file with its relative path (so `billing/invoice.cob` appears in the **FILE** column, matching what you write in `transactions.conf`). CEMT INQUIRE TRANSACTION shows the same form in the **PROGRAM** column, eliding a long middle (`billing/.../authz.cob`) so the leading

application and the file basename always stay visible. Hidden entries (dotfile-prefixed, e.g. `.git/`, `.swp` files) are skipped during the catalogue walk; recursion is capped at 8 levels.

CEDA TRANSACTION accepts a relative subdirectory path in the **PROGRAM** form field. `..` traversal is rejected; so are empty path segments (`a//b.rexx`). Absolute paths are accepted unchanged for the rare case of running one-off programs from outside the runtime tree.

brickscompile accepts either a single file or a directory. In directory mode it walks recursively (same depth cap + hidden-skip rules as CEDA PROGRAM) and reports per-file pass/fail; exit code is 0 only if every file parses cleanly.

Program caching

The first time a TRANSID is dispatched, its program is parsed and the AST is cached (see *Parsed-program cache* in the README); subsequent dispatches skip the parse. Each running task has its own heap and stack — there is no shared mutable state between concurrently running tasks of the same TRANSID.

Programs interact with three layers:

1. **The 3270 terminal**, through EXEC CICS SEND MAP / RECEIVE MAP (and the unstructured SEND TEXT / RECEIVE pairing). Maps are defined in the BMS-style DSL described in Chapter 3.
2. **Persistent data**, through KSDS file commands (Chapter 7) and TS-queue commands (Chapter 9). Both are backed by an embedded bbolt B+tree at `data/files.boltdb`.
3. **Other programs**, through LINK (synchronous, with COMMAREA), XCTL (transfer of control, no return), and RETURN TRANSID (pseudo-conversational chaining). The Execute Interface Block (EIB) exposes session and request state (Chapter 11).

REXX and COBOL share **the same EXEC CICS dispatcher** (`cics/handler.go`). Every command documented in Part 2 is therefore available, with identical syntax and identical semantics, from either language.

A typical pseudo-conversational task has the shape:

ADDRESS CICS

```
EXEC CICS RECEIVE MAP('CUST1')           END-EXEC  /* read prior screen */
... validate / compute ...
EXEC CICS SEND    MAP('CUST1') FROM(SCR.) ERASE END-EXEC
EXEC CICS RETURN  TRANSID('CUST')        END-EXEC  /* chain back to self */
```

The dispatcher invokes the program once per ENTER. State that must survive between invocations rides in the COMMAREA on RETURN; the chained task receives it through EIBCALEN and the DFHCOMMAREA data area.

Chapter 2. The EXEC CICS command environment

Surface forms

Bricks accepts two equivalent surface forms inside an ADDRESS CICS scope.

1. IBM-canonical EXEC CICS ... END-EXEC — the form CICS programmers recognize. Available from REXX and COBOL.

ADDRESS CICS

```
EXEC CICS ASSIGN  USERID(USR) TERMID(TRM)          END-EXEC
EXEC CICS SEND    MAP('HEL01') FROM(SCR.) ERASE     END-EXEC
EXEC CICS RETURN                                     END-EXEC
```

In REXX, a small preprocessor (`rexx/preprocess.go`, called from `rexx.Parse`) rewrites each EXEC CICS ... END-EXEC block into the equivalent quoted-string command before lexing. Multi-line bodies are collapsed into a single command string; trailing newlines preserve source line numbers in error messages. Comments and string literals are left alone.

In COBOL, the parser collects every token between EXEC CICS and END-EXEC and reconstructs the body verbatim for the same `cics.ParseCommand` REXX uses.

2. Bare string under ADDRESS CICS — terser; available **only in REXX**.

ADDRESS CICS

```
"ASSIGN USERID(USR) TERMID(TRM)"
"SEND MAP('HEL01') FROM(SCR.) ERASE"
"RETURN"
```

Both forms route through `cics.ParseCommand` and dispatch to the same handler. After every command the handler writes EIBRESP, EIBRESP2, and RC into the program's frame.

Argument-passing rules

Inside the parentheses of an option:

- A bare identifier is treated as a **variable name**. The handler reads the value at runtime, and (where the option is an output) writes the result back into that variable.
- A quoted string literal ('...' or "...") is treated as a **literal value**. The handler does not write to literals; passing a literal to an output-only option is rejected with INVREQ.

This distinction matters most for length and key fields: `LENGTH(LEN)` both reads the requested length on input and writes the actual length back; `LENGTH(80)` only sets the input length.

Programming model summary

Concept	Bricks implementation
Task	One invocation of one TRANSID (<code>session.TxCB</code>).
Program	A parsed REXX or COBOL source file, cached at L1/L2.
Pseudo-conversational chain	EXEC CICS RETURN TRANSID(t) COMMAREA(d).
Conversational	Loop in the program; each RECEIVE MAP blocks on input.
Inter-program call	LINK PROGRAM(name) COMMAREA(var) (synchronous).

Concept	Bricks implementation
Storage shared across tasks	KSDS files, TS queues.
Storage shared within one task	REXX variables / COBOL WORKING-STORAGE only.
Implicit task end	Program runs off the end, or <code>STOP RUN</code> (COBOL).
Forced task end	<code>EXEC CICS ABEND.</code>

Response-code handling (**RESP**, **RESP2**, **DFHRESP**)

Every `EXEC CICS` verb accepts the IBM-canonical `RESP(var)` and `RESP2(var)` clauses. After dispatch the runtime writes the numeric response into the named host variables, in addition to the always-set `EIBRESP` / `EIBRESP2` frame fields. Programs may therefore write either style:

```
EXEC CICS READ FILE('CUST') INTO(REC) RIDFLD(KEY)
                                RESP(WS-RC) END-EXEC.
IF WS-RC = DFHRESP(NORMAL)    PERFORM USE-REC.
IF WS-RC = DFHRESP(NOTFND)    PERFORM CREATE-REC.

EXEC CICS WRITE FILE('LOG') FROM(BUF) RIDFLD(KEY) END-EXEC.
IF EIBRESP = DFHRESP(DUPREC) PERFORM HANDLE-DUP.
```

`DFHRESP(NAME)` is a compile-time function: the COBOL parser resolves it to the numeric constant from `runtime/cobolcopy/DFHRESP.cpy` (and from the matching table in `cobol/parser.go`). An unknown name is a parse-time error with a “see `DFHRESP.cpy` for the list” hint.

Length as an input bound

`LENGTH(n)` on `READQ TS`, `RECEIVE INTO`, `LINK COMMAREA`, and `RETURN COMMAREA` is treated as IBM’s in/out parameter — the caller’s value caps the bytes returned (or passed forward, for `LINK/RETURN`); the post-dispatch write-back stores the actual byte count. A `LENGTH(80)` on `READQ TS` against a 200-byte item now hands the program 80 bytes and reports `80`, matching real CICS instead of overflowing the buffer.

On `READ FILE`, `READNEXT`, and `READPREV` (file-record family), `LENGTH` is `OUTPUT`-only — the store decides record size; there is no operator-supplied truncation bound. This is a bricks-defensible simplification for the KSDS-fixed-length model; real VSAM variable-length permits `LENGTH-as-max-buffer`. The write-back-then- read-as-input pattern (call 1 writes `len(rec1)` into `LEN`, call 2 re-reads `LEN` as a cap) would silently corrupt subsequent reads, so bricks ignores the input value entirely on these verbs.

Chapter 3. The map DSL

Bricks ships its own line-oriented, BMS-flavoured map DSL (parsed by `mapdsl/`). Maps live as `*.map` files in the directory configured by `maps_dir` (default `runtime/map/`). Each file contains one or more `MAP ... ENDMAP` blocks. Comments start with `*`.

Format

```
[MAPSET <name>]                                -- optional, namespace for the file
MAP <name> SIZE rowsxcols
  [FIELD AT r,c LEN n <attrs> "literal"]
  [INPUT <fieldname> AT r,c LEN n <attrs> [DEFAULT "value"]]
  [STOP AT r,c]
  [CURSOR AT {fieldname | r,c}]
ENDMAP
[MAP <name> SIZE rowsxcols                      -- multiple MAP blocks per file
...
ENDMAP]
```

Statements

MAPSET <name> (*optional, must be the first non-comment line*) Names the mapset — the namespace every MAP block in this file belongs to. The catalog uses **MAPSET.NAME** as its key so two different files can declare a map named **MAPA** without colliding. When the line is omitted, the mapset defaults to the file's basename (uppercased, extension stripped): a file named **help1.map** has implicit mapset **HELP1**.

MAP <name> SIZE rowsxcols The map header. Names must be unique **within their mapset** (case-insensitive); typical sizes are **24x80** (mod 2) and **43x80** (mod 4). A single **.map** file may declare any number of MAP blocks back-to-back — useful when porting a BMS mapset that contained several DFHMDI macros under one DFHMSD.

FIELD AT r,c LEN n <attrs> "literal" A display-only field that paints the literal at row *r*, column *c*, length *n*. Useful for labels, headings, and panel chrome.

INPUT <fieldname> AT r,c LEN n <attrs> [DEFAULT "value"] A named input field. The name is how **SEND MAP FROM(STEM.)** and **RECEIVE MAP INTO(STEM.)** route data to / from this position (**STEM.<fieldname>**). Use **PROT** to make the field write-protected but still named — useful for output values painted by **SEND MAP FROM(STEM.)**.

STOP AT r,c An autoskip stop attribute that ends the preceding input field.

CURSOR AT <fieldname> or CURSOR AT r,c The home position for the cursor when the map is sent. The named form resolves to (**field.Row**, **field.Col + 1**) — i.e. one byte to the right of the leading 3270 attribute byte, which is the writable cell. Forward references are allowed; names are resolved after the whole map is parsed. When omitted, the renderer falls back to the first input field.

ENDMAP Terminates the map.

Converting legacy BMS sources

The **bricksconvert** CLI utility (under **cmd/bricksconvert**) converts IBM CICS **BMS map source** (**DFHMSD** / **DFHMDI** / **DFHMDF** macros) into this DSL — the recommended path for porting an existing CICS application's screens onto bricks without rewriting every panel by hand. See the BMS conversion section in the README for usage details.

Attributes

PROT, UNPROT, BRIGHT, DIM, UNDERLINE, HIDDEN, NUMERIC, MDT, BLINK, REVERSE, COLOR=BLUE|RED|PINK|GREEN|TURQUOISE|YELLOW

UNDERLINE is the canonical IBM BMS spelling (HIGHLIGHT=UNDERLINE). UNDERSCORE is accepted as a legacy alias for hand-written bricks DSL maps that already use it; new maps should prefer UNDERLINE.

Catalogue lifecycle

mapdsl.NewCatalog(dir) parses every *.map file at startup and self-refreshes on subsequent edits. Each Lookup(name) stats the directory plus the source file backing the requested name; on mtime change the directory is reparsed and swapped atomically. A failed re-parse keeps the prior catalogue in place — the operator can find the broken file with CEMT PERFORM RESCAN MAP (see the README).

Sending a map by mapset

EXEC CICS SEND MAP('NAME') MAPSET('SET') FROM(stem) resolves through the qualified MAPSET.NAME index, so:

```
EXEC CICS SEND MAP('MAPA') MAPSET('AM01') FROM(AM01) ERASE END-EXEC.
EXEC CICS SEND MAP('MAPA') MAPSET('AM16') FROM(AP16) ERASE END-EXEC.
EXEC CICS SEND MAP('MAPB') MAPSET('AM99') FROM(AM99) ERASE END-EXEC.
```

each picks the right map even when several files share the same bare map name. When MAPSET(...) is omitted, the catalog tries a bare-name lookup; if that name appears in more than one mapset the runtime returns the short diagnostic SEND MAP: map "MAPA" is ambiguous; specify MAPSET rather than silently picking one. Programs that only ever reference unique bare names (the typical case for the runtime/map shipped with bricks) keep working without any code change.

RECEIVE MAP accepts the same MAPSET(...) clause with identical resolution semantics.

Example

```
* CICS sign-on screen
MAP CSSN SIZE 24x80
  FIELD AT 1,28 LEN 24 PROT BRIGHT COLOR=TURQUOISE  "BRICKS SIGN-ON"
  FIELD AT 6,15 LEN 9  PROT                                "Userid:"
  INPUT USERID  AT 6,25 LEN 8 UNDERSCORE COLOR=GREEN
  STOP          AT 6,34
  FIELD AT 8,15 LEN 9  PROT                                "Password:"
  INPUT PASSWORD AT 8,25 LEN 8 HIDDEN UNDERSCORE COLOR=RED
  STOP          AT 8,34
  CURSOR        AT USERID
ENDMAP
```

Part 4. EXEC CICS command reference

Part 4 begins here. This part documents the core non-file, non-SQL, non-WEB EXEC CICS verbs. File and queue commands are in **Part 5**; EXEC SQL is in **Part 6**; EXEC CICS WEB is in **Part 7**.

This part documents every EXEC CICS command bricks implements. Each command page follows the same layout: **Format**, **Description**, **Options**, **Conditions**, **Example**.

Commands are grouped by function:

- Chapter 4. Terminal I/O
- Chapter 5. Program control
- Chapter 6. System services
- Chapter 7. KSDS files
- Chapter 8. KSDS browse
- Chapter 9. Temporary storage

Cross-cutting reference:

- Chapter 10. Recovery and condition handling
- Chapter 11. The EIB
- Chapter 12. Response codes
- Chapter 13. Commands not implemented

Chapter 4. Terminal I/O commands

SEND MAP

Send a BMS-style map to the 3270 terminal and wait for the operator to press an AID key.

Format

```
EXEC CICS SEND MAP(name)
                [MAPSET(set)] [FROM(stem.)]
                [ERASE | ERASEAUP | DATAONLY] [MAPONLY]
                [CURSOR(position)]
                [FREEKB] [ALARM] [FRSET]
                [RESP(var)] [RESP2(var)]
END-EXEC
```

Description Loads the named map from the catalogue, populates each named field with the value of <stem>.<fieldname> (when FROM is supplied), paints the screen, and waits for the operator to press an AID key (ENTER, PF_n, PA_n, CLEAR). On return, the captured response is stored on the TCB so a subsequent RECEIVE MAP can pull modified field values back into the program. EIBAID and EIBCP0SN are updated with the AID character and 1-based cursor position.

IBM-canonical clauses honoured: MAPSET(set) qualifies the lookup to disambiguate same-named maps across mapsets (see Chapter 3); ERASEAUP wipes unprotected fields only (approximated as ERASE in bricks); MAPONLY paints just the map's literals + INPUT defaults, ignoring any FROM(stem.).

FREEKB, ALARM, and FRSET are accepted for source compatibility — FREEKB and FRSET are subsumed by the fixed WCC the go3270 library writes; ALARM is parsed but not yet tunnelled to the wire.

If FROM is omitted, the map renders using its INPUT DEFAULT values. If FROM is a literal string, every named field on the map is filled with that literal (rare; supported for parity with BMS).

Three paint modes

Flag	Field set	Blocking?	When to use
ERASE	Every field in the map	Yes — waits for AID	Initial paint, full refresh.
(neither)	Only fields the program populated in FROM(stem)	Yes — waits for AID	In-place update inside a conversational loop (rare; usually ERASE is fine).
DATAONLY	Only fields the program populated in FROM(stem)	No — returns immediately	Background refresh while a different task is mid-input.

ERASE clears the screen first and emits **every** field in the map (with either the program’s FROM value or the map default). Without ERASE, bricks does a **partial repaint**: only fields the program explicitly populated in FROM(stem) are sent; fields the program didn’t touch stay on the terminal exactly as the operator left them. Use ERASE for the initial paint of a screen or a full refresh; omit it for in-place partial updates.

DATAONLY — background partial paint DATAONLY is the bricks idiom for **background, fire-and-forget partial paint**, used to refresh part of the screen — typically a clock or other status fields — while a different task on the same terminal is blocking inside SEND MAP waiting for the operator’s next keystroke. It layers two things on top of the plain no-ERASE partial-paint behaviour:

1. **No-block:** DATAONLY makes SEND MAP return immediately after writing the screen, without entering the AID-wait. The call is fire-and-forget — the operator’s next AID is captured by whatever other SEND MAP or RECEIVE MAP happens to be blocking for it.
2. **Scheduler-safe:** when an EXEC CICS START INTERVAL(...) timer fires on a terminal whose current task is in input-wait, the bricks scheduler dispatches the queued task **inline** in the timer goroutine instead of poisoning the conn read deadline (which would abend the in-flight task with an i/o timeout). The inline-dispatched task is restricted to write-only verbs: SEND MAP DATAONLY, file I/O, TS queue, ASSIGN, START. Any blocking verb (RECEIVE MAP, SEND MAP without DATAONLY, CONVERSE) returns INVREQ in that context — the scheduler goroutine must not block on conn.Read.

This together lets a transaction self-schedule a periodic refresh via:

```
EXEC CICS START TRANSID('CHAT') INTERVAL(000002) FROM('TICK') END-EXEC
EXEC CICS RETURN END-EXEC
```

When the START fires, the same transaction is dispatched a second time. The dispatched copy detects “I am a tick” by checking the RETRIEVE payload, paints only the fields it wants to refresh (SEND MAP ... DATAONLY — the operator’s input field is not in the populated set, so it’s never

repainted), schedules the next tick, and returns. The main task on the same terminal stays blocked in its outer `SEND MAP` throughout. See `runtime/rexx/chat.rexx` for the worked example.

DATAONLY and **ERASE** are mutually exclusive — the runtime rejects the combination at parse time. Note: bricks does not implement the real-CICS **DATAONLY** attribute-byte semantics (application-supplied attribute bytes via `X'00'` sentinels); programs that need a synchronous partial repaint use the plain no-**ERASE** form, and programs that need a background refresh use **DATAONLY** with the **START-INTERVAL** pattern above.

Options **MAP(name)** — *required* Name of a map in the catalogue (case-insensitive). **MAPFAIL** if absent.

FROM(stem.) Stem variable whose tails name the map’s input fields. The stem’s trailing dot is optional: **FROM(SCR)** and **FROM(SCR.)** are equivalent.

ERASE Clears the screen before painting AND emits every field in the map. Without this flag, only fields the program populated in **FROM(stem)** are sent — fields the program didn’t touch stay on the terminal unchanged (partial repaint).

DATAONLY Fire-and-forget partial paint: **SEND** returns immediately without waiting for an AID. Implies no-**ERASE** partial-repaint semantics. Mutually exclusive with **ERASE**.

CURSOR(position) 1-based cursor position. Overrides the map’s **CURSOR AT** clause.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Map sent, AID received.
MAPFAIL	36	Named map not found in the catalogue.
IOERR	17	The terminal write failed.

Example

```
SCR.GREETING = 'Hello, ' || USR
EXEC CICS SEND MAP('HEL01') FROM(SCR.) ERASE END-EXEC
```

Runtime field attribute overrides A map’s **COLOR=** clause is the *default* colour for each field. Programs can override the colour of any named field at runtime by setting a sibling element named `<FIELD>-C` in COBOL or `<stem>.<FIELD>-C` in REXX, before the **SEND MAP**. The override wins; an empty / unset value means “use map default.”

COBOL

```
COPY DFHCOLOR.
01 SCR.
   05 STATUS    PIC X(10).
   05 STATUS-C  PIC X(9) VALUE SPACES.
...
IF BALANCE < 0
   MOVE DFHRED  TO STATUS-C
```

```
ELSE
    MOVE DFHGREEN TO STATUS-C
END-IF.
EXEC CICS SEND MAP('BALC') FROM(SCR) ERASE END-EXEC.
```

REXX

```
if balance < 0 then scr.status_c = 'RED'
    else scr.status_c = 'GREEN'
'EXEC CICS SEND MAP(BALC) FROM(SCR.) ERASE'
```

Accepted colour mnemonics: BLUE RED PINK GREEN TURQUOISE YELLOW NEUTRAL DEFAULT (also the DFH-prefixed canonical names from DFHCOLOR.cpy: DFHBLUE DFHRED DFHPINK DFHGREEN DFHTURQ DFHYELLO DFHNEUTR DFHDFT). Unknown strings degrade to the device default.

REXX symbol caveat. REXX evaluates stem tails by value, not literal: if your program has assigned a bare variable STATUS_C anywhere, then `scr.status_c = 'RED'` binds to the value of STATUS_C, not the literal tail STATUS_C. Defensive form is `scr.'STATUS_C' = 'RED'` (quoted tail).

Highlight (-H) and attribute byte (-A) overrides follow the same convention and are planned for a follow-up release.

See `runtime/cobol/balc.cob` and `runtime/rexx/balc.rexx` for complete worked examples.

RECEIVE MAP

Retrieve the operator's input from the most recent **SEND MAP**.

Format

```
EXEC CICS RECEIVE MAP(name)
    [INTO(stem.)]
END-EXEC
```

Description Pulls the response stored by the most recent **SEND MAP** into a stem, one tail per named field on the map (<stem>.<fieldname>). When **INTO** is omitted, the default stem is **MAP..**

A **RECEIVE MAP** with no matching prior **SEND MAP** returns **MAPFAIL**.

Options **MAP(name)** — *required* The same map name passed to the prior **SEND MAP**.

INTO(stem.) Destination stem. Default **MAP..**

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Fields retrieved.
MAPFAIL	36	No prior matching SEND MAP , or named map not found.

Example

```
EXEC CICS RECEIVE MAP('CUST1') INTO(IN.) END-EXEC
ACTION = IN.ACTION
CKEY   = IN.CUSTNO
```

CONVERSE

A single verb that fuses `SEND MAP` and `RECEIVE MAP` against the same map. Semantically identical to issuing the two halves back-to-back; provided because real-world CICS programs written between the 1970s and the 1990s used `CONVERSE` heavily, and porting them shouldn't require a `SEND/RECEIVE` rewrite.

Format `CONVERSE` has two IBM-canonical forms — the 3270 mapped form and the raw-text TS form. Bricks supports both.

Mapped form (3270):

```
EXEC CICS CONVERSE MAP(name) [MAPSET(set)]
                        [FROM(area) | FROMMAP(area)]
                        [INTO(area)]
                        [ERASE] [CURSOR(pos)]
                        [RESP(var)] [RESP2(var)]
END-EXEC
```

Text form (TS):

```
EXEC CICS CONVERSE FROM(area) [FROMLENGTH(n)]
                        INTO(target) [TOLLENGTH(var) | MAXLENGTH(n)]
                        [STRFIELD] [DEFRESP]
                        [RESP(var)] [RESP2(var)]
END-EXEC
```

In the mapped form: `FROM(area)` and `FROMMAP(area)` are accepted spellings for the outbound stem; either works, both feed the `SEND` half. `INTO(area)` is the inbound stem; it feeds the `RECEIVE` half. `MAPSET`, `ERASE`, `CURSOR` pass through to both halves.

In the text form (no `MAP`): bricks dispatches `SEND TEXT` then `RECEIVE INTO`. `FROMLENGTH(n)` caps the outbound bytes; `TOLLENGTH(var)` / `MAXLENGTH(n)` cap the inbound buffer. `STRFIELD` (structured-field stream) and `DEFRESP` (definite- response) are accepted silently — bricks doesn't tunnel either wire bit today but program source ports cleanly.

Behaviour `CONVERSE` issues `SEND MAP` first, then `RECEIVE MAP` against the same `MAP(name)`. If the `SEND` half returns anything other than `RESP-NORMAL`, the `RECEIVE` half is **skipped** and the `SEND`'s response code is returned — matches real CICS semantics (no screen ever made it to the terminal, so there's no operator response to wait for).

Example

```

EXEC CICS CONVERSE MAP('TIM1') FROM(SCR) INTO(SCR)
                ERASE
END-EXEC.
IF EIBAID = PF03 THEN
    EXEC CICS RETURN END-EXEC
END-IF.

```

This replaces the longer pair:

```

EXEC CICS SEND MAP('TIM1') FROM(SCR) ERASE END-EXEC.
EXEC CICS RECEIVE MAP('TIM1') INTO(SCR)      END-EXEC.

```

Live example: `runtime/cobol/timc.cob` uses `CONVERSE` for both the input screen and the reminder screen.

SEND TEXT

Free-form text output without a BMS map.

Format

```

EXEC CICS SEND TEXT FROM(area)
                [LENGTH(n)]
                [ERASE]
END-EXEC

```

Description Real-CICS `SEND TEXT`. The body is treated as a flat row-major buffer: every `cols` bytes (default 80) lands on the next row, starting at row 0. **3270 has no LF/CR**, so programs must pad each logical line to the column width and concatenate (REXX `LEFT(s,80)`, COBOL `PIC X(80)` group children).

`LENGTH(n)` truncates the body to `n` bytes; bytes past `rows*cols` are dropped. `ERASE` clears the screen before painting; without it, existing fields stay behind. Like `SEND MAP`, the call paints synchronously and waits for an AID, so the operator has time to read the screen before `RETURN`.

Works identically from REXX and COBOL.

Options `FROM(area)` — *required* The buffer to paint. Can be a REXX variable or a COBOL group item.

LENGTH(n) Truncate the body to this length.

ERASE Clear the screen first.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Body painted, AID received.
INVREQ	16	<code>FROM</code> missing.
IOERR	17	The terminal write failed.

Condition	EIBRESP	Cause
-----------	---------	-------

Example See Chapter 29. Worked examples, the GETC program, for the canonical multi-row SEND TEXT pattern.

RECEIVE

Retrieve the unedited terminal line typed by the operator at the blank prompt — typically used to pick up command-line arguments.

Format

```
EXEC CICS RECEIVE INTO(buffer)
                        [LENGTH(len)]
END-EXEC
```

Description Returns the unedited terminal line the operator typed at the blank prompt (TRANSID prefix included), e.g. EXAM 1 2 3. Single-shot per task: a second RECEIVE in the same task, or any RECEIVE in a chained RETURN TRANSID task, returns EOC (RESP=6).

When LENGTH(var) is a bare variable, the actual byte count of the line is written back. The receiving COBOL field's PIC X(n) width handles padding/truncation via standard MOVE semantics; programs detect truncation by comparing len against n.

Options INTO(buffer) — *required* Destination variable for the line.

LENGTH(len) When a bare variable, receives the actual length on return.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Line returned.
EOC	6	No input available (already consumed in this task chain).

Example

```
EXEC CICS RECEIVE INTO(BUF) LENGTH(LEN) END-EXEC  /* "GETC 100" */
PARSE VAR BUF TID CKEY .

EXEC CICS RECEIVE INTO(WS-INPUT) LENGTH(WS-LEN) END-EXEC
UNSTRING WS-INPUT DELIMITED BY ' '
      INTO WS-TID WS-A WS-B WS-C
END-UNSTRING.
```


Chapter 5. Program control commands

RETURN

End the current task; optionally chain to another TRANSID with a COMMAREA.

Format

```
EXEC CICS RETURN [TRANSID(id)]  
                  [COMMAREA(data)]  
END-EXEC
```

Description Sets `tcb.NextTransid` and `tcb.Commarea` and ends the task. With no `TRANSID`, control falls back to the blank prompt. With a `TRANSID`, the dispatcher invokes that `TRANSID` next, with the supplied `COMMAREA` available to the new task as `DFHCOMMAREA` and its length as `EIBCALEN`.

Options `TRANSID(id)` The next `TRANSID` to dispatch.

`COMMAREA(data)` Bytes to flow into the next task.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Task ended.

Example

```
EXEC CICS RETURN TRANSID('CUST') COMMAREA(SAVEAREA) END-EXEC
```

XCTL

Transfer control to another program. The current program does not resume.

Format

```
EXEC CICS XCTL PROGRAM(name)  
              [COMMAREA(data) [LENGTH(n)]]  
              [RESP(var)] [RESP2(var)]  
END-EXEC
```

`LENGTH(n)` on `COMMAREA` is the IBM-canonical input bound — caller’s value caps the bytes passed to the receiving program. `CHANNEL(...)` (the CICS-TS container alternative to `COMMAREA`) is rejected with a short “not supported in bricks (use `COMMAREA`)” message — single-region bricks doesn’t model channels.

Description Transfer of control. The named program runs in place of the current one; the `COMMAREA`, if supplied, becomes its `DFHCOMMAREA`. The current task’s chain continues with the new program; `RETURN TRANSID` semantics carry through unchanged.

Options **PROGRAM(name)** — *required* The target program. Resolved through runtime/transactions.conf.

COMMAREA(data) Bytes to flow to the target.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Control transferred.
PGMIDERR	27	Target program not in transactions.conf.
INVREQ	16	PROGRAM missing.

Example

```
EXEC CICS XCTL PROGRAM('VALIDATE') COMMAREA(SCR) END-EXEC
```

LINK

Synchronously call a sub-program with bidirectional COMMAREA.

Format

```
EXEC CICS LINK PROGRAM(name)
           [COMMAREA(var | 'literal')]
END-EXEC
```

Description The target program runs in a fresh frame with its DFHCOMMAREA preloaded from the caller's COMMAREA(var) (or literal). When the sub-program exits, its final DFHCOMMAREA is written back to the caller's variable.

Caller state — NextTransid, Commarea, LastResponse, LastMapName — is saved before the LINK and restored on return, so the LINK is transparent to the caller's pseudo-conversational context.

The per-transaction ACL is rechecked on the target so a low-privilege caller cannot escalate by linking into an admin program.

REXX and COBOL programs can LINK to each other freely; DFHCOMMAREA is marshalled as opaque bytes.

Options **PROGRAM(name)** — *required* The sub-program to call. Resolved through runtime/transactions.conf.

COMMAREA(var | 'literal') Bidirectional data area. A bare variable is read on entry and written on exit; a literal is read-only.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Sub-program returned cleanly.
PGMIDERR	27	Target not in <code>transactions.conf</code> , sub-program errored, or the caller is not authorised for the target.

Example

```

SAV = ''                                /* working area */
EXEC CICS LINK PROGRAM('CUSV') COMMAREA(SAV) END-EXEC
IF EIBRESP <> 0 THEN SIGNAL ERR
PARSE VAR SAV STATUS '|' MSG

```

ABEND

Abnormally terminate the current task.

Format

```

EXEC CICS ABEND [ABCODE(code)]
                [NODUMP] [CANCEL]
                [RESP(var)] [RESP2(var)]
END-EXEC

```

CANCEL bypasses any installed **HANDLE ABEND** trap — the task terminates rather than recovering. **NODUMP** is accepted silently (bricks has no dump facility).

Description Ends the task with the supplied 4-character abend code (default AAAA). Bricks logs the abend on the operator console and reflects it in the TxCB's status; pseudo-conversational chains are broken (no `NextTransid` is honoured after an **ABEND**).

Options **ABCODE(code)** 4-character code to record. Defaults to AAAA.

NODUMP Accepted for parity with IBM CICS; no dumps are produced anyway.

Conditions **ABEND** does not return; nothing follows it in the program flow.

Example

```

IF EIBRESP <> 0 THEN
  EXEC CICS ABEND ABCODE('FI01') END-EXEC

```

START

Schedule a transaction to fire on this same terminal after a delay (or at a wall-clock time), optionally passing a byte payload the new task picks up via RETRIEVE. This is bricks’s implementation of CICS’s classic deferred-work pattern: a task queues “do X in 60 seconds” and ends; the terminal returns to the blank prompt; when the timer elapses, bricks dispatches X automatically.

Format

```
EXEC CICS START TRANSID(name)
      [INTERVAL(hhmmss) | TIME(hhmmss)]
      [FROM(area) [LENGTH(n)]]
      [TERMID(tttt)]
      [REQID(handle)] [PROTECT]
      [RESP(var)] [RESP2(var)]
END-EXEC
```

REQID(handle) tags the START with a request-id handle. IBM uses this to disambiguate when cancelling or deferring; bricks’s single-region scheduler accepts the option without action so program source ports cleanly. **PROTECT** declares the START as recoverable — bricks already enrolls every START in the task-end SYNCPOINT, so this is silent passthrough. **SYSID**, **QUEUE**, the hh/mm/ss split forms — explicitly rejected with a short “not supported” message.

Options **TRANSID(name)** (*required*) 4-character transaction id, must exist in run-time/transactions.conf. The dispatcher applies the normal per-transaction ACL at fire time, so the operator must still be authorised to invoke name.

INTERVAL(hhmmss) Delay before firing, expressed as up to 6 digits in HHMMSS form (left-padded with zeroes). **INTERVAL(30)** = 30 s; **INTERVAL(000130)** = 1 min 30 s; **INTERVAL(010000)** = 1 h. Omit (or pair with **INTERVAL(0)**) to fire on the next prompt-loop iteration.

TIME(hhmmss) Absolute wall-clock target, also HHMMSS. Respects the operator’s time_zone= setting. If the target is earlier than now, bricks rolls forward to the same time tomorrow (real-CICS semantics). Mutually exclusive with **INTERVAL**.

FROM(area) [LENGTH(n)] Payload bytes the new task will pull back via RETRIEVE. **LENGTH** truncates area to n bytes; if omitted, the entire string value of area is queued. The bytes are copied at START time, so the issuing task may mutate area afterwards without disturbing the queued payload.

TERMID(tttt) Must equal the issuing terminal’s EIBTRMID in this version. Cross-terminal STARTs return RESP=INVREQ with “only same-terminal STARTs are supported.”

Scope and limitations (v1)

- **Same-terminal only.** A START always fires on the issuing TCB. No-TERMID background tasks and TERMID(other) are rejected; both require infrastructure (synthetic TCBs, cross-session authorisation) that’s deferred.
- **In-memory queue.** Pending STARTs do **not** survive a bricks restart. If you need recovery-protected scheduling you’ll have to wait for START PROTECT (out of scope).

- **No REQID, SYSID, QUEUE, PROTECT, HOURS, MINUTES, SECONDS options.** Each returns RESP-INVREQ with the option name in the error string so a port can be triaged.
- **One pending payload per task.** When the START fires, the FROM bytes go into tcb.RetrieveBuf. The new task's first RETRIEVE drains it; a second RETRIEVE returns RESP-ENDDATA (29).

Conditions

- RESP-NORMAL on successful enqueue.
- RESP-INVREQ for missing TRANSID, cross-terminal TERMID, unsupported option, or malformed INTERVAL/TIME.

Example

```
EXEC CICS START TRANSID('TIMC') INTERVAL('000030')
      FROM(MSG)
END-EXEC.
```

Schedules TIMC to fire on this terminal in 30 seconds with the contents of MSG queued as the payload. See runtime/cobol/timc.cob and runtime/rexx/timr.rexx for the worked examples.

RETRIEVE

Pull back the FROM(...) payload of the START that scheduled the *current* task. RETRIEVE is the receiving half of the START / RETRIEVE pair; the first thing a transaction fired by START typically does.

Format

```
EXEC CICS RETRIEVE INTO(var) [LENGTH(n)]
      [RESP(var)] [RESP2(var)]
END-EXEC
```

LENGTH(n) is the IBM in/out parameter — caller's value caps the bytes returned, the post-call value is the actual byte count.

Options INTO(var) (*required*) Variable / DATA area to receive the payload bytes. Must be a name reference, not a string literal.

LENGTH(lenvar) Optional. The variable named here is written with the actual byte length of the payload after a successful read, so the caller can size the consumer code correctly.

Behaviour

- Cold dispatch (operator typed the TRANSID at the prompt) → RESP-ENDDATA (29), nothing written to INTO.
- Scheduled re-entry → RESP-NORMAL, payload bytes copied into INTO, tcb.RetrieveBuf cleared.
- Second RETRIEVE in the same task → RESP-ENDDATA (the buffer is cleared on first read; v1 supports one pending payload per task).

Idiom The canonical START / RETRIEVE program tests EIBRESP first to discover *which dispatch path* it's on, then branches:

```
MOVE SPACES TO BUF.
EXEC CICS RETRIEVE INTO(BUF) END-EXEC.

IF EIBRESP = RESP-NORMAL THEN
    PERFORM SHOW-REMINDER      *> scheduled fire
    EXEC CICS RETURN END-EXEC
END-IF.

*> Cold path: prompt the operator, schedule next fire.
PERFORM PROMPT-AND-SCHEDULE.
```

REXX equivalent in runtime/rexx/timr.rexx; COBOL in runtime/cobol/timc.cob.

Chapter 6. System services

ASSIGN

Read EIB, session, and environment fields into the program.

Format

```
EXEC CICS ASSIGN <FIELD>(target) [<FIELD>(target) ...]
END-EXEC
```

Description Reads one or more system fields into program variables. Multiple options can be combined in a single ASSIGN call. Each <FIELD> is one of the keywords below; **target** is the variable that receives the value.

Options

Option	Returns
USERID(t)	Signed-on userid (empty until CSSN succeeds).
TERMIN(t) / EIBTRMID(t)	Unique 4-digit terminal id (T0001 ...).
EIBAID(t)	Single-byte AID character of the most recent SEND MAP / RECEIVE MAP. Compare with C2X(EIBAID) = 'F3' (REXX) or EIBAID = X'F3' (COBOL) to detect PF3, etc.
EIBCP0SN(t)	1-based cursor position from the most recent map response.
EIBCALEN(t)	Length of DFHCOMMAREA flowed in from the caller.
TWALENG(t) / TCTUALENG(t)	Always 0 (bricks does not allocate a TWA / TCTUA).
SCREENHT(t) / SCREENWD(t)	Negotiated terminal rows / columns.
ALTSCRNHT(t) / ALTSCRNWD(t)	Same values; bricks does not distinguish primary and alternate sizes.

Option	Returns
CONNECTED(t) / CONNTIME(t)	Wall-clock connect timestamp YYYY-MM-DD HH:MM:SS.
TLS(t)	yes when the session is on the TLS listener, no otherwise.
DATE(t)	Today as YYYYMMDD. (<i>bricks-specific</i>)
TIME(t)	Now as HHMMSS. (<i>bricks-specific</i>)
TODAYYR(t) / TODAYMO(t) / TODAYDY(t)	Today's year / month / day individually. (<i>bricks-specific</i>)
DAYCOUNT(t)	Days since 1900-01-01 (the IBM ABSTIME epoch). Subtract two values for an exact day delta. Behaviour change in round 2: previously returned Unix-epoch days; bricks now matches real CICS.
SYSID(t)	Local CICS region ID (BRKS in bricks).
APPLID(t)	Region VTAM applid (BRICKS01 in bricks).
NETID(t)	Network ID of the connected terminal (TCPIP in bricks; placeholder for a future config knob).
ABCODE(t)	Most recent abend code (the value EIBABCODE carries). Empty before any ABEND.
EIBTASKN(t)	Numeric task number — the integer body of TxCB.ID.
STARTCODE(t)	How the task was started. T0 (terminal operator) is the bricks default; future work threads S (started by EXEC CICS START) through the scheduler.

The bricks-specific options exist primarily so the COBOL subset can do date math without REXX-style intrinsic functions; see `runtime/cobol/qagc.cob` for an example that computes age in years from a birthdate.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	All requested fields returned.
INVREQ	16	A field name is unknown, or its target is a literal rather than a variable.

Example

```
EXEC CICS ASSIGN USERID(USR)
                TERMID(TRM)
                SCREENHT(SCRH)
```

SCREENWD(SCRW)

END-EXEC

ASKTIME

Refresh the EIB date/time fields and, optionally, return the current time as a 15-digit absolute timestamp (ABSTIME).

Format

EXEC CICS ASKTIME [ABSTIME(target)]

END-EXEC

Description ASKTIME re-reads the system clock and writes the current date and time into EIBDATE and EIBTIME. Both fields follow the IBM CICS formats:

- EIBDATE — packed-style decimal 0CYYYYY, where C is (century - 19) and YYYY is the two-digit year and Julian day-of-year. For 2026-05-12 (Julian day 132), EIBDATE = 1026132.
- EIBTIME — six-digit HHMMSS.

If ABSTIME(target) is given, target receives a 15-digit decimal string representing milliseconds since 1900-01-01 00:00:00.000. Pass that value to FORMATTIME to break it back into formatted date / time components.

ASKTIME is the only way to get a fresh ABSTIME; the bricks-specific ASSIGN DATE / TIME shortcuts return formatted strings only.

Options ABSTIME(target) Variable that receives the 15-character absolute time. Optional; when omitted, only EIBDATE and EIBTIME are refreshed.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Always (clock reads cannot fail).

Example

```
EXEC CICS ASKTIME ABSTIME(NOW) END-EXEC
EXEC CICS FORMATTIME ABSTIME(NOW)
                YYYYMMDD(TODAY)
                TIME(CLOCK) TIMESEP(':')
END-EXEC
SAY 'Stamped at' TODAY CLOCK
```

FORMATTIME

Decode an ABSTIME value into formatted date and time fields.

Format

```
EXEC CICS FORMATTIME ABSTIME(source)
      [DATE(t)] [DATEFORM(fmt)] [DATESEP(c)]
      [YYYYMMDD(t)] [MMDDYYYY(t)] [DDMMYYYY(t)]
      [MMDDYY(t)] [DDMMYY(t)]
      [YYYYDDD(t)] [YYDDD(t)]
      [TIME(t)] [TIMESEP(c)]
      [TIMEZONE(name)]
      [YEAR(t)] [MONTHOFYEAR(t)] [DAYOFMONTH(t)]
      [DAYOFWEEK(t)] [DAYCOUNT(t)]
      [RESP(var)] [RESP2(var)]
```

END-EXEC

TIMEZONE(name) is the IBM-canonical per-call zone override — accepts IANA names (*America/New_York*, *Europe/Paris*) plus the UTC alias. Unknown zone → **INVREQ** with a short clean message. Without **TIMEZONE**, the handler’s configured location (from the `time_zone` line of `bricks.cnf`) is used.

Description **FORMATTIME** is the IBM-standard companion to **ASKTIME**. Given a 15-digit **ABSTIME** value (typically returned by **ASKTIME ABSTIME(...)**, but any 15-digit decimal millisecond stamp is accepted), it writes the requested representations into the named target variables.

Any combination of output options may be requested in a single call; options that are omitted cost nothing.

Options **ABSTIME(source)** (*required*) The 15-digit decimal absolute time to decode.

DATE(t) with optional **DATEFORM(fmt)** and **DATESEP(c)** Generic date target. **DATEFORM** is one of **YYYYMMDD** (default), **MMDDYYYY**, **DDMMYYYY**, **MMDDYY**, **DDMMYY**. **DATESEP(c)** inserts a one-character separator between components (e.g. '-' → **2026-05-12**); omit **DATESEP** for a packed string with no separators.

YYYYMMDD / **MMDDYYYY** / **DDMMYYYY** / **MMDDYY** / **DDMMYY** Direct date format targets; each respects **DATESEP(c)** if given.

YYYYDDD(t) / **YYDDD(t)** Julian-style date with three-digit day-of-year. Honour **DATESEP**.

TIME(t) with optional **TIMESEP(c)** Six-digit **HMMSS**, or with **TIMESEP(':')** formatted as **HH:MM:SS**.

YEAR(t) — four-digit year (**2026**). **MONTHOFYEAR(t)** — 1..12. **DAYOFMONTH(t)** — 1..31. **DAYOFWEEK(t)** — 0 (Sunday) .. 6 (Saturday), per IBM CICS. **DAYCOUNT(t)** — days since **1900-01-01** (signed, integer).

Conditions

Condition	EIBRESP	Cause
NORMAL	0	All requested fields written.
INVREQ	16	ABSTIME is missing, not 15 decimal digits, or out of range.

Example

```
EXEC CICS ASKTIME ABSTIME(WS-NOW) END-EXEC
EXEC CICS FORMATTIME ABSTIME(WS-NOW)
                DATE(WS-DATE)  DATEFORM('DDMMYYYY')  DATESEP('/')
                TIME(WS-TIME)   TIMESEP(':')
                DAYOFWEEK(WS-DOW)

END-EXEC.
*  WS-DATE → "12/05/2026"      WS-TIME → "10:30:45"      WS-DOW → "2"
```

Part 5. EXEC CICS file and queue commands

Part 5 begins here. This part covers the persistent and ephemeral data services — VSAM-style KSDS files and the temporary- storage / transient-data queue families. Both surfaces are per-task and bound by the same SYNCPOINT mechanism described in Chapter 10.

Chapter 7. KSDS file commands

VSAM in Bricks. Bricks's KSDS file surface is the application- programmer's view of the persistent record store. Each FILE name in EXEC CICS READ FILE(name) corresponds to a B+tree bucket in the embedded `data/files.boltddb` database. A separate DEFINE CLUSTER step is **optional**: the first EXEC CICS WRITE creates the bucket if it isn't already there, and subsequent READ / REWRITE / DELETE / STARTBR / READNEXT / READPREV / RESETBR / ENDBR see it. Records are opaque byte payloads; the application chooses the layout (typically a pipe-delimited group of fields, the convention every sample uses).

Operators who want the classic mainframe lifecycle — declare a cluster with DEFINE CLUSTER, bulk-load it from a sequential file with REPRO, drop it with DELETE, enumerate the catalogue with LISTCAT, dump records with PRINT — should use the **IDCA** transaction described under *VSAM file management* below. Files created through IDCAMS and files auto-created by EXEC CICS WRITE are byte-identical: same bucket, same `_catalog` row, same CEMT INQUIRE FILE view.

These commands operate on a **single record**, identified by a key, in a CICS FILE. Each FILE is a bbolt bucket inside `data/files.boltddb`; record bodies are opaque bytes (the application chooses the layout). See *How file storage works* in the README for the on-disk model.

READ

Read a record from a KSDS by key.

Format

```
EXEC CICS READ FILE(name)
                {INTO(target) | SET(target)}
                RIDFLD(key)
```

```

[UPDATE]
[LENGTH(len)]
END-EXEC

```

Description B+tree exact-key lookup, $O(\log n)$. The record bytes (whatever the application stored) come back into the target. **UPDATE** records a per-session lock on the key, gating a subsequent **REWRITE** on the same **FILE**.

When **LENGTH(var)** is a bare variable, the actual record length is written back.

Options **FILE(name)** — *required* The CICS FILE name.

INTO(target) / **SET(target)** — *one required* Destination for the record body. **INTO** and **SET** are accepted interchangeably here.

RIDFLD(key) — *required* The key to look up.

UPDATE Record a per-session lock so a subsequent **REWRITE** can update this record.

LENGTH(len) Receives the record's actual length when the option is a bare variable.

Bricks deviation from IBM canonical **LENGTH(var)** on **READ FILE** is **OUTPUT-only**, not **input-as-max-** buffer. Real CICS VSAM variable-length reads **LENGTH(var)** as a cap on the bytes returned and writes back the actual size; bricks ignores the input value and only writes back. The write-back-then- read-as-input shape would otherwise corrupt subsequent reads in a loop (**LENGTH(LEN)** on call 1 stores 4; call 2 silently caps a 13-byte record to 4). The bricks KSDS store is fixed-size-per-record so there is no operator-meaningful truncation here.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Record returned.
NOTFND	13	Key not present.
INVREQ	16	Missing required option, or invalid file name.
IOERR	17	Underlying store error.

Example

```

EXEC CICS READ FILE('CUSTOMERS')
        INTO(REC) RIDFLD(CKEY)
END-EXEC
IF EIBRESP = 13 THEN MSG = 'Customer not found'

```

WRITE

Insert a new record into a KSDS.

Format

```
EXEC CICS WRITE FILE(name)
        FROM(data) [LENGTH(n)]
        RIDFLD(key)
        [MASSINSERT]
        [RESP(var)] [RESP2(var)]
END-EXEC
```

LENGTH(n) is the IBM in/out parameter — caller's value caps the bytes written, returned value is the actual bytes stored. **MASSINSERT** is accepted for source compatibility; bricks's KSDS engine is uniformly fast so the flag is a silent hint.

Description B+tree insert. The bucket for the **FILE** is created implicitly on first **WRITE** — there is no **EXEC CICS DEFINE FILE** step. **DUPREC** if a record with the same key already exists.

The write commits in a single bbolt **Update** transaction with **fsync** on success.

Options **FILE(name)** — *required* Target FILE. Created if absent.

FROM(data) — *required* The record bytes to store.

RIDFLD(key) — *required* The key under which to store the record.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Record written.
DUPREC	14	A record with this key already exists.
INVREQ	16	Missing required option, or invalid name.
IOERR	17	Underlying store error.

Example

```
REC = NM || '|' || AD || '|' || CY || '|' || PH
EXEC CICS WRITE FILE('CUSTOMERS') FROM(REC) RIDFLD(CKEY) END-EXEC
```

REWRITE

Replace the record locked by the most recent **READ ... UPDATE**.

Format

```
EXEC CICS REWRITE FILE(name)
        FROM(data)
END-EXEC
```

Description Overwrites the value at the key locked by the most recent READ FILE ... UPDATE on the same FILE. Releases the per-FCB update lock at end of transaction.

Bricks deviation from IBM canonical: READ UPDATE + REWRITE do **not** survive the CECI PF5 boundary. Real conversational CICS programs hold the UPDATE lock across the operator's pause; CECI in bricks treats each PF5 press as an implicit SYNCPOINT, which releases non-HOLD locks. Recommended patterns: type both verbs on one PF5 input (assembled as a single command), or wrap the pair with ENQ resource HOLD ... DEQ resource so the lock persists across the press. Outside CECI (REXX or COBOL programs running as a normal task) the lock survives until the next SYNCPOINT or task end, matching real CICS.

Options FILE(name) — *required* FROM(data) — *required*

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Record updated.
INVREQ	16	No prior READ ... UPDATE, or the lock has been released.
IOERR	17	Underlying store error.

Example

```
EXEC CICS READ FILE('CUSTOMERS') INTO(REC) RIDFLD(CKEY) UPDATE END-EXEC
PARSE VAR REC NM '|' AD '|' CY '|' PH
PH = NEWPHONE                               /* mutate */
REC = NM || '|' || AD || '|' || CY || '|' || PH
EXEC CICS REWRITE FILE('CUSTOMERS') FROM(REC) END-EXEC
```

DELETE

Remove a record from a KSDS.

Format

```
EXEC CICS DELETE FILE(name)
                    [RIDFLD(key) [KEYLENGTH(n) GENERIC [NUMREC(var)]]]
                    [RESP(var)] [RESP2(var)]
END-EXEC
```

Description If RIDFLD is supplied, deletes that key. Otherwise deletes the key locked by the most recent READ ... UPDATE on the same FILE.

IBM-canonical GENERIC delete-by-prefix: with KEYLENGTH(n) GENERIC, every record whose key starts with the first n bytes of RIDFLD is deleted. The count goes back through NUMREC(var) when supplied; RC = NOTFND when no record matches.

Options **FILE(name)** — *required*

RIDFLD(key) Optional. When omitted, the most recent READ-UPDATE key is used.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Record deleted.
NOTFND	13	Key not present.
INVREQ	16	No RIDFLD and no prior READ-UPDATE; or invalid name.
IOERR	17	Underlying store error.

Example

```
EXEC CICS DELETE FILE('CUSTOMERS') RIDFLD(CKEY) END-EXEC
```

VSAM file management — the IDCA (IDCAMS) transaction

IDCA is the BRICKS equivalent of mainframe **IDCAMS** / **AMSERV** — an Access Method Services screen for declaring, loading, dropping, listing, and printing VSAM-style KSDS clusters without writing an application program. It is a built-in transaction (no entry in `run-time/transactions.conf` is required) and is **admin-only**: only signed-on users in the **admin** group see the screen.

Invocation At the > prompt on any 3270 terminal, type:

IDCA

ENTER drops you into a SYSIN / SYSPRINT screen. Type one or more control statements in the SYSIN area, press ENTER to run them, and the SYSPRINT area shows classic **IDC0001I FUNCTION COMPLETED...**-style messages plus the highest condition code. PF5 clears the input; PF7 / PF8 scroll the listing; PF3 exits.

Supported commands

Command	Syntax
DEFINE CLUSTER	DEFINE CLUSTER (NAME(name) [RECORDSIZE(avg max)] [KEYS(len off)] [INDEXED] [REPLACE])
REPRO (load)	REPRO INFILE(seq-file) OUTFILE(cluster) [KEYS(len off)] [SKIP(n)] [COUNT(n)]
REPRO (unload)	REPRO INFILE(cluster) OUTFILE(seq-file) [COUNT(n)]
DELETE	DELETE name [CLUSTER]
LISTCAT	LISTCAT [LEVEL(prefix)] [NAMES \ ALL]
PRINT	PRINT INFILE(cluster) [CHARACTER \ HEX \ DUMP] [COUNT(n)] [FROMKEY(k)] [TOKEY(k)]

Tokenisation is case-insensitive for keywords; values inside parens (file names, keys) are taken verbatim. A trailing `-` on a line is a continuation marker (JCL convention). Lines starting with `*` and text bracketed by `/* ... */` are comments.

Sequential file convention `REPRO INFILE(...)` and `REPRO OUTFILE(...)` name files **relative to `runtime_dir`** (`./runtime/` by default — see the `runtime_dir` knob in `bricks.cnf`). Subdirectories are allowed, so an operator can keep loads under `loads/`, dumps under `dumps/`, ad-hoc files under `tmp/`, etc.:

```
REPRO INFILE(tmp/orders.in)          OUTFILE(ORDERS)          KEYS(7 0)
REPRO INFILE(loads/jan/customers.in) OUTFILE(CUSTOMERS)      KEYS(7 0)
REPRO INFILE(ORDERS)                 OUTFILE(dumps/orders.dat)
```

Bricks treats a name with a `.` in it as a sequential file and a name without one as a cluster, so subdirectory-bearing paths still pick `LOAD` vs `UNLOAD` direction correctly as long as the seq side has an extension. Each line of a sequential file becomes one record. The `KEYS(len off)` clause carves the key out of those bytes (length first, then offset — matching real IDCAMS); omit it and Bricks defaults to the cluster’s declared `KeyMax` from `DEFINE CLUSTER`.

Parent directories are not auto-created. Writing to `OUTFILE(dumps/cust.dat)` when `runtime/dumps/` doesn’t exist fails cleanly — `mkdir -p runtime/dumps` on the host first.

Security Operator-supplied paths are sandboxed to `runtime_dir`. The resolver applies four independent traversal-defence layers before any file is opened: a per-segment lexical filter (alphanumeric + `_-`, only, rejects `..`, rejects leading-dot, rejects absolute paths and backslashes), a `filepath.Clean` round-trip check, an absolute-prefix invariant against `runtime_dir`, and a `filepath.EvalSymlinks` check that rejects symlinks whose target leaves the sandbox. Attempts like `INFILE(..etc/passwd)`, `INFILE(/etc/passwd)`, or `INFILE(symlink)` where the symlink points outside the root all return `IDC3009I` with condition code 8 and never reach `os.Open`. The IDCA transaction is also administrated at the screen level; the path sandbox is defence in depth on top of that.

Worked example

```
DEFINE CLUSTER (NAME(CUSTOMERS) RECORDSIZE(80 80) KEYS(7 0))
REPRO INFILE(tmp/customers.in) OUTFILE(CUSTOMERS) KEYS(7 0)
LISTCAT LEVEL(CUST)
PRINT INFILE(CUSTOMERS) CHARACTER COUNT(5)
```

`LISTCAT` produces a CEMT-style fixed-column listing:

```
CATALOG LISTING -- 1 ENTRY(S)
  NAME                RECORDS  KEYMAX  RECMAX  LAST-MOD
  CUSTOMERS              42       7       80  28May26 15:04
IDC0001I FUNCTION COMPLETED, CONDITION CODE IS 0
```

Condition codes Same scale as classic IDCAMS:

Code	Meaning
0	success

Code	Meaning
4	warning (e.g. duplicate records bypassed on REPRO, empty LISTCAT)
8	error (file not found, file locked by another terminal, DEFINE without REPLACE on an existing cluster)
12	severe (parse error, I/O failure)

The highest code any single statement produced is reported on the final `IDC0001I FUNCTION COMPLETED, HIGHEST CONDITION CODE WAS nn` line.

Lock behaviour and concurrency DELETE and REPRO (load mode) check the file's lock state before proceeding. A file held under EXEC CICS READ ... UPDATE by another terminal is reported as `IDC3009I ** name LOCKED BY term=Txxxx, NOT DELETED` with condition code 8 — the same refusal the CEDA VSAM purge screen produces. Locks held by the operator's own terminal are ignored (so a user can DELETE a file they themselves are mid-update on, if they really want to).

Every IDCAMS command produces one line in the audit log (`log/<timestamp>.log`):

```
idcams=DEFINE target=CUSTOMERS term=T0001 user=admin status=OK detail="keymax=7 recmax=80"
idcams=REPRO target=CUSTOMERS term=T0001 user=admin status=OK detail="source=customers.in records=42 skip
idcams=DELETE target=CUSTOMERS term=T0001 user=admin status=OK
```

Relationship to CEDA VSAM CEDA VSAM and IDCA are two views of the same catalogue:

- **CEDA VSAM** — interactive table of every cluster with one-keystroke purge (P selector + secondary type-the-name confirmation). Use when you want to browse and purge a handful of files visually.
- **IDCA** — command-driven AMS. Use when you want to script a reproducible cluster lifecycle, declare metadata up front, or bulk-load from a sequential file.

Both go through the same audit logging and same lock checks; both operate on the same `data/files.boltddb` buckets that EXEC CICS WRITE populates from REXX and COBOL.

Offline IDCAMS — the idcams CLI `idcams` is the **standalone** counterpart to the IDCA transaction. Same engine, same control statements, same audit log format — but it opens `data/files.boltddb` directly from the shell, so the bricks region must be **stopped** while it runs. This is the classic mainframe batch-IDCAMS model: shut the region down, run a deck of DEFINE / REPRO / DELETE statements, bring the region back up.

```
idcams [-c bricks.cnf] [-d datadir] [-r runtimeidir] -sysin deck.idcams
idcams [-c bricks.cnf] -e "DEFINE CLUSTER (NAME(X))"
idcams                                     # reads SYSIN from stdin
```

Flags:

Flag	Purpose
-c <path>	path to bricks.cnf (default: ./bricks.cnf then ../bricks.cnf)
-d <dir>	data directory holding files.boltdb (overrides bricks.cnf)
-r <dir>	runtime_dir sandbox root for REPRO INFILE/OUTFILE (overrides bricks.cnf)
-sysin <file>	read IDCAMS control statements from this file
-e <stmt>	execute one inline statement and exit
-logdir <dir>	audit-log directory (default: bricks.cnf log_location)
-q	suppress the SYSIN echo on stdout (listing only)

Exit code is the highest IDCAMS condition code (0=OK, 4=warning, 8=error, 12=severe), so the CLI plugs into shell pipelines and CI scripts directly. A REPRO that bypassed duplicate records exits 4; a LISTCAT on an empty catalogue exits 4; a DELETE of a missing file exits 8.

Worked example:

```
$ idcams -sysin reload.idcams
idcams - data=data tmp=runtime/tmp sysin=reload.idcams
-----
IDCAMS  SYSTEM SERVICES                                BRICKS / KICKS
                                           TIME: 14:21:08

      DEFINE CLUSTER (NAME(CUSTOMERS) RECORDSIZE(80 80) KEYS(7 0) REPLACE)
IDC0508I CLUSTER CUSTOMERS DEFINED  (KEYMAX=7, RECMAX=80)
IDC0001I FUNCTION COMPLETED, CONDITION CODE IS 0

      REPRO INFILE(customers.in) OUTFILE(CUSTOMERS) KEYS(7 0)
IDC0005I NUMBER OF RECORDS PROCESSED WAS 12450
IDC0001I FUNCTION COMPLETED, CONDITION CODE IS 0

IDC0001I FUNCTION COMPLETED, HIGHEST CONDITION CODE WAS 0
$ echo $?
0
```

If the bricks region is still running, the CLI detects the bbolt lock contention and exits with code 16 plus a hint to use the IDCA transaction instead — no hang, no partial write, no chance of corrupting an in-flight region’s view of the catalogue.

Chapter 8. KSDS browse commands

A **browse** walks a CICS FILE in B+tree key order. The browse runs inside a bbolt MVCC read transaction, so the cursor sees a stable point-in-time snapshot — concurrent **WRITE** / **REWRITE** / **DELETE** on the same FILE do not disturb an in-progress browse.

The browse is **per-task** and tied to one FILE. A program may have multiple browses open on different FILEs at once; a second STARTBR on the same FILE replaces the first (STARTBR is implicitly idempotent in CICS).

The dispatcher releases any cursor the program forgot to ENDBR via a `defer handler.CloseBrowses()` at task end.

Bricks deviation from IBM canonical: browse cursors survive SYNCPOINT. Real CICS closes VSAM RPL browses at every SYNCPOINT; bricks browses are bbolt MVCC snapshots and remain valid until the matching ENDBR (or the dispatcher's task-end defer). Closing them at SYNCPOINT would be theater — the MVCC snapshot is already isolated from concurrent writers. This is what lets a STARTBR on CECI PF5 #1 drive READNEXT on PF5 #2 (each press is a per-PF5 implicit SYNCPOINT).

STARTBR

Open a browse cursor on a KSDS file.

Format

```
EXEC CICS STARTBR FILE(name)
                        [RIDFLD(start)]
                        [GTEQ | EQUAL]
                        [GENERIC]
                        [KEYLENGTH(n)]
                        [REQID(handle)]
                        [RESP(var)] [RESP2(var)]

END-EXEC
```

REQID(handle) is the IBM-canonical concurrent-browse handle: a program may hold several independent browses against the same file by tagging each STARTBR / READNEXT / READPREV / ENDBR / RESETBR with a distinct integer handle. Without REQID the file's single default cursor is used — backward-compatible with every existing bricks program.

Description Opens a B+tree browse cursor on the file. With no RIDFLD, positions on the first key. With RIDFLD and GTEQ (the IBM default and bricks default), positions on the first key start. With EQUAL, requires an exact match (NOTFND if absent). With GENERIC and KEYLENGTH(n), positions on (and walks only through) keys whose first n bytes match the first n bytes of RIDFLD.

Options FILE(name) — *required*

RIDFLD(start) Starting key. Default: first key in the file.

GTEQ | EQUAL Comparison rule. Default GTEQ.

GENERIC Restrict the walk to keys whose prefix matches.

KEYLENGTH(n) With GENERIC, the prefix length to match.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Cursor opened.
NOTFND	13	EQUAL requested but no such key.
INVREQ	16	Missing required option, invalid name, etc.
IOERR	17	Underlying store error.

Example

```
EXEC CICS STARTBR FILE('CUSTOMERS')
      RIDFLD('NY-')
      GENERIC KEYLENGTH(3)
END-EXEC
```

READNEXT

Step the open browse cursor forward by one record.

Format

```
EXEC CICS READNEXT FILE(name)
      {INTO(target) | SET(target)}
      [RIDFLD(keyvar)]
      [LENGTH(len)]
END-EXEC
```

Description Advances the cursor and returns the next record (or the first record if this is the first **READNEXT** after **STARTBR**). When **RIDFLD** is a bare variable, the matching key is written back; when **LENGTH** is a bare variable, the actual record length is written back.

Returns **ENDFILE** past the last key (or past the end of the **GENERIC** prefix). Records that were deleted by a concurrent transaction between **STARTBR** and the read are skipped automatically (bounded forward loop, no goroutine-stack risk).

Options **FILE(name)** — *required*

INTO(target) / **SET(target)** — *one required*

RIDFLD(keyvar) Receives the key of the record returned.

LENGTH(len) Receives the actual record length.

Bricks deviation from IBM canonical **LENGTH(var)** on **READNEXT** (browse-family) is **OUTPUT-only**, not **input-as-max-buffer**. Real CICS reads the caller's **LENGTH(var)** as a cap on the bytes returned and writes back the actual size; bricks ignores the input value and only writes back. The write-back-then- read-as-input shape would otherwise corrupt subsequent reads in a loop (**LENGTH(LEN)** on call 1 stores 4; call 2 caps a 13-byte record to 4). The bricks KSDS store is fixed-size-per-record so there is no operator-meaningful truncation here. A **LENGTH(20)** literal on a 13-byte record returns the full 13 bytes (the literal is silently ignored).

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Record returned.
ENDFILE	20	Past the last key (or out of GENERIC prefix).
INVREQ	16	No STARTBR is open on this FILE .
IOERR	17	Underlying store error.

Example See **STARTBR** example, then:

```
DO FOREVER
  EXEC CICS READNEXT FILE('CUSTOMERS') INTO(REC) RIDFLD(K) END-EXEC
  IF EIBRESP = 20 THEN LEAVE
  SAY K ':' REC
END
```

READPREV

Step the open browse cursor backward by one record.

Format

```
EXEC CICS READPREV FILE(name)
                        {INTO(target) | SET(target)}
                        [RIDFLD(keyvar)]
                        [LENGTH(len)]
END-EXEC
```

Description Same writeback rules as **READNEXT**. Returns **ENDFILE** before the first key (or before the start of the **GENERIC** prefix). Useful for paginating backward through a key range.

Options As for **READNEXT**.

Bricks deviation from IBM canonical Same shape as **READNEXT**: **LENGTH(var)** is **OUTPUT**-only, not **input-as-max-buffer**. The write-back-then-read-as-input pattern would corrupt subsequent reads in a backward walk; bricks ignores the operator-supplied input value and only writes back the actual record size. Real VSAM variable-length permits **LENGTH-as-max-** buffer; the bricks KSDS-fixed model has no operator-meaningful truncation here.

Conditions As for **READNEXT**, with **ENDFILE** meaning *before the first key*.

Example

```
EXEC CICS READPREV FILE('CUSTOMERS') INTO(REC) RIDFLD(K) END-EXEC
```

RESETBR

Reposition the cursor of an open browse without closing the underlying read transaction.

Format

```
EXEC CICS RESETBR FILE(name)
          RIDFLD(start)
          [GTEQ | EQUAL]
          [GENERIC]
          [KEYLENGTH(n)]
END-EXEC
```

Description Cheaper than `ENDBR + STARTBR` when the program wants to jump within the same browse session — the read snapshot is preserved.

Options As for `STARTBR`. `RIDFLD` is required.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Cursor repositioned.
INVREQ	16	No <code>STARTBR</code> is open on this FILE.
NOTFND	13	<code>EQUAL</code> requested but no such key.

Example

```
EXEC CICS RESETBR FILE('CUSTOMERS') RIDFLD('00500') END-EXEC
```

ENDBR

Close an open browse cursor.

Format

```
EXEC CICS ENDBR FILE(name)
END-EXEC
```

Description Releases the cursor and the underlying bbolt read transaction. The dispatcher releases any cursor the program forgot to `ENDBR` at task end, but explicit `ENDBR` is recommended.

Options `FILE(name)` — *required*

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Cursor closed.
INVREQ	16	No STARTBR is open on this FILE .

Example

```
EXEC CICS ENDBR FILE('CUSTOMERS') END-EXEC
```

Chapter 9. Temporary storage and transient data commands

Bricks supports two related queue families:

- **Temporary Storage (TS)** — ordered, persistent sequences of opaque byte items inside the bbolt database. Each queue is a sub-bucket whose keys are 8-byte big-endian item numbers (1, 2, 3 ...) and whose values are the item payloads. Item-addressed; reads and writes are $O(\log N)$. Use for ephemeral state, scratch storage, producer/consumer chaining inside the database.
- **Transient Data extra-partition (TD)** — sequential text files in a sandboxed on-disk directory (`tmp_dir`). Line-oriented; read-once-and-advance / append-only semantics. Use for staging imports from text files into VSAM and exports out of it. The REXX **LINEIN** / **LINEOUT** / **STREAM** family hits the same backend, so a file written by COBOL is readable by REXX and vice-versa.

The encoding contract for TD files is strict: ASCII bytes only (`0x09` TAB, `0x20–0x7E` printable), no EBCDIC, no UTF; lines are terminated by a single LF (`0x0A`); CR (`0x0D`) is rejected on write with **INVREQ**. The sandbox enforces a flat namespace under `tmp_dir` — no sub-directories, no leading dot, no `..`, no slashes in the queue name. See The `tmp_dir` sandbox below for the full rules.

READQ TS

Read an item from a temporary storage queue.

Format

```
EXEC CICS READQ TS QUEUE(name)
                        INTO(target)
                        [ITEM(n) | NEXT]
                        [LENGTH(len)]
                        [NUMITEMS(num)]
```

END-EXEC

QNAME is accepted as a synonym for QUEUE.

Description READQ TS runs in one of two modes:

1. **Cursor mode** — advance the **per-task implicit cursor**: the first cursor-less READQ on a queue returns item 1, the second returns item 2, and so on. The cursor is keyed on the

running TxCB and released when the task ends — a fresh invocation of the same TRANSID starts at item 1 again.

2. **Explicit-item mode** — `ITEM(n)` reads that exact item without touching the cursor. Useful for random-access reads where the program knows the item number in advance.

The mode is selected by IBM-canonical strict semantics. Cursor mode fires when `NEXT` is present OR when `ITEM` is absent. Explicit-item mode fires when `ITEM(n)` is present without `NEXT`; the variable's current value names the item to read and must be a positive integer. The four operator patterns are:

```

READQ TS QUEUE(q) INTO(d)                – cursor mode; item number not returned
READQ TS QUEUE(q) INTO(d) NEXT            – cursor mode; same as above
READQ TS QUEUE(q) INTO(d) NEXT ITEM(N)    – cursor mode; writes item number to N
READQ TS QUEUE(q) INTO(d) ITEM(N)         – explicit mode; N must be a positive integer input

```

Bricks deviation from IBM canonical: the per-task implicit cursor survives `SYNCPPOINT`. Real CICS clears the read cursor on recoverable TSQs at every `SYNCPPOINT`; bricks does not. The bricks design treats `SYNCPPOINT` as a unit-of-work commit boundary that does not touch cursor state, browse cursors, TD handles, WEB sessions, or DOCUMENT tokens. This is what lets a CECI operator walk a queue across PF5 presses (each press is a per-PF5 implicit `SYNCPPOINT`), and what lets a long-running REXX program issue periodic `SYNCPPOINTS` without rewinding its read loop.

Options `QUEUE(name)` / `QNAME(name)` — *required*

`INTO(target)` — *required*

`ITEM(n)` | `NEXT` `NEXT` (or `ITEM` absent) selects cursor mode. `ITEM(n)` without `NEXT` is **strict input**: the variable must hold a positive integer or `INVREQ` fires with a short error pointing the operator at `NEXT`. `NEXT ITEM(var)` is the cursor-loop form — the resolved item number is written back into `var` on success.

`LENGTH(len)` Receives the actual item length.

`NUMITEMS(num)` Receives the queue's current high-water item count.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Item returned.
ITEMERR	26	<code>ITEM(n)</code> out of range, or implicit cursor past the last item.
QIDERR	44	Queue does not exist.
INVREQ	16	Missing required option, invalid name, or <code>ITEM(var)</code> (no <code>NEXT</code>) when <code>var</code> is unset / non-numeric.

Example

```
DO FOREVER
  EXEC CICS READQ TS QUEUE(QNM) INTO(REC) NEXT END-EXEC
  IF EIBRESP = 26 THEN LEAVE          /* end of queue */
  SAY REC
END
```

WRITEQ TS

Append a new item to a queue, or rewrite an existing item.

Format

```
EXEC CICS WRITEQ TS QUEUE(name)
                        FROM(data)
                        [ITEM(n) REWRITE]
                        [NUMITEMS(var)]
                        [MAIN | AUXILIARY]
                        [RESP(var)] [RESP2(var)]
END-EXEC
```

QNAME is accepted as a synonym for **QUEUE**. **NUMITEMS(var)** receives the queue's current item count after the write — IBM canonical; lets programs drive paging without a follow-up **READQ**. **MAIN** and **AUXILIARY** are storage-class hints from real CICS; bricks uses a uniform bbolt-backed store so both are accepted silently.

Description In **append** mode (no **ITEM REWRITE**), the item is added at the next sequence number; an in-memory high-water-mark counter assigns the number without scanning. When **ITEM(var)** is a bare variable, the assigned item number is written back.

In **rewrite** mode (both **ITEM(n)** and **REWRITE** present), item **n** is replaced.

The write commits in a single bbolt transaction so a crash mid-write leaves either the prior state or the new one — never a partial.

Options **QUEUE(name)** / **QNAME(name)** — *required* **FROM(data)** — *required* **ITEM(n) REWRITE** Both required for rewrite mode; either alone is rejected.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Item written.
ITEMERR	26	ITEM(n) REWRITE and item n does not exist.
INVREQ	16	Missing required option, invalid name, or ITEM/REWRITE not paired.

Condition	EIBRESP	Cause
IOERR	17	Underlying store error.

Example

```
EXEC CICS WRITEQ TS QUEUE('AUDIT') FROM(MSG) ITEM(I) END-EXEC
SAY 'Wrote item' I
```

DELETEQ TS

Delete an entire temporary storage queue.

Format

```
EXEC CICS DELETEQ TS QUEUE(name) END-EXEC
```

Description Drops the queue's sub-bucket and resets the in-memory counters and cursors. Subsequent `WRITEQ` will recreate it starting at item 1.

Options `QUEUE(name)` / `QNAME(name)` — *required*

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Queue deleted.
QIDERR	44	Queue does not exist.

Example

```
EXEC CICS DELETEQ TS QUEUE('AUDIT') END-EXEC
```

READQ TD

Read the next line from a sequential text file in `tmp_dir`.

Format

```
EXEC CICS READQ TD QUEUE(name)
                INTO(target)
                [LENGTH(len)]
END-EXEC
```

`QNAME` is accepted as a synonym for `QUEUE`. `name` is a flat filename under `tmp_dir`; sub-directories and traversal are rejected with `INVREQ`.

Description Reads the next LF-terminated line from `tmp_dir/name`, strips the terminating LF, and copies the payload into `target`. The file is auto-opened in read mode on the first call per task and the read cursor advances item-by-item across subsequent calls. A program that `WRITEs` then `READs` the same queue causes the handler to close the write handle and reopen for read; the read cursor restarts at line 1. The handle is closed automatically at task end.

Options `QUEUE(name)` / `QNAME(name)` — *required*

`INTO(target)` — *required*

`LENGTH(len)` Receives the line's byte count (after the LF strip).

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Line returned.
QZERO	12	End-of-file. <code>target</code> is untouched; bricks closes the handle so a later <code>READQ</code> on the same queue rewinds to line 1.
QIDERR	44	File does not exist in <code>tmp_dir</code> .
INVREQ	16	Missing <code>INTO</code> / invalid name / sandbox rejection.
IOERR	17	Underlying file-system error.

Example

```
PERFORM IMPORT-ONE UNTIL DONE-FLAG = 'Y'.
...
IMPORT-ONE.
  MOVE SPACES TO REC.
  EXEC CICS READQ TD QUEUE('orders.sample.txt') INTO(REC) END-EXEC.
  IF EIBRESP = 12 THEN
    MOVE 'Y' TO DONE-FLAG
  END-IF.
  IF EIBRESP = 0 THEN
    PERFORM HANDLE-RECORD
  END-IF.
```

WRITEQ TD

Append a line to a sequential text file in `tmp_dir`.

Format

```
EXEC CICS WRITEQ TD QUEUE(name)
                        FROM(data)
                        [LENGTH(len)]
```

END-EXEC

QNAME is accepted as a synonym for QUEUE.

Description Appends **data** plus a single LF to **tmp_dir/name**. The file is auto-opened in append mode (creating it if absent) on the first call per task. Trailing ASCII spaces on **data** are right-stripped before write — COBOL PIC X(n) values arrive padded to the right and the canonical text-file convention is to strip them. The payload is validated byte-by-byte: anything outside 0x09 (TAB), 0x0A (LF), or 0x20–0x7E (printable ASCII) causes the write to fail with **INVREQ** and a byte-offset diagnostic. CR (0x0D) is explicitly rejected — bricks emits Unix-style LF-only files.

Options **QUEUE(name)** / **QNAME(name)** — *required*

FROM(data) — *required*

LENGTH(len) Accepted for syntactic compatibility; the actual byte count is **LENGTH(data)** after **rstrip**.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Line appended.
INVREQ	16	Missing FROM / invalid name / sandbox rejection / non-ASCII byte / CR in payload.
IOERR	17	Underlying file-system error.

Example

```
DO I = 1 TO REC.0
  EXEC CICS WRITEQ TD QUEUE('export.txt') FROM(REC.I) END-EXEC
END
```

DELETEQ TD

Delete a sequential text file in **tmp_dir**.

Format

```
EXEC CICS DELETEQ TD QUEUE(name) END-EXEC
```

Description Closes any handle the task has open on `name`, then unlinks the file. Already-gone is treated as success (matches the `DELETE FILE` forgiving semantics in bricks). Use to reset an export staging file before a fresh batch.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	File deleted (or already absent).
INVREQ	16	Invalid name / sandbox rejection.
IOERR	17	Underlying file-system error.

Example

```
EXEC CICS DELETEQ TD QUEUE('export.txt') END-EXEC.
```

The `tmp_dir` sandbox

`tmp_dir` is configured in `bricks.cnf`:

```
tmp_dir = runtime/tmp
```

Defaults to `runtime/tmp` (under `runtime_dir`) when the line is omitted. The directory is created at startup if missing.

Name rules. A queue name must match `[A-Za-z0-9._-]{1,255}` — no leading dot, no `..`, no slash, no backslash, no NUL. Bricks also runs `filepath.Rel` against every resolved path as defense-in-depth, so symlink shenanigans bounce too.

Encoding rules. ASCII only — no EBCDIC, no UTF-8, no UTF-16. The write path validates every byte and rejects with `INVREQ` on the first violation, naming the offending offset and hex value. Lines terminate with LF only; CR is rejected on write. The read path splits on LF and preserves any CR bytes it finds verbatim — bricks does **not** silently strip them. If you need to import a file authored on Windows, pre-strip the CRs:

```
tr -d '\r' < windows.csv > runtime/tmp/clean.csv
```

Cross-language interop. Both COBOL (`READQ TD` / `WRITEQ TD`) and REXX (`LINEIN` / `LINEOUT` / `STREAM`) hit the same `*cics.TmpStore` backend. A file produced by one language is readable by the other; the only constraint is that both sides agree on the column format inside each line (canonical bricks convention: pipe-delimited).

Task-end cleanup. Every handle the program opens is closed automatically when the task ends. A program that forgets to call `STREAM CLOSE` or `DELETEQ TD` does not leak descriptors.

Chapter 10. Recovery and condition handling

This chapter covers two related families:

- **Unit-of-work commands** — SYNCPOINT and SYNCPOINT ROLLBACK group multiple file / TS mutations into one atomic step that can be committed or undone.
- **Non-local control commands** — HANDLE CONDITION, IGNORE CONDITION, HANDLE AID, and HANDLE ABEND arm program labels that the dispatcher branches to on a non-NORMAL response, on a particular AID key, or when the task abends. Together with the per-call EIBRESP test (Chapter 12) and REXX SIGNAL ON ERROR (Chapter 18) they cover the full range of CICS error-handling idioms.

How the unit of work works

Each task owns a **journal** of pending undo entries that lives on its TxCB. Every successful WRITE / REWRITE / DELETE and every WRITEQ TS / DELETEQ TS appends an inverse operation to the journal *immediately after* the underlying bbolt write commits.

- SYNCPOINT clears the journal — the work becomes irrevocably committed, and a new unit of work begins.
- SYNCPOINT ROLLBACK walks the journal in reverse and applies each inverse op (re-writing pre-images, re-creating deleted records, restoring queue contents, etc.), then clears the journal.
- RETURN performs an **implicit SYNCPOINT** before the task ends.
- ABEND performs an **implicit SYNCPOINT ROLLBACK** *unless* a HANDLE ABEND exit catches it; the exit may then choose whether to commit or roll back explicitly.

In-flight uncommitted writes are visible to concurrent tasks (this matches IBM CICS under READ NO UPDATE); the rollback model exists to recover *the current task* from its own partially-applied work, not to provide multi-task isolation.

How the condition / AID / abend traps work

After every EXEC CICS command, the dispatcher writes EIBRESP and then consults three optional per-task tables:

Table	Populated by	Consulted	Branches when
Condition map	HANDLE CONDITION / IGNORE CONDITION	After every command	EIBRESP $\neq$ NORMAL and a label is armed for that condition (or for ERROR).
AID map	HANDLE AID	After every command that updates EIBAID (SEND MAP, RECEIVE MAP, RECEIVE)	The new EIBAID matches an armed key (PF1–PF24, PA1–PA3, ENTER, CLEAR, or the catch-all ANYKEY).
Abend exit	HANDLE ABEND	When the task abends	An exit is armed and not cancelled.

When a trap fires, control transfers to the named label as if the program had GO TO'd (COBOL) or SIGNAL'd (REXX) it directly. The trap *stays armed* across the branch — re-arming after each fire is not required (and the IGNORE / bare-name disarm forms exist for when the program wants the trap to stop firing).

SYNCPOINT

Commit the current unit of work and begin a new one.

Format

EXEC CICS SYNCPOINT END-EXEC

Description Clears the task's undo journal. All file / TS writes performed since the last SYNCPOINT (or since the task began) are made permanent; they can no longer be rolled back. A fresh, empty journal is started and subsequent mutations accumulate against it.

SYNCPOINT is implicit on EXEC CICS RETURN — programs that run to completion never need to call it explicitly. The explicit form is useful in long-running tasks that want to checkpoint partial progress, or in programs that mix recoverable phases with phases where rollback is no longer meaningful.

Options None.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Journal cleared.

SYNCPOINT cannot fail in bricks because the underlying bbolt writes have already committed by the time the journal entry was appended.

Example

```
* Phase 1: build the audit row, commit immediately so a later  
* validation failure does not undo the audit trail.
```

```
EXEC CICS WRITEQ TS QUEUE('AUDIT') FROM(WS-LOG) END-EXEC.
```

```
EXEC CICS SYNCPOINT END-EXEC.
```

```
* Phase 2: real business work; rolled back if validation fails.
```

```
EXEC CICS REWRITE FILE('CUSTOMERS') FROM(WS-CUST) END-EXEC.
```

```
IF WS-INVALID
```

```
    EXEC CICS SYNCPOINT ROLLBACK END-EXEC
```

```
    EXEC CICS ABEND ABCODE('VAL1') END-EXEC
```

```
END-IF.
```

SYNCPOINT ROLLBACK

Discard the current unit of work and begin a new one.

Format

EXEC CICS SYNCPOINT ROLLBACK END-EXEC

Description Walks the task's undo journal in reverse and applies each inverse operation:

- WRITE is undone by deleting the new key.
- REWRITE is undone by re-writing the captured pre-image.
- DELETE is undone by re-writing the deleted record.
- WRITEQ TS (append) is undone by deleting the appended item.
- WRITEQ TS ... REWRITE is undone by re-writing the pre-image.
- DELETEQ TS is undone by restoring every prior item from a snapshot taken before the delete.

After the walk the journal is cleared and a new unit of work begins.

ROLLBACK is implicit when EXEC CICS ABEND runs **without** a HANDLE ABEND exit. When an exit *does* intercept the abend, the exit code decides whether to call SYNCPOINT ROLLBACK (typical) or SYNCPOINT (commit partial work and continue).

Options ROLLBACK is the only option — it is what distinguishes this form from plain SYNCPOINT.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	All journal entries successfully reverted.
ROLLEDBACK	82	One or more inverse ops failed (the underlying bbolt error is logged). The journal is still cleared; some writes may remain partially undone.

Example

ADDRESS CICS

```
EXEC CICS WRITE FILE('ACCT') RIDFLD(DEBIT_ID) FROM(DEBIT_REC) END-EXEC
EXEC CICS WRITE FILE('ACCT') RIDFLD(CREDIT_ID) FROM(CREDIT_REC) END-EXEC
```

```
IF DEBIT_TOTAL <> CREDIT_TOTAL THEN DO
  EXEC CICS SYNCPOINT ROLLBACK END-EXEC      /* both writes undone */
  EXEC CICS ABEND ABCODE('IMBL') END-EXEC
END
```

```
EXEC CICS SYNCPOINT END-EXEC                /* commit both writes */
```

HANDLE CONDITION

Arm program labels to receive control on named EXEC CICS conditions.

Format

```
EXEC CICS HANDLE CONDITION cond1[(label)] cond2[(label)] ...  
END-EXEC
```

Description Builds a per-task map from condition name to label. After every EXEC CICS command, when `EIBRESP` $\neq$ `NORMAL`, the dispatcher consults the map:

1. If the *exact* condition name is armed, control branches to the named label.
2. Otherwise, if the catch-all `ERROR` is armed, control branches to that label.
3. Otherwise, the response code is left in `EIBRESP` for the program to test, exactly as if `HANDLE CONDITION` had never been issued.

The `(label)` form arms the trap; the bare condition name (no parens) **disarms** it. Arming a previously-armed condition replaces the old label.

`HANDLE CONDITION` is the legacy CICS error-handling style. Modern programs that use `RESP(rc)` style (or `REXX SIGNAL ON ERROR`) do not need it; the two styles can coexist within the same program because both ultimately read `EIBRESP`.

Options `condN(label)` Arm `condN` (one of the names in Chapter 12. Response codes, e.g. `NOTFND`, `DUPREC`, `MAPFAIL`, `INVREQ`, `IOERR`, `LENGERR`, `QIDERR`, `ITEMERR`, `ENDFILE`, `PGMIDERR`, `EOC`) so it branches to `label`.

`condN` (*no parens*) Disarm `condN`.

ERROR(label) Arm a catch-all that fires for any non-`NORMAL` response not matched by a more specific arm.

A single `HANDLE CONDITION` may combine any number of arm and disarm options.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Map updated.
INVREQ	16	Unknown condition name.

Example

```
ADDRESS CICS
```

```
EXEC CICS HANDLE CONDITION NOTFND(NOREC)  
                        DUPREC(DUP)  
                        ERROR(OOPS)
```

```
END-EXEC
```

```
EXEC CICS READ FILE('CUSTOMERS') INTO(REC) RIDFLD(CKEY) END-EXEC
```



```
SAY 'Found:' REC
EXIT
```

```
NOREC: SAY 'No customer with key' CKEY; EXIT 4
DUP:    SAY 'Duplicate key on insert'; EXIT 8
OOPS:   SAY 'Other CICS error, RC=' EIBRESP; EXIT 12
```

IGNORE CONDITION

Suppress the `HANDLE CONDITION` trap for one or more conditions.

Format

```
EXEC CICS IGNORE CONDITION cond1 cond2 ...
END-EXEC
```

Description Marks each named condition as *ignored*: even if `HANDLE CONDITION` previously armed it, the trap will no longer fire and the program must test `EIBRESP` itself. `IGNORE CONDITION` stays in effect until a later `HANDLE CONDITION cond(label)` re-arms the same condition.

The distinction between *unarmed* (never armed, or disarmed by the bare-name form) and *ignored* matters only when `ERROR(label)` is armed: an unarmed condition still falls through to `ERROR`, but an ignored one does not.

Options Each operand is a bare condition name — no parentheses, no label. Multiple conditions may be combined in a single call.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Map updated.
INVREQ	16	Unknown condition name.

Example

```
EXEC CICS HANDLE CONDITION ERROR(BADIO) END-EXEC.
```

```
* We *expect* NOTFND on this READ – don't trip the ERROR trap.
EXEC CICS IGNORE CONDITION NOTFND END-EXEC.
```

```
EXEC CICS READ FILE('CUSTOMERS') INTO(WS-REC) RIDFLD(WS-KEY) END-EXEC.
IF EIBRESP = 13
    PERFORM CREATE-NEW-CUSTOMER
END-IF.
```

HANDLE AID

Arm program labels to receive control on a particular attention key.

Format

```
EXEC CICS HANDLE AID key1[(label)] key2[(label)] ...  
END-EXEC
```

Description Builds a per-task map from AID byte to label. After any command that updates EIBAID (SEND MAP, RECEIVE MAP, RECEIVE), the dispatcher consults the map and branches if the new EIBAID matches an armed key.

The bare key name (no parens) disarms a previously-armed key. **ANYKEY** is a catch-all matched by any AID for which no specific arm exists.

Options **keyN(label)** / **keyN** keyN is one of ENTER, CLEAR, PA1–PA3, PF1–PF24, or ANYKEY. Parenthesised arms the trap; bare disarms it.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Map updated.
INVREQ	16	Unknown AID name.

Example

```
ADDRESS CICS
```

```
EXEC CICS HANDLE AID PF3(QUIT) PF12(QUIT) CLEAR(QUIT) END-EXEC
```

```
DO FOREVER
```

```
    EXEC CICS RECEIVE MAP('CUST1') INTO(SCR.) END-EXEC
```

```
    ... process ...
```

```
    EXEC CICS SEND MAP('CUST1') FROM(SCR.) ERASE END-EXEC
```

```
END
```

```
QUIT:
```

```
    EXEC CICS RETURN END-EXEC
```

HANDLE ABEND

Arm a program-level abend exit.

Format

```
EXEC CICS HANDLE ABEND { LABEL(label) | PROGRAM(name) | CANCEL | RESET }  
END-EXEC
```

Description Registers an exit that runs when the task abends (either via EXEC CICS ABEND or via an unhandled runtime error). The exit *replaces* the implicit SYNCPOINT ROLLBACK that an uncaught abend would perform — it is now the exit’s responsibility to call SYNCPOINT or SYNCPOINT ROLLBACK explicitly.

EIBABCODE is set to the abend code (default AAAA) before the exit receives control.

Options **LABEL(label)** Branch to `label` within the current program when the task abends. This is the form REXX programs almost always use.

PROGRAM(name) XCTL to the named program when the task abends. Currently accepted by the parser; the runtime treats **PROGRAM(name)** as equivalent to **LABEL(name)** (i.e. it branches inside the current program rather than transferring control).

CANCEL Disarm the current exit and restore the previous one (one level of nesting is preserved).

RESET Re-enable an exit previously suspended by **CANCEL**.

Exactly one of LABEL, PROGRAM, CANCEL, RESET must be given.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Exit registered or restored.
INVREQ	16	Conflicting / missing options, or RESET with no prior exit.

Example

```
ADDRESS CICS
```

```
EXEC CICS HANDLE ABEND LABEL(CLEANUP) END-EXEC
```

```
EXEC CICS WRITE FILE('LEDGER') RIDFLD(K) FROM(REC) END-EXEC
```

```
EXEC CICS WRITE FILE('AUDIT') RIDFLD(K) FROM(REC) END-EXEC
```

```
/* something later may ABEND */
```

```
EXIT
```

```
CLEANUP:
```

```
  SAY 'Caught abend' EIBABCODE
```

```
  EXEC CICS SYNCPOINT ROLLBACK END-EXEC      /* undo both writes */
```

```
  EXEC CICS RETURN END-EXEC
```

Chapter 11. The Execute Interface Block (EIB)

The EIB is a per-task scratch area populated by the dispatcher before the program runs and by each EXEC CICS command on return. In bricks the EIB is exposed as a set of well-known variable

names in the program's frame; both REXX and COBOL auto-inject the names that aren't already declared.

Field	Set by	Meaning
EIBAID	SEND MAP / RECEIVE MAP	Single-byte AID character of the most recent map response. Use <code>C2X(EIBAID)</code> (REXX) or compare with <code>X'F3'</code> (COBOL) to detect PF/PA/CLEAR/ENTER.
EIBCP0SN	SEND MAP / RECEIVE MAP	1-based cursor position of the most recent map response.
EIBCALEN	dispatcher (per-task entry)	Length of <code>DFHCOMMAREA</code> flowed in from the prior task or <code>LINK</code> caller. Zero when no <code>COMMAREA</code> was passed.
EIBTRMID	dispatcher	This terminal's id. Same as <code>ASSIGN TERMID(...)</code> .
EIBRESP	every EXEC CICS	The response code of the most recent command (see Chapter 12).
EIBRESP2	every EXEC CICS	The secondary response code of the most recent command (currently always 0).
RC	every EXEC CICS	Mirror of <code>EIBRESP</code> , for REXX <code>IF RC <> 0</code> style.
DFHCOMMAREA	dispatcher / LINK / RETURN	The <code>COMMAREA</code> bytes; in COBOL, auto-injected as <code>PIC X(2000)</code> if not declared.

REXX

`EIBAID` and friends are ordinary REXX variables that the handlers write through the `cics.Frame` interface. A REXX program tests them exactly like any other variable:

```
IF EIBRESP <> 0 THEN SAY 'Command failed, RC=' EIBRESP
IF C2X(EIBAID) = 'F3' THEN EXEC CICS RETURN END-EXEC
```

`SIGNAL ON ERROR` (Chapter 18) is the alternative to per-call `IF EIBRESP <>`.

COBOL

The same names are auto-injected as `PIC` items if the program doesn't declare them (`cobol.ensureSystemItems`). Test them with ordinary `IF` clauses:

```
IF EIBRESP NOT = 0
    DISPLAY 'Command failed'
END-IF.
```

```

IF EIBRID = X'F3'
    EXEC CICS RETURN END-EXEC
END-IF.

```

There is no COBOL equivalent of `SIGNAL ON ERROR` yet.

Chapter 12. Response codes

After every `EXEC CICS` command the handler writes `EIBRESP`, `EIBRESP2`, and `RC` into the program's frame. The full set of constants is in `cics/resp.go`; the values a typical bricks program tests for are:

Constant	Value	Meaning
<code>NORMAL</code>	0	Success.
<code>ERROR</code>	1	Generic error.
<code>EOC</code>	6	End-of-chain (second <code>RECEIVE</code> in same task).
<code>NOTFND</code>	13	Record / key not found.
<code>DUPREC</code>	14	<code>WRITE</code> collided with an existing key.
<code>DUPKEY</code>	15	Duplicate key on a non-unique index (reserved).
<code>INVREQ</code>	16	Invalid request: bad option combination, missing data store, invalid name.
<code>IOERR</code>	17	Underlying store / network IO failed.
<code>NOSPACE</code>	18	Out of storage (reserved).
<code>NOTOPEN</code>	19	File not open (reserved).
<code>ENDFILE</code>	20	Browse walked past the last key (or out of <code>GENERIC</code> prefix).
<code>LENGERR</code>	22	Length mismatch.
<code>QZERO</code>	23	Queue is empty (reserved).
<code>ITEMERR</code>	26	TS item out of range; typical at end-of-queue.
<code>PGMIDERR</code>	27	<code>LINK</code> / <code>XCTL</code> target not in <code>transactions.conf</code> , or sub-program errored, or caller is not authorised for the target.
<code>MAPFAIL</code>	36	<code>SEND MAP</code> / <code>RECEIVE MAP</code> could not find or render the named map.

Idiomatic error handling**REXX, per-call test:**

```
EXEC CICS READ FILE('CUSTOMERS') INTO(REC) RIDFLD(K) END-EXEC
SELECT
  WHEN EIBRESP = 0 THEN NOP
  WHEN EIBRESP = 13 THEN MSG = 'Customer' K 'not found'
  OTHERWISE          MSG = 'I/O error, RC=' EIBRESP
END
```

REXX, signal-on:

```
SIGNAL ON ERROR NAME CICSERR
EXEC CICS READ FILE('CUSTOMERS') INTO(REC) RIDFLD(K) END-EXEC
...
EXIT
CICSERR:
  SAY 'CICS error at line' SIGL ', RC=' RC
  EXIT
```

COBOL:

```
EXEC CICS READ FILE('CUSTOMERS') INTO(REC) RIDFLD(CKEY) END-EXEC
EVALUATE EIBRESP
  WHEN 0    CONTINUE
  WHEN 13   MOVE 'Customer not found' TO MSG
  WHEN OTHER MOVE 'I/O error' TO MSG
END-EVALUATE.
```

Chapter 13. Commands not implemented

The following commands are not implemented and the parser does not recognize them at all:

Command	Use
GETMAIN, FREEMAIN	Dynamic storage. REXX has dynamic variables; COBOL has <code>WORKING-STORAGE</code> .
LOAD, RELEASE	Explicit program-fetch lifecycle. Bricks caches program text on first dispatch.
WAIT EVENT, POST	ECB-driven inter-task signalling — bricks's scheduler model does not expose ECBs.

DELAY

```
EXEC CICS DELAY
```

```

{INTERVAL(hhmmss) | TIME(hhmmss) |
  FOR HOURS(h) MINUTES(m) SECONDS(s) MILLISECS(ms) |
  UNTIL HOURS(h) MINUTES(m) SECONDS(s)}
[RESP(rc) RESP2(rc2)]

```

END-EXEC.

Suspends the issuing task until the given duration elapses (**INTERVAL** / **FOR**) or until the named time-of-day arrives (**TIME** / **UNTIL**). A bare **DELAY** with no clauses is a **NORMAL** yield. **INTERVAL** and **TIME** accept the same **HHMMSS** digit forms as **START**. **FOR** / **UNTIL** are the IBM-canonical compositional alternatives — supply any subset of **HOURS**, **MINUTES**, **SECONDS**, **MILLISECS**; the millisecond clause is the only sub-second form available.

Bricks sleeps the calling goroutine; PA1 break-out continues to work because the connection's prompt loop is unaffected. A long **DELAY** on a terminal that disconnects mid-sleep ends when the conn read deadline poisons the next dispatch — the task is not held alive past the connection.

CANCEL

```

EXEC CICS CANCEL
      {REQID(handle) | TRANSID(name) [TERMID(t)]}
      [RESP(rc) RESP2(rc2)]

```

END-EXEC.

Drops a pending **START** from the in-region scheduler. **REQID(handle)** matches against the 1–8-character handle the originating **START** supplied; bricks records the handle on the queued entry. **TRANSID(name)** drops the soonest pending **START** to that transid on **TERMID** (default = the issuing terminal). **RESP=NOTFND** (13) when no match.

The scheduler is in-memory; a bricks restart drops every pending entry, after which any **CANCEL** returns **NOTFND**.

ENQ / DEQ

```

EXEC CICS ENQ RESOURCE(name) [LENGTH(n)] [HOLD] [NOSUSPEND]
      [RESP(rc) RESP2(rc2)]

```

END-EXEC.

```

EXEC CICS DEQ RESOURCE(name) [LENGTH(n)]
      [RESP(rc) RESP2(rc2)]

```

END-EXEC.

Named-resource locking with region scope. **ENQ** blocks until the resource is acquired; with **NOSUSPEND**, an already-held resource returns **RESP=ENQBUSY** (55) immediately. A re-entrant **ENQ** by the same task on the same resource is a **NORMAL** no-op. The default lock scope is the next **SYNCPPOINT** (commit or rollback) or task end; **HOLD** extends the scope past **SYNCPPOINT** so the lock persists until an explicit **DEQ** or task end.

LENGTH(n) is accepted for source compatibility but bricks uses the entire resolved **RESOURCE** value as the key (uppercased, trimmed).

A non-**NOSUSPEND** **ENQ** that has waited five minutes for the holder to release returns **ENQBUSY** rather than blocking indefinitely. That cap converts a dropped-owner edge case into a clean recoverable

status; legitimate same-region holders are never close to that limit.

WRITE OPERATOR

```
EXEC CICS WRITE OPERATOR
    TEXT(area) [LENGTH(n)]
    [ROUTECODES(codes)] [NUMROUTES(n)]
    [EVENTUAL | ACTION | CRITICAL | IMMEDIATE]
    [CONSNAME(name)]
    [RESP(rc) RESP2(rc2)]
END-EXEC.
```

Writes `TEXT(area)` to the bricks console with the `SYS` tag. The issuing `TERMINID` is included so the operator can correlate the message with the task that wrote it. `LENGTH(n)` bounds the bytes written. `ROUTECODES`, `NUMROUTES`, `CONSNAME`, `EVENTUAL`, `ACTION`, `CRITICAL`, and `IMMEDIATE` are accepted for source compatibility but fold into a single console write — bricks has no multi-console fan-out.

The `REPLY` form (`REPLY(var) MAXLENGTH(n) REPLYLENGTH(var) [TIMEOUT(n)]`) is rejected with `RESP=INVREQ`. Bricks has no operator-console input port to source the reply from, so accepting it silently would hang the task. A program that needs operator acknowledgement must reach the operator through `SEND MAP` on a configured operator terminal.

SEND CONTROL

```
EXEC CICS SEND CONTROL [ERASE | ERASEAUP]
    [FREEKB] [ALARM] [FRSET]
    [CURSOR(pos)] [PRINT] [LDC(name)] [STRFIELD]
    [RESP(rc) RESP2(rc2)]
END-EXEC.
```

3270 control order without a data payload — used between a `SEND MAP DATAONLY` pre-paint and the next operator interaction to clear the screen, unlock the keyboard, or reset MDTs without re-painting any field content. `ERASE` and `ERASEAUP` clear the screen; the other flags are accepted but their wire-bits are not tunnelled by the underlying go3270 layer (`FREEKB` and `FRSET` are part of every paint’s fixed WCC, so they have no separate effect; `ALARM` / `CURSOR` / `PRINT` / `STRFIELD` / `LDC` are silently accepted so program source compiles). A flag-less `SEND CONTROL` is a `NORMAL` no-op.

Recently added

- **CONVERSE** — see Chapter 4. Syntactic sugar for `SEND MAP + RECEIVE MAP`.
- **START / RETRIEVE** — see Chapter 5. Same-terminal scheduling with payload pass-through. Cross-terminal and no-`TERMINID` (headless) `STARTs` are still deferred; the v1 surface rejects them with `RESP=INVREQ` and a clear message.
- **DELAY, CANCEL, ENQ / DEQ, WRITE OPERATOR, SEND CONTROL** — round-4 verbs documented above.
- **WEB *** (server-side, Phase 1) — see “EXEC CICS WEB” below. The client-side surface (`WEB OPEN / SEND / RECEIVE / CONVERSE / CLOSE`) and the `DOCUMENT` API land in Phase 2; `URIMAP / TLS / mTLS` in Phase 3.

Part 7. EXEC CICS WEB command reference

Part 7 begins here. Bricks ships a complete EXEC CICS WEB family — both the **inbound** (server-side, the WAPI listener) and the **outbound** (client-side, programs calling external HTTP services). The **DOCUMENT** API for assembling response bodies from many fragments is documented here too, alongside the URIMAP catalogue, inbound mTLS, the **CONVERSE WEB** alias, and the dedicated copybooks.

EXEC CICS WEB — server side (Phase 1)

bricks ships an inbound HTTP listener that turns each matched request into a transaction dispatch via the EXEC CICS WEB * verbs. Enable it in `bricks.cnf` with `enable_wapi=yes`. Define routes in `runtime/web_routes.conf` — the URIMAP layer; the matched TRANSID is looked up in `transactions.conf` exactly as a 3270 operator's typed dispatch would. Both files reload on mtime change, so adding a new route requires no restart.

#	method	path-pattern	transid	[groups]	[response_timeout]
	GET	/api/customer/{id}	WAPI	public	
	POST	/api/customer	WAPC	admin	
	GET	/api/orders/{id}	OAPI	users	
	DELETE	/admin/cache/{key}	CACR	admin	5s

The matching `transactions.conf` rows route TRANSID → program (REXX or COBOL) — same format used by 3270 dispatch:

```
WAPI:rexx:wapi.rexx:public,users,admin
WAPC:cobol:wapic.cob:admin
OAPI:rexx:orders.rexx:users,admin
CACR:cobol:cacheclr.cob:admin
```

A single program is therefore reachable from **both** the 3270 prompt (the operator types its TRANSID) and the HTTP endpoint (a client hits the matching URL). The transaction distinguishes the front door by which family of verbs it calls — EXEC CICS WEB * returns INVREQ on a 3270 task, and EXEC CICS SEND MAP is meaningless on a web task — but most programs naturally use one set or the other.

A {name} placeholder in the path is exposed to the transaction via `WEB READ QUERYPARM('name')`, sharing one namespace with the inbound `?name=...` query string (path captures win on conflict).

Authentication — the same users.conf that CSSN uses

A web client identifies itself using **HTTP Basic authentication**, and bricks verifies the credentials against the exact same `users.conf` and bcrypt hash that the CSSN sign-on transaction uses on the 3270. The credential carrier is different (an HTTP header instead of a 3270 password field); everything downstream is identical.

Layer	3270 / CSSN	WAPI / HTTP
Credential store	<code>runtime/users.conf</code>	<code>runtime/users.conf</code> (same file)
Hashing	bcrypt	bcrypt
Verification call	<code>auth.FileStore.Authenticate(user, pass)</code>	<code>auth.FileStore.Authenticate(user, pass)</code> (same call)

Layer	3270 / CSSN	WAPI / HTTP
Credential carrier	Hidden PASSWORD field on the CSSN map, posted by SEND→RECEIVE	Authorization: Basic base64(user:pass) header
Operator-visible state	sess.Authenticated=true ; UCB attached for the whole 3270 session until CSSF LOGOFF	One-shot per request — sess.Authenticated=true for the dispatch only
What the transaction sees	EXEC CICS ASSIGN USERID(...) returns the operator	Same — EXEC CICS ASSIGN USERID(...) returns the bcrypt-verified user

The dispatch flow per request:

1. **No Authorization header** — if the route's **groups** column contains the literal token **public**, the request is accepted as anonymous (**UserID=""**, **Authenticated=false**); otherwise the response is **401 Unauthorized** with **WWW-Authenticate: Basic realm="bricks"** so a browser shows the standard credential prompt.
2. **Authorization: Basic base64(user:pass)** — the credential is base64-decoded, split on **:**, then **auth.FileStore.Authenticate** runs (same path CSSN uses). On success the user's **users.conf** groups are intersected with the route's groups.
3. **Authenticated AND a group matches** — dispatch proceeds with **sess.Authenticated=true**, **sess.UserID=<username>**, **sess.Groups=<users.conf groups>**. The dispatcher's **IsAllowed** gate runs again against the **TRANSID**'s **transactions.conf** groups, so **both** layers must permit the caller before the program runs.
4. **Authenticated but no group match** — **403 Forbidden**.
5. **Authentication fails** (wrong password, unknown user, malformed header) — **401 Unauthorized**.

ACL policy is **opt-in**: a **web_routes.conf** row with no **groups** column denies every request, log warning at boot. Add the literal **public** to permit anonymous access; add specific group names (**users**, **admin**, etc.) to require Basic-Auth against **users.conf** for a user belonging to that group.

A user **alice** listed in **users.conf** as **alice:<bcrypt-hash>:users,admin** can run the **same** **TRANSID** from both front doors with the same credential:

```
curl -u alice:alice-password http://localhost:8080/api/orders/42
# → 200 + JSON body, sess.UserID=alice
```

```
curl -u alice:wrong-password http://localhost:8080/api/orders/42
# → 401 Unauthorized
```

```
curl http://localhost:8080/api/orders/42
# → 401 Unauthorized (no Authorization header)
```

...and at a 3270 emulator she signs on with CSSN → **alice / alice-password** and types the same **TRANSID**. The program sees the same **EIBTRMID / USERID / EIBCALEN** set and serves both callers identically. Most programs don't even need to know which front door they came from.

What's intentionally NOT yet supported:

- **Bearer tokens / cookies / sessions** — every request is stateless and re-authenticates against `users.conf`. There is no equivalent of the 3270 “stay signed on until CSSF LOGOFF” state across HTTP requests. The token-mint endpoint and **Authorization: Bearer ...** flow land in Phase 3.
- **OAuth / JWT / OIDC** — Phase 3+ work.
- **Per-request authentication via the CSSN transaction itself** — CSSN is a 3270-only screen artefact; pointing a route at it doesn’t produce the browser UX a credential prompt should have. The Phase 3 token endpoint will be the right shape for that.

Server-side verb set

The Phase 1 verb set:

Verb	Use
WEB EXTRACT METHOD(var) PATH(var) ...	Request metadata: METHOD / SCHEME / HOST / PORT / PATH / QUERYSTRING / HTTPVERSION / CLIENTADDR / SERVERADDR. Each option is the name of a target variable.
WEB READ HTTPHEADER(name) VALUE(var) [LENGTH(var)]	Read one inbound header. NOTFND when absent.
WEB STARTBROWSE HTTPHEADER / WEB READNEXT HTTPHEADER NAME(var) VALUE(var) [NAMELENGTH(var)] [VALUELENGTH(var)] / WEB ENDBROWSE HTTPHEADER	Walk every inbound header in sorted order. ENDFILE when exhausted.
WEB READ QUERYPARAM(name) VALUE(var)	Read query parameter OR routing-table {name} capture. NOTFND when absent.
WEB STARTBROWSE QUERYPARAM / WEB READNEXT QUERYPARAM ... / WEB ENDBROWSE QUERYPARAM	Iterate every parameter / capture.
WEB READ FORMFIELD(name) VALUE(var)	Read one application/x-www-form-urlencoded field. Body parsed lazily on first call.
WEB STARTBROWSE FORMFIELD / WEB READNEXT FORMFIELD ... / WEB ENDBROWSE FORMFIELD	Iterate form fields.
WEB RECEIVE INTO(var) [MAXLENGTH(n)] [LENGTH(var)] [TYPE(var)] [MEDIATYPE(var)]	Read the raw request body. LENGERR when over MAXLENGTH .
WEB WRITE HTTPHEADER(name) VALUE(value)	Set / append an outbound response header.
WEB SEND FROM(buf) [MEDIATYPE(s)] [STATUSCODE(n)]	Emit response status + body. First SEND wins for status/MEDIATYPE; subsequent SEND calls in the same task append to the body (matches CICS).
WEB PARSE URL URL(s) SCHEMENAME(var) HOST(var) PORT(var) PATH(var) QUERYSTRING(var) [HOSTLENGTH(var)] ...	Direction-agnostic URL splitter.
WEB CONVERTTIME DATESTRING(s) ABSTIME(var)	RFC 1123 / 850 / asctime → milliseconds since 1970-01-01 UTC.

The full REXX sample is `runtime/rexx/wapi.rexx`; in COBOL the helpful copybooks are **DFHWBSC**

(status-code constants), DFHWBUH (common header-name literals), DFHWBMT (MIME-type literals), and DFHWBMETH (HTTP method literals).

EXEC CICS WEB — client side (Phase 2a)

The same **WEB** verb family also drives **outbound** HTTP — a bricks transaction calling a remote HTTP service. No `bricks.cnf` toggle is required; bricks builds one shared `*http.Client` at startup (tuned by `web_client_timeout` / `web_client_max_idle_conns` / `web_client_tls_skip_verify`) and every task reuses it. The host's CA bundle validates outbound HTTPS certificates by default — point at any normal public HTTPS endpoint and it just works.

The client uses a per-task **session token**: **WEB OPEN** returns a 32-character hex token; subsequent **WEB SEND** / **WEB RECEIVE** / **WEB CONVERSE** / **WEB CLOSE** all thread that token via `SESSTOKEN(...)`. A token still open at task end is auto-closed (dispatcher defer, mirrors `CloseBrowses` / `CloseAllTD`).

The Phase 2a verb set:

Verb	Use
WEB OPEN HOST(s) [PORT(n)] [SCHEME(s)] SESSTOKEN(var)	Open a logical client session. PORT defaults to 80 (HTTP) / 443 (HTTPS). SCHEME defaults to <code>http</code> . The token is written into the variable named by <code>SESSTOKEN</code> .
WEB CONVERSE SESSTOKEN(t) METHOD(s) PATH(s) [FROM(buf)] [LENGTH(n)] [QUERYSTRING(s)] [MEDIATYPE(s)] INTO(var) [STATUSCODE(var)] [STATUSTEXT(var)] [MEDIATYPE(var)] [TYPE(var)] [MAXLENGTH(n)] [LENGTH(var)]	One-shot send + receive. Builds the request, fires it through the shared client, writes the response body into <code>INTO</code> ; <code>LENGERR</code> when the body exceeds <code>MAXLENGTH</code> (default 1 MiB).
WEB SEND SESSTOKEN(t) METHOD(s) PATH(s) [FROM(buf)] [QUERYSTRING(s)] [MEDIATYPE(s)]	Stage a request on the session (no round-trip yet).
WEB RECEIVE SESSTOKEN(t) INTO(var) [STATUSCODE(var)] [STATUSTEXT(var)] [MEDIATYPE(var)] [MAXLENGTH(n)]	Fire the staged request; populate the named output variables from the response.
WEB CLOSE SESSTOKEN(t)	Release the session and the underlying response body.
WEB WRITE HTTPHEADER(name) VALUE(v) SESSTOKEN(t)	Set an outbound <i>request</i> header on the session. Repeated <code>WRITE</code> calls for the same name produce multi-valued headers (matches <code>http.Header.Add</code>). Must precede the next <code>SEND</code> / <code>CONVERSE</code> – headers committed at request build time.
WEB READ HTTPHEADER(name) VALUE(var) [LENGTH(var)] SESSTOKEN(t)	Read one response header by name from the most-recent <code>SEND/CONVERSE/RECEIVE</code> on the session. Case-insensitive name match. <code>NOTFND</code> when absent; <code>INVREQ</code> when no response has landed yet.

Verb	Use
WEB STARTBROWSE HTTPHEADER SESSTOKEN(t) / WEB READNEXT HTTPHEADER NAME(var) VALUE(var) [NAMELENGTH(var)] [VALUELENGTH(var)] SESSTOKEN(t) / WEB ENDBROWSE HTTPHEADER SESSTOKEN(t)	Iterate every response header in sorted order. ENDFILE when exhausted; INVREQ when no STARTBROWSE has run on the session.
WEB EXTRACT SESSTOKEN(t) [SCHEME(var)] [HOST(var)] [PORT(var)] [PATH(var)] [HOSTLENGTH(var)] [PORTNUMBER(var)]	Inspect the session's resolved endpoint <i>after</i> OPEN. Distinct from WEB EXTRACT URIMAP(name) (no token) and from the server-side request-meta form (no token, different option set).

Bricks issues `Accept: */*` and `User-Agent: bricks-cics/1.0` on every outbound request by default; both can be overridden with `WEB WRITE HTTPHEADER('User-Agent') VALUE(...) SESSTOKEN(t)` before the next `SEND / CONVERSE` on that session. The `runtime/cobol/whdr.cob` sample (`TRANSID WHDR`) walks through the `WRITE`-then-`READ` flow end-to-end.

RESP codes: `NORMAL`, `NOTFND` (unknown `SESSTOKEN`), `INVREQ` (missing required option, web client not configured), `IOERR` (network / TLS / timeout), `LENGERR` (body over `MAXLENGTH`).

COBOL copybook for the client surface

`runtime/cobolcopy/DFHWBSI.cpy` carries the standard session-token + body buffer templates:

```
01 DFH-WB-SESS      PIC X(36).
01 DFH-WB-BODY      PIC X(8192).
01 DFH-WB-URL       PIC X(2048).
```

Pair it with the existing `DFHWBSC` (status codes) and `DFHWMETH` (HTTP method literals) so a CICS-style program reads idiomatically:

```
COPY DFHRESP.
COPY DFHWBSC.
COPY DFHWMETH.
COPY DFHWBSI.

EXEC CICS WEB OPEN HOST('api.github.com') PORT(443)
                SCHEME('HTTPS')
                SESSTOKEN(DFH-WB-SESS) END-EXEC.

EXEC CICS WEB CONVERSE SESSTOKEN(DFH-WB-SESS)
                METHOD(WB-GET)
                PATH('/zen')
                INTO(DFH-WB-BODY)
                STATUSCODE(STAT) END-EXEC.

EXEC CICS WEB CLOSE SESSTOKEN(DFH-WB-SESS) END-EXEC.

IF STAT = DFHRESP-WB-OK ...
```

URIMAP — symbolic outbound endpoints (Phase 3a)

A URIMAP entry in `runtime/web_routes.conf` defines a named upstream service that outbound transactions reference by symbol. Both layers of `web_routes.conf` (routes for inbound, URIMAPs for outbound) live in the same file and hot-reload together.

```
# Format: URIMAP NAME scheme://host[:port][/path-prefix]
URIMAP  GITHUB    https://api.github.com
URIMAP  WEATHER   https://api.openweathermap.org/data/2.5
URIMAP  INTRANET  https://api.internal:8443/v1
```

Names are 1..8 uppercase characters (CICS resource-name limit). The path-prefix is optional; when set, it becomes the leading segment of every PATH the program later supplies on `WEB SEND` / `WEB CONVERSE`. So `URIMAP WEATHER ...data/2.5` paired with `WEB CONVERSE PATH('/weather?city=NYC')` issues against `https://api.openweathermap.org/data/2.5/weather?city=NYC`.

The URIMAP-aware verb shapes:

Verb	Use
<code>WEB OPEN URIMAP(name) SESSTOKEN(var)</code>	Open a client session against the named endpoint. SCHEME / HOST / PORT come from the URIMAP row; explicit <code>HOST(...)</code> / <code>PORT(...)</code> / <code>SCHEME(...)</code> on the same command override per-field (matches IBM CICS). <code>NOTFND</code> when the name is unknown.
<code>WEB SEND URIMAP(name) METHOD(s) PATH(s)</code> <code>[FROM(buf)] [QUERYSTRING(s)] [MEDIATYPE(s)]</code> <code>INTO(var) [STATUSCODE(var)]</code> <code>[STATUSTEXT(var)] [LENGTH(var)]</code> <code>[MAXLENGTH(n)]</code>	One-shot <code>OPEN</code> + <code>CONVERSE</code> + <code>CLOSE</code> : no <code>SESSTOKEN</code> required. Resolves the URIMAP, opens a fresh session, sends the request, populates <code>INTO</code> and the named output variables from the response, and releases the session. Use for fire-and-forget outbound calls where session-token plumbing has no value. (Phase 3b.)
<code>WEB EXTRACT URIMAP(name) [SCHEME(var)]</code> <code>[HOST(var)] [PORT(var)] [PATH(var)]</code> <code>[HOSTLENGTH(var)] [PORTNUMBER(var)]</code>	Inspect a URIMAP definition without opening a session. Used by programs that branch on the resolved endpoint or need to display it to an operator. <code>NOTFND</code> when the name is unknown.

URIMAP rows load even when `enable_wapi=no` — outbound-only deployments are supported without exposing any inbound listener.

The `runtime/cobolcopy/DFHWBUM.cpy` copybook ships the conventional `WORKING-STORAGE` variables:

```
01 DFH-WB-UM-NAME    PIC X(8).
01 DFH-WB-UM-SCHEME  PIC X(8).
01 DFH-WB-UM-HOST    PIC X(64).
01 DFH-WB-UM-PORT    PIC X(5).
01 DFH-WB-UM-PATH    PIC X(128).
```

Outbound TLS verification (Phase 3a)

By default the outbound HTTPS client verifies upstream certificates against the host system's trust store — the OS's `/etc/ssl/certs/...` bundle on Linux, the Keychain on macOS, Windows trust store on Windows. Three `bricks.cnf` knobs adjust:

Knob	Use
<code>web_client_ca_bundle</code>	Path to a PEM file of trusted CAs. When set, bricks uses ONLY these for verification — the system trust store is ignored. Useful for internal CAs that aren't installed system-wide.
<code>web_client_cert</code> + <code>web_client_key</code>	PEM client-cert and key for outbound mTLS. Presented to upstream services that require the caller to authenticate with a certificate. Both must be set together; one without the other is a startup error.
<code>web_client_tls_skip_verify</code>	<code>yes</code> accepts ANY upstream certificate — test-only escape hatch for self-signed endpoints. Logs a loud startup WARNING ; never use in production.

Operator visibility (Phase 3a)

- **CEMT INQUIRE URIMAP** — pages the URIMAP rows loaded from `web_routes.conf` (NAME / SCHEME / HOST / PORT / PATH-PREFIX).
- **CEMT INQUIRE WEB** — pages the inbound HTTP requests currently in dispatch (METHOD / URI / TERM / TRANSID / USER / AGE). When no requests are in flight, the screen still shows a one-row summary with the lifetime request / error totals.

Phase 2a sample — `wzen.cob`

`runtime/cobol/wzen.cob` (TRANSID `WZEN`) is the shipped end-to-end demonstration. It fetches GitHub's `/zen` endpoint over HTTPS — a single-line plain-text response, no JSON parsing needed — and renders the result on the `WZEN1` 3270 map. The program demonstrates the full client lifecycle, branches cleanly on each `EIBRESP`, and surfaces network / HTTP failures with a distinct error path so the operator sees what went wrong on the 3270 screen rather than getting an opaque `abend`.

Drive it from a 3270 emulator: sign on (`CSSN` → `admin`), type `WZEN`, press ENTER. The first line under the header shows the actual URL fetched (`GET https://api.github.com/zen`) so the demo is self-documenting; the body appears on row 8.

Phase 2b sample — `whdr.cob` (custom request + response headers)

`runtime/cobol/whdr.cob` (TRANSID `WHDR`) is the canonical end-to-end demonstration of `WEB WRITE HTTPHEADER` and `WEB READ HTTPHEADER` on a client session. It opens the `GITHUB` URIMAP, sets two outbound headers (`User-Agent: bricks-whdr/1.0`, `Accept: text/plain`), issues `GET /zen`, then reads three response headers — two that always land on a GitHub response (`Content-Type`, `Server`) and one deliberately-absent header to exercise the `NOTFND` branch. The 3270 screen shows the request headers sent, the response headers received, and the body excerpt.

Drive it the same way as WZEN: sign on with CSSN, type WHDR, press ENTER. Pair the screen with CEMT INQUIRE WEB in a second 3270 session to watch the per-route counters move each time you rerun the transaction.

EXEC CICS DOCUMENT — chunked body builder

The DOCUMENT API is the buffer-builder that pairs with WEB SEND DOCTOKEN(t) for programs assembling large responses from many small fragments. Classic CICS pattern for emitting HTML / XML / JSON in pieces; the bricks implementation also serves as the chunk store for WEB RETRIEVE DOCTOKEN(var) when programs want the inbound body as a document handle instead of a flat INTO(buf).

Verb	Use
DOCUMENT CREATE DOCTOKEN(var) [SYMBOLLIST(s)] [LISTLENGTH(n)] [DELIMITER(s)]	Allocate a fresh empty document; return its token in the named variable. Optional SYMBOLLIST seeds a name=value&... symbol table that subsequent INSERT SYMBOL(...) substitutes from.
DOCUMENT INSERT DOCTOKEN(t) [FROM(buf) [LENGTH(n)] \ TEXT(buf) \ BINARY(buf) \ SYMBOL(name) \ DOCUMENT(other-token)] [AT(position)]	Append (or splice with AT(position)) one chunk to the document. SYMBOL(name) substitutes from the bound symbol list (NOTFND when unknown); DOCUMENT(other-token) splices another document verbatim (NOTFND when unknown); TEXT and BINARY are codepage-aware in real CICS but collapse to FROM in bricks (single-codepage). LENGERR when the resulting buffer would exceed 4 MiB.
DOCUMENT SET DOCTOKEN(t) SYMBOLLIST(s) [LISTLENGTH(n)] [DELIMITER(d)] [UNESCAPED]	(Re-)bind the symbol list a subsequent INSERT SYMBOL substitutes from. Pass an empty SYMBOLLIST to clear.
DOCUMENT RETRIEVE DOCTOKEN(t) INTO(buf) LENGTH(var) [CHARACTERSET(s)] [DATAONLY \ HOSTCODEPAGE]	Read the assembled body back into INTO; LENGTH receives the byte count. CHARACTERSET / DATAONLY / HOSTCODEPAGE are accepted but ignored (bricks is single-codepage ASCII).
DOCUMENT DELETE DOCTOKEN(t)	Release the document. NOTFND when the token isn't known. Forgotten documents are released at task end automatically.
WEB SEND DOCTOKEN(t) [STATUSCODE(n)] [MEDIATYPE(s)]	Emit the named document as the outbound response body. The DOCUMENT INSERT chain has already built it; SEND just splices the bytes onto BodyOut. NOTFND when the token isn't known.
WEB RETRIEVE DOCTOKEN(var)	Inbound-body-as-document. Allocates a fresh document, copies the request body into it, and returns the token. Alternative to WEB RECEIVE INTO(buf) for programs that prefer the document API.

The standard COBOL declaration:

```
COPY DFHDCDOC.    *> DFH-DOC-TOK (PIC X(36)) + delimiter constants

EXEC CICS DOCUMENT CREATE DOCTOKEN(DFH-DOC-TOK) END-EXEC.
EXEC CICS DOCUMENT INSERT DOCTOKEN(DFH-DOC-TOK)
                        FROM('<html><body>') END-EXEC.
EXEC CICS DOCUMENT INSERT DOCTOKEN(DFH-DOC-TOK)
                        FROM(BODY-CHUNK) END-EXEC.
EXEC CICS DOCUMENT INSERT DOCTOKEN(DFH-DOC-TOK)
                        FROM('</body></html>') END-EXEC.
EXEC CICS WEB SEND DOCTOKEN(DFH-DOC-TOK)
                  MEDIATYPE('text/html')
                  STATUSCODE(200) END-EXEC.
```

RESP codes: **NORMAL**, **NOTFND** (unknown DOCTOKEN, unknown SYMBOL, unknown spliced DOCUMENT), **INVREQ** (missing required option), **LENGERR** (INSERT would exceed 4 MiB), **IOERR** (token generation failed).

EXEC CICS WEB — Phase 3b additions

Phase 3b closes out the master Phase-3 surface. Each addition slots onto the foundations laid by 3a (URIMAP catalogue, active- request tracking, outbound TLS extras) without breaking existing verb shapes.

CONVERSE WEB — alias for WEB CONVERSE

Some shops write the keywords in either order; bricks normalises both at the parser, so the two forms below dispatch identical:

```
EXEC CICS WEB CONVERSE SESSTOKEN(T) METHOD('GET')
                        PATH('/x') INTO(B) END-EXEC.
EXEC CICS CONVERSE WEB SESSTOKEN(T) METHOD('GET')
                        PATH('/x') INTO(B) END-EXEC.
```

WEB EXTRACT TCPIPService — inspect the receiving listener

When **WEB EXTRACT** carries the **TCPIPService** keyword (or the legacy **EXTRACT TCPIP** form without **WEB**), the verb reports the listener that received the inbound request:

Output	Source
TCPIPService(var)	Symbolic listener name — WAPI for the plain port, WAPITLS for the TLS port.
PORTNUMBER(var)	Numeric port the listener is bound to.
IPADDRESS(var)	The server's bind address (mirrors <code>bricks.cnf::dns_name</code>).
CLIENT(var)	Caller's IP address (mirrors WEB EXTRACT CLIENTADDR).
AUTHENTICATE(var)	BASIC when the request carried Authorization: Basic ... , blank otherwise.

The legacy form drops the `WEB` keyword entirely — the parser recognises `EXTRACT TCPIP CLIENT(...)` `AUTHENTICATE(...)` and routes it through the same handler. Useful for programs ported from pre-CICS-TS web-services code.

Inbound mTLS + WEB EXTRACT CERTIFICATE

Set `web_inbound_client_ca=ca.pem` to require every TLS client to present a certificate signed by `ca.pem`. The TLS handshake rejects unauthorised clients before any verb runs — no transaction is dispatched at all. Inside the dispatched program, `WEB EXTRACT CERTIFICATE` reads the verified peer cert:

Output	Source on the peer cert
<code>COMMONNAME(var)</code>	Subject CN.
<code>ORGANISATION(var)</code>	Subject O.
<code>COUNTRY(var)</code>	Subject C.
<code>SERIALNUM(var)</code>	Hex serial number.
<code>ISSUER(var)</code>	Issuer CN.
<code>ISSUERORG(var)</code>	Issuer O.

`EIBRESP = NOTFND` on plain (non-TLS) requests and on TLS requests that carried no client cert — so one `TRANSID` can serve both authenticated and anonymous callers and branch on the resp.

```

COPY DFHRESP.
COPY DFHWBCC.  *> CN / ORG / CO / SERIAL / ISSUER buffers
EXEC CICS WEB EXTRACT CERTIFICATE
      COMMONNAME(DFH-WB-CN)
      ORGANISATION(DFH-WB-ORG)
      COUNTRY(DFH-WB-CO)
      SERIALNUM(DFH-WB-SERIAL)
      ISSUER(DFH-WB-ISSUER)
END-EXEC.
EVALUATE EIBRESP
  WHEN DFHRESP(NORMAL)
    *> Verified cert -- use the fields.
  WHEN DFHRESP(NOTFND)
    *> Plain HTTP or TLS without a client cert.
END-EVALUATE.
```

HTTP/2 on the inbound TLS listener

Go's `net/http` server auto-negotiates HTTP/2 over the TLS listener via ALPN whenever `tls.Config.NextProtos` is left unset — which bricks does on the WAPI TLS port. No knob is required; clients that advertise h2 in their ALPN list get HTTP/2, clients that don't fall back to HTTP/1.1. Verify with `curl --http2 -v https://localhost:443/...` and look for **ALPN**, server accepted to use h2 in the trace output. Cleartext HTTP/2 (h2c) is not enabled — the plain port stays on HTTP/1.1 to avoid a transitive dependency on golang.org/x/net/http2/h2c.

CEDA URIMAP — live edit the URIMAP catalogue

The Phase 3a CEMT INQUIRE URIMAP browser becomes operator-editable in 3b. Type CEDA URIMAP (or CEDA M) from the 3270 prompt for the list view:

S	NAME	SCHEME	HOST	PORT	PATH
_	GITHUB	https	api.github.com	443	-
_	WEATHER	https	api.openweathermap.org	443	/data/2.5
_	INTRANET	https	api.internal	8443	/v1

- Type **A** in the selector column for **Alter** — opens a form pre-loaded with the row's endpoint URL so a one-character edit is enough to repoint the URIMAP.
- Type **D** in the selector column for **Delete** — a Y/N confirmation overlay opens; the row is removed only when the operator types Y.
- Press **PF6** for **Define new** — opens an empty form for the NAME (1..8 chars) and END-POINT URL (scheme://host[:port]/path-prefix).

Every committed change is written atomically through a temp + rename to `web_routes.conf`, with an mtime guard that refuses the write if the file was modified externally between when the screen loaded and when the operator pressed PF5. Each successful mutation logs an audit line in the bricks log:

```
ceda=URIMAP op=DEFINE user=ADMIN term=T0001 target=NEWAPI detail=https://internal.svc:8443/v2  
so the audit trail of who changed which row when is greppable.
```

The new in-process surface that backs the screen lives on `*web.Table: AddURIMap, AlterURIMap, DeleteURIMap, and Mtime()` for the optimistic-concurrency guard. Programs in REXX or COBOL **do not call these directly** — they're the CEDA implementation, not part of the EXEC CICS surface.

Part 3. The REXX language

Part 3 begins here. This part describes the REXX dialect bricks accepts and the small preprocessor that rewrites EXEC CICS blocks into the same parsed command shape COBOL produces. Every command in **Part 4** (EXEC CICS), **Part 6** (EXEC SQL) and **Part 7** (EXEC CICS WEB) is available verbatim from REXX with identical semantics.

Chapter 14. REXX program structure

A bricks REXX program is a flat sequence of statements with optional labels and procedures. Execution begins at the first statement; the program ends when execution falls off the bottom, or hits EXIT. A program file is plain UTF-8 (ASCII printable + tabs + LF only is recommended for terminal output). There is no fixed column discipline — newlines and whitespace are token separators.

```
/* HEL0 — minimum REXX program */  
ADDRESS CICS  
EXEC CICS SEND MAP('HEL01') ERASE END-EXEC  
EXEC CICS RETURN END-EXEC
```

Comments

Block comments `/* ... */` may span multiple lines; nesting is not supported. A REXX program traditionally opens with a one-line header comment so the operator gets a description in CEDA / CEMT when applicable.

Statements

Statements are separated by newlines or `;`. The parser does not require a terminator after the last statement of a block. Labels have the form **NAME:** and may appear at any statement position.

Procedures

A label followed by `PROCEDURE [EXPOSE list]` defines a procedure with its own variable scope. `EXPOSE` re-routes named variables to the caller's frame recursively. Both simple names and stem roots (trailing dot) are accepted in the `EXPOSE` list. `CALL` by name dispatches to the matching `PROCEDURE` body; the same name as a built-in function shadows the built-in.

```
CALL GREET 'Alice'
EXIT
```

```
GREET: PROCEDURE EXPOSE LANG.
  PARSE ARG NAME
  SAY LANG.HELLO NAME
  RETURN
```

A procedure returns to its caller on `RETURN [expr]` or by falling off the bottom. The optional `expr` is assigned to the caller's special variable `RESULT`; a bare `RETURN` or a fall-off-end drops `RESULT` so the caller's existing value is not silently masked. Recursion is permitted (the only ceiling is the interpreter's `MaxSteps` runaway guard).

ADDRESS

`ADDRESS <env>` switches the active command handler. Bare strings inside an `ADDRESS` scope are commands routed to that handler; bricks ships a `CICS` handler. `ADDRESS` with no argument clears the active environment, after which a bare string is a syntax error.

```
ADDRESS CICS
"SEND MAP('CUST1') FROM(SCR.) ERASE"
"RETURN"
```

A command's return code is assigned to the special variable `RC`. A non-zero `RC` fires `SIGNAL ON ERROR` (if armed) — see Chapter 18.

Special variables

Name	Set by	Cleared by
RC	Every command (bare string) executed under an <code>ADDRESS</code> env, every <code>EXEC CICS</code> / <code>EXEC SQL</code> .	Survives until the next command updates it.

Name	Set by	Cleared by
RESULT	CALL fn ... when the called procedure does RETURN expr.	A bare RETURN, a fall-off-end, or DROP RESULT.
SIGL	Any direct SIGNAL and every condition trap (set to the source line of the failing statement).	Survives the trap; persists across normal flow.
SQLCODE / SQLSTATE / SQLERRMC	Every EXEC SQL verb.	Next EXEC SQL.

These names are otherwise ordinary writable variables; nothing stops a program from assigning them, though doing so is bad hygiene.

The EXEC CICS / EXEC SQL preprocessor

Bricks parses EXEC CICS verb operand-list END-EXEC (and the EXEC SQL form) as a single command at the same statement position a bare string would occupy under ADDRESS CICS. The two forms compile to the same dispatch path, so

```
EXEC CICS SEND MAP('HEL01') ERASE END-EXEC
```

and

```
ADDRESS CICS
"SEND MAP('HEL01') ERASE"
```

are equivalent. The EXEC form is preferred in shipped programs because it doesn't depend on the current ADDRESS environment and survives an explicit ADDRESS switch.

Chapter 15. Variables and stems

Naming rules

- Names are case-insensitive; internally everything is uppercased before lookup.
- A name begins with a letter or _ and may continue with letters, digits, _, or !, ?, #, @, \$. Stem-tail segments are separated by ..
- The literal RC, SIGL, RESULT, SQLCODE, SQLSTATE, SQLERRMC are reserved for runtime-managed values (see Chapter 14 — Special variables).

Simple variables and NOVALUE

An unset simple variable resolves to its own uppercased name — the classic REXX NOVALUE convention. SAY X when X has never been assigned prints X. If SIGNAL ON NOVALUE is armed (see Chapter 18) the reference traps instead of resolving to the name.

Assignment is var = expr; the right-hand side is evaluated once and stored verbatim (as a string — REXX has only one runtime type).

Stems

A stem is a variable name that ends in `..`. A bare assignment to the stem (`STEM. = value`) installs a default that any tail inherits when it has no explicit value. Per-tail values override the default:

```
STEM. = 'unset'
STEM.42 = 'forty-two'
SAY STEM.1    /* "unset"      -- default      */
SAY STEM.42   /* "forty-two" -- explicit tail */
```

Compound-symbol tail substitution

Non-numeric tail segments are resolved at every reference. With `J = 3`, the symbol `A.J` reads or writes `A.3`. The rules:

Tail segment shape	Behaviour
STEM.42 (numeric)	Literal — the digit run becomes the tail directly.
STEM.NAME where NAME is set	Substituted — value of NAME becomes the tail.
STEM.NAME where NAME is not set (NOVALUE)	Literal — the bare uppercased name becomes the tail.
STEM.I.J (multi-segment)	Each segment substituted independently — with <code>I=1</code> , <code>J=2</code> , references <code>STEM.1.2</code> .

The NOVALUE-stays-literal rule is the source of bricks's most common porting bug; see Appendix B for the standard pitfall.

DROP

`DROP name [name ...]` removes one or more variables from the current frame. A trailing dot on a stem name drops the default AND every tail — useful for resetting an accumulator between paginated reads:

```
DROP RECS.      /* clear the whole stem */
DROP A B C      /* drop three simples */
DROP STEM.42    /* drop just one tail */
```

PROCEDURE and EXPOSE

Inside a `PROCEDURE`, the parent frame's variables are invisible unless named on the `EXPOSE` list. `EXPOSE` accepts both simple names and stem roots:

```
GREET: PROCEDURE EXPOSE LANG. SUFFIX
...
```

`LANG.` exposes the parent's entire `LANG.` stem (default + all tails); `SUFFIX` exposes a single simple variable. Exposure is transitive — a procedure that exposes a name from a caller which also exposed it from its caller sees the outermost binding.

Chapter 16. Control flow

IF / THEN / ELSE

IF *expr* THEN *stmt-or-block* [ELSE *stmt-or-block*]

The THEN keyword is mandatory. Both arms accept either a single statement or a DO ... END block. Strict boolean evaluation: the condition must evaluate to 0 or 1; anything else raises SYNTAX (so IF X where X = 'foo' is an error, not a truthy test).

```
IF AID = 'F3' THEN DO
    EXEC CICS RETURN END-EXEC
    EXIT
END
ELSE
    CALL REFRESH
```

SELECT / WHEN / OTHERWISE / END

```
SELECT
    WHEN expr THEN stmt
    WHEN expr THEN stmt
    ...
    OTHERWISE stmt-or-block
END
```

WHEN arms are tested in order; the first match's body runs and the rest are skipped. OTHERWISE is optional but if no WHEN matched and OTHERWISE is absent, the runtime raises SYNTAX. Use OTHERWISE NOP for an explicit no-op default.

DO family

Form	Behaviour
DO ... END	Group several statements as one body (most useful inside IF / WHEN).
DO N ... END	Repeat the body N times. N is any expression that evaluates to a non-negative integer.
DO <i>ctrl</i> = <i>start</i> TO <i>end</i> [BY <i>step</i>] ... END	Counted loop. BY defaults to 1. The loop runs while <i>ctrl</i> is between <i>start</i> and <i>end</i> (inclusive). DO <i>ctrl</i> =1 TO 5 BY 0 is rejected at runtime (infinite-loop guard).
DO WHILE <i>expr</i> ... END	Pre-test loop — body runs while <i>expr</i> is truthy.
DO UNTIL <i>expr</i> ... END	Post-test loop — body runs at least once, then re-tests.
DO FOREVER ... END	Unbounded; exit only via LEAVE / RETURN / EXIT / SIGNAL.
DO <i>ctrl</i> OVER <i>stem.</i> ... END	Iterate <i>ctrl</i> over every set tail of <i>stem.</i> . Numeric tails are visited in numeric order, then non-numeric tails in lexicographic order. The default value (<i>stem.</i> = ...) is not visited.

The control variable in `DO ctrl = / DO ctrl OVER` is set on each iteration. `LEAVE [ctrl]` and `ITERATE [ctrl]` target either the innermost loop (no argument) or the loop whose control variable matches the named argument:

```
DO I = 1 TO 100
  DO J = 1 TO 100
    IF GRID.I.J = '*' THEN ITERATE I /* skip rest of inner */
  END
END
```

CALL / RETURN / EXIT

- `CALL name [arg1, arg2, ...]` — call a procedure or a built-in function in statement position. Arguments are expressions, evaluated left-to-right and made visible to the callee via `ARG(n)`. A `CALL` to a built-in throws away the returned value; a user procedure's `RETURN expr` populates `RESULT`.
- `RETURN [expr]` — return from a procedure (or terminate a top-level program). When `expr` is present and the caller was `CALL`, the value lands in `RESULT`; without `expr`, `RESULT` is dropped.
- `EXIT [expr]` — terminate the entire program immediately. The optional `expr` becomes the process exit code (clamped to a byte by the host OS); without it the program exits 0.

SIGNAL

- `SIGNAL label` — non-local jump to a label. Active `DO / SELECT / PROCEDURE` frames are unwound first. `SIGL` is set to the source line of the `SIGNAL` statement.
- `SIGNAL ON cond [NAME label]` — arm a condition trap (see Chapter 18).
- `SIGNAL OFF cond` — disarm.

INTERPRET

`INTERPRET expr` evaluates `expr` to a string and executes that string as REXX source in the current frame. Labels and procedures inside the snippet are scoped to that one evaluation (they do not leak into the surrounding program).

NUMERIC

Form	Effect
<code>NUMERIC DIGITS n</code>	Sets the precision used to round arithmetic results. Default 9.
<code>NUMERIC FUZZ n</code>	Sets the tolerance applied to numeric comparisons. Default 0.
<code>NUMERIC FORM SCIENTIFIC / NUMERIC FORM ENGINEERING</code>	Accepted syntactically but bricks always renders numbers in plain decimal — neither form changes output today.

Arithmetic is `float64` internally, then rounded to `NUMERIC DIGITS` significant figures before storing. The `%` and `//` operators (integer divide and modulo) bypass that rounding.

NOP, PUSH, PULL, QUEUE, OPTIONS, TRACE

- **NOP** — explicit no-op (clearer than an empty `DO ... END` in a `WHEN`).
- **TRACE** [`opt`] — parsed and accepted but ignored; bricks does not run an interpretive tracer.
- **OPTIONS** ... — accepted and ignored (no recognised options).
- **PUSH / PULL / QUEUE / PARSE PULL** — accepted; the associated terminal data queue is always empty in bricks. Read forms return the empty string; write forms are silent no-ops. Use `EXEC CICS RECEIVE` for terminal input.

Bare strings

A bare string at statement position is a command sent to the current `ADDRESS` environment (see Chapter 14). Outside an active environment it is a syntax error.

```
ADDRESS CICS
"SEND MAP('CUST1') FROM(SCR.) ERASE"
"RETURN"
```

Chapter 17. PARSE templates

`PARSE` [`UPPER`] {`VAR` `var` | `VALUE` `expr` `WITH` | `ARG` | `PULL`} `template`

The source decides where the input string comes from; the template decides how it's chopped up and which pieces land in which variables.

Sources

Source	Input string
<code>VAR</code> <code>var</code>	Current value of the named variable.
<code>VALUE</code> <code>expr</code> <code>WITH</code>	The result of evaluating <code>expr</code> . The literal <code>WITH</code> keyword separates the source expression from the template.
<code>ARG</code> [<code>,</code> <code>ARG</code> , ...]	The arguments the program (or procedure) was called with. Each comma-separated segment of the template is one argument, in order — <code>PARSE ARG H, W</code> peels <code>ARG(1)</code> into the first sub-template and <code>ARG(2)</code> into the second.
<code>PULL</code>	The terminal-input queue, which in bricks is always empty. Use <code>EXEC CICS RECEIVE</code> for real input.

`PARSE` `UPPER` uppercases the source string before splitting it across the template — convenient for case-insensitive command parsing.

Template elements

Element	Effect
Bare variable name	Consumes the next whitespace-delimited word. In a <i>run</i> of bare variables, all but the last get one word each; the last gets every remaining word/character up to the next anchor.
. (period)	Placeholder — consumes one token without binding it. Useful as a “skip” in a variable run.
'literal'	String anchor. Matches the next occurrence of <code>literal</code> in the source and splits there. The literal itself is discarded.
n (positive integer)	Absolute column marker — jumps to column n (1-based) in the source. Variables before the marker get everything from the previous column up to column n-1.
+n / -n	Relative column marker — moves the cursor n characters forward or backward from the current position.

Worked examples

```

/* Whitespace tokens. TID = first word, CKEY = second word,
 * '.' discards the rest of the line. */
PARSE VAR LINE TID CKEY .

/* Pipe-delimited fields. */
PARSE VAR REC NM '|' AD '|' CY '|' PH

/* Fixed columns. */
PARSE VAR ROW 1 NAME 21 ADDR 51 PHONE

/* Multi-arg PARSE – peels two call arguments. */
GREET: PROCEDURE
  PARSE ARG NAME, GREETING
  SAY GREETING NAME
  RETURN

/* PARSE UPPER on user command, case-insensitive switch. */
PARSE UPPER VAR LINE VERB REST
SELECT
  WHEN VERB = 'LIST' THEN CALL LIST_HANDLER REST
  WHEN VERB = 'ADD' THEN CALL ADD_HANDLER REST
  OTHERWISE SAY 'unknown verb' VERB
END

```

Not supported

Feature	Status
PARSE LOWER	Not parsed (only UPPER is). Lowercase the source explicitly with LOWER(VAR) then PARSE VAR.
PARSE SOURCE	Not wired — returns empty.
PARSE VERSION	Not wired — returns empty.
PARSE NUMERIC	Not wired.
Regex / pattern matching in templates	REXX has none; use POS / LASTPOS + SUBSTR for ad-hoc splits.

Chapter 18. Conditions and SIGNAL ON

SIGNAL ON {ERROR | NOVALUE | SYNTAX | HALT} [NAME label]

SIGNAL OFF cond

SIGNAL label

SIGNAL ON cond [NAME label] arms a trap. The optional NAME clause picks an explicit handler label; without it, bricks defaults the handler label to the condition name (so SIGNAL ON ERROR jumps to a label named ERROR:).

When a trapped condition fires:

- SIGL is set to the source line of the failing statement.
- Control transfers to the handler label, unwinding active DO / SELECT / PROCEDURE frames first.
- The trap remains armed for the next occurrence.

SIGNAL OFF cond disarms.

Recognised conditions

Condition	Fires on
ERROR	An EXEC CICS or EXEC SQL command returns a non-zero RC (i.e. the bare-string command path under ADDRESS CICS).
NOVALUE	A reference to a simple or compound variable that has never been assigned.
SYNTAX	Any other interpreter error — bad numeric, divide by zero, unknown function, out-of-range subscript, ...
HALT	External halt request. Parsed and reserved; bricks has no fan-in for an external halt today, so the condition never fires.

FAILURE, NOTREADY, and LOSTDIGITS from standard REXX are not recognised — SIGNAL ON FAILURE parses but never fires.

Example

```
SIGNAL ON ERROR NAME CICSERR
SIGNAL ON SYNTAX NAME OOPS
```

```
ADDRESS CICS
EXEC CICS READ FILE('CUSTOMERS') INTO(REC) RIDFLD(CKEY) END-EXEC
EXIT
```

```
CICSERR:
  SAY 'EXEC CICS failed at line' SIGL ' , RC=' RC
  EXIT 12
```

```
OOPS:
  SAY 'Interpreter error at line' SIGL
  EXIT 16
```

Without an armed trap, the legacy “test EIBRESP after every verb” pattern still works. For per-condition (rather than blanket) traps, use `EXEC CICS HANDLE CONDITION` (Chapter 10); the two styles can coexist.

Chapter 19. Built-in functions

The bricks REXX interpreter ships with the functions listed below. A `CALL` to any of these in statement position throws away the return value; the same name in expression position returns the value. Built-in lookups are case-insensitive and take precedence over user-defined `PROCEDURE` names of the same spelling.

Length / index

Signature	Returns
<code>LENGTH(s)</code>	Byte length of <code>s</code> .
<code>POS(needle, hay [, start])</code>	1-based byte position of the first occurrence of <code>needle</code> in <code>hay</code> starting at <code>start</code> (default 1), or 0 if not found.
<code>LASTPOS(needle, hay [, end])</code>	1-based byte position of the last occurrence at or before <code>end</code> (default = end-of-string), or 0.
<code>WORDS(s)</code>	Number of whitespace-delimited words in <code>s</code> .
<code>WORDPOS(phrase, s [, start])</code>	1-based word index where <code>phrase</code> first appears as a sub-sequence of words, or 0.
<code>WORDINDEX(s, n)</code>	1-based character position of the start of word <code>n</code> , or 0 if <code>n</code> is past the end.
<code>WORDLENGTH(s, n)</code>	Length of word <code>n</code> , or 0.
<code>COUNTSTR(needle, hay)</code>	Non-overlapping occurrence count.

Substring / words

Signature	Returns
SUBSTR(<i>s</i> , <i>start</i> [, <i>length</i> [, <i>pad</i>]])	Substring starting at 1-based <i>start</i> . With <i>length</i> , the result is exactly <i>length</i> characters (right-padded with <i>pad</i> , default space). Negative <i>length</i> raises SYNTAX.
LEFT(<i>s</i> , <i>n</i> [, <i>pad</i>])	Leftmost <i>n</i> characters, padded on the right with <i>pad</i> (default space).
RIGHT(<i>s</i> , <i>n</i> [, <i>pad</i>])	Rightmost <i>n</i> characters, padded on the left with <i>pad</i> .
SUBWORD(<i>s</i> , <i>n</i> [, <i>length</i>])	Words <i>n</i> through <i>n+length-1</i> (default: to end). Single space between words in the result.
WORD(<i>s</i> , <i>n</i>)	Word <i>n</i> , or empty string.
DELSTR(<i>s</i> , <i>n</i> [, <i>length</i>])	<i>s</i> with <i>length</i> characters from position <i>n</i> removed (default: through end).
DELWORD(<i>s</i> , <i>n</i> [, <i>length</i>])	<i>s</i> with <i>length</i> words from word <i>n</i> removed.
INSERT(<i>new</i> , <i>target</i> [, <i>n</i> [, <i>length</i> [, <i>pad</i>]]])	Insert <i>new</i> into <i>target</i> after position <i>n</i> (default 0 = prepend), padded to <i>length</i> first.
OVERLAY(<i>new</i> , <i>target</i> [, <i>n</i> [, <i>length</i> [, <i>pad</i>]]])	Overwrite <i>target</i> at position <i>n</i> with <i>new</i> , extending <i>target</i> if needed.
CHANGESTR(<i>needle</i> , <i>hay</i> , <i>repl</i> [, <i>count</i>])	Replace every occurrence (or the first <i>count</i>) of <i>needle</i> with <i>repl</i> .

Whitespace / case

Signature	Returns
STRIP(<i>s</i> [, <i>opt</i> [, <i>ch</i>]])	Trim characters <i>ch</i> (default space) from L (left), T (right), or B (both, default).
SPACE(<i>s</i> [, <i>n</i> [, <i>pad</i>]])	Collapse internal whitespace; insert <i>n</i> copies of <i>pad</i> (default 1 space) between words. <i>n=0</i> removes whitespace entirely.
CENTER(<i>s</i> , <i>n</i> [, <i>pad</i>]) / CENTRE(<i>s</i> , <i>n</i> [, <i>pad</i>])	Centre <i>s</i> in a field of width <i>n</i> ; <i>pad</i> with <i>pad</i> (default space). Truncates equally from both ends when <i>s</i> is longer than <i>n</i> .
JUSTIFY(<i>s</i> , <i>n</i> [, <i>pad</i>])	Right-justify the last word in width <i>n</i> , padding earlier words.
UPPER(<i>s</i>) / LOWER(<i>s</i>)	ASCII case conversion.
REVERSE(<i>s</i>)	Byte-reverse <i>s</i> .
COPIES(<i>s</i> , <i>n</i>)	<i>s</i> repeated <i>n</i> times.
ABBREV(<i>s</i> , <i>prefix</i> [, <i>minlen</i>])	Returns 1 if <i>prefix</i> is a prefix of <i>s</i> with at least <i>minlen</i> characters; else 0.

Translation / bits

Signature	Returns
TRANSLATE(s [, out [, in [, pad]]])	Per-character map from <code>in</code> to <code>out</code> . With no <code>out/in</code> , uppercases. With only <code>out</code> , <code>in</code> defaults to <code>XRANGE()</code> (the full byte range), giving a per-position translation table.
VERIFY(s, ref [, opt [, start]])	1-based position of the first character in <code>s</code> (from <code>start</code>) that is not in <code>ref</code> (<code>opt</code> ='N', default) or that IS in <code>ref</code> (<code>opt</code> ='M'); 0 if none.
COMPARE(s1, s2 [, pad])	1-based position of the first differing byte after pad-equalising lengths with <code>pad</code> (default space); 0 if equal.
XRANGE([from [, to]])	All bytes from <code>from</code> (default '\x00') to <code>to</code> (default '\xFF') inclusive.
BITAND(s1 [, s2 [, pad]]) / BITOR(...) / BITXOR(...)	Bitwise logical operation; lengths equalised with <code>pad</code> .

Conversion

Signature	Returns
C2X(s)	Uppercase hex of every byte. Canonical way to compare <code>EIBAID</code> to a PF-key code: <code>IF C2X(EIBAID) = 'F7' THEN ...</code> (PF7).
X2C(hex)	Decode <code>hex</code> (spaces ignored) to a byte string.
D2X(n [, width])	Decimal integer to uppercase hex, optionally zero-padded to <code>width</code> .
X2D(hex [, n])	Hex to decimal. With <code>n</code> , treats the value as an <code>n</code> -digit signed integer (two's complement when the top bit is set).
C2D(s [, n])	Byte string to decimal. With <code>n</code> , signed two's complement over <code>n</code> bytes.
D2C(n [, len])	Integer to byte string, optionally <code>len</code> bytes wide.
B2X(bits)	Bit string (0/1 chars) to hex.
X2B(hex)	Hex to bit string.

Numeric

Signature	Returns
ABS(n)	Absolute value.
MAX(n, ...)	Maximum of all arguments.
MIN(n, ...)	Minimum of all arguments.
INT(n)	Integer part (truncate toward zero).
TRUNC(n [, places])	Truncate to <code>places</code> decimal positions (default 0).
MOD(a, b)	Modulo. Same value as <code>a // b</code> .

Signature	Returns
<code>SIGN(n)</code>	-1 if <code>n<0</code> , 0 if <code>n=0</code> , +1 if <code>n>0</code> .
<code>FORMAT(n [, before [, after [, exp [, exppt]]])</code>	Numeric formatting with before characters in the integer part and after decimal places. exp / exppt accepted syntactically but ignored.
<code>DIGITS()</code> / <code>FUZZ()</code> / <code>FORM()</code>	Return the current NUMERIC DIGITS / FUZZ / FORM settings.

Type / data

Signature	Returns
<code>DATATYPE(s)</code>	'NUM' if <code>s</code> parses as a number; 'CHAR' otherwise.
<code>DATATYPE(s, 'N')</code>	1 / 0 — is <code>s</code> numeric?
<code>DATATYPE(s, 'W')</code>	1 / 0 — is <code>s</code> a whole number?
<code>DATATYPE(s, 'A')</code>	1 / 0 — is <code>s</code> alphanumeric (letters, digits, no spaces)?

Date / time

Signature	Returns
<code>DATE()</code> / <code>DATE('N')</code>	Today as dd Mmm yyyy.
<code>DATE('S')</code>	Today as yyyyymmdd.
<code>DATE('E')</code>	Today as dd/mm/yy.
<code>DATE('U')</code>	Today as mm/dd/yy.
<code>DATE('O')</code>	Today as yy/mm/dd.
<code>DATE('B')</code>	Days since 0001-01-01.
<code>DATE('B', 'yyyyymmdd', 'S')</code>	Basedate of a given date — subtract two basedates for an exact day delta.
<code>TIME()</code> / <code>TIME('N')</code>	hh:mm:ss.
<code>TIME('L')</code>	hh:mm:ss.uuuuuu with microseconds.
<code>TIME('S')</code>	Seconds since midnight.

Variables / arguments

Signature	Returns
<code>VALUE(name)</code>	Read the value of the variable named at runtime — useful for indirect reads (<code>VALUE('SCR.ROW' J)</code>).
<code>VALUE(name, newval)</code>	Set the variable named at runtime; returns the prior value. The two-argument form usually appears under <code>CALL VALUE 'SCR.ROW' J, LINE.</code>

Signature	Returns
<code>ARG()</code>	Number of arguments the current procedure was called with.
<code>ARG(n)</code>	The <i>n</i> th argument (1-based), or empty if out of range.
<code>RANDOM([min [, max]])</code>	Random integer in [min, max] (default [0, 999]).
<code>ERRORTEXT(code)</code>	Echoes the numeric code as a string (placeholder — bricks does not maintain the standard REXX error-text table).

Stream I/O

The stream functions read and write text files in the `tmp_dir` sandbox — the same backend that powers COBOL's `EXEC CICS READQ TD / WRITEQ TD / DELETEQ TD` (see Chapter 9). The sandbox is strict: ASCII only, LF-terminated, no traversal.

Function	Form	Behaviour
<code>LINEIN(name)</code>	next line	Auto-opens <i>name</i> for read on first call; returns the next line (LF stripped). Empty string at EOF.
<code>LINEIN(name, 1)</code>	rewind	Closes and reopens <i>name</i> , then returns line 1. Only 1 is honoured; any other line number is treated as the “next line” form.
<code>LINEOUT(name)</code>	close	Closes the named stream. Returns 0.
<code>LINEOUT(name, line)</code>	append	Auto-opens <i>name</i> for append on first call; appends <i>line</i> '\n'. Returns 0 on success, 1 on error.
<code>LINES(name)</code>	EOF probe	Returns 1 while more data is available, 0 at EOF.
<code>CHARIN(name, start, n)</code>	byte read	Reads <i>n</i> bytes (default 1). <i>start</i> = 1 rewinds first; other <i>start</i> values are ignored.
<code>CHAROUT(name, s)</code>	byte write	Appends raw bytes. Same ASCII/LF validation as <code>LINEOUT</code> .
<code>CHARS(name)</code>	bytes remaining	Returns the file size minus the current read position.
<code>STREAM(name, 'S')</code>	state	Returns 'READY', 'NOTREADY', or 'ERROR'.

Function	Form	Behaviour
STREAM(name, 'D')	description	Returns a short status string ('OK' / parse-time reason).
STREAM(name, 'C', cmd)	command	OPEN READ / OPEN WRITE / OPEN APPEND / CLOSE / QUERY EXISTS / QUERY SIZE / DELETE.

```

/* Append a row, then read every row back. */
CALL LINEOUT 'export.txt', 'C0000099|WIDGET-Z|7|9.95'
CALL LINEOUT 'export.txt' /* close the writer */
DO WHILE LINES('export.txt') > 0
    SAY LINEIN('export.txt')
END
CALL STREAM 'export.txt', 'C', 'CLOSE'

```

Every handle a program opens is closed automatically at task end; an interpreter that forgets to call CLOSE does not leak descriptors. A sandbox violation ('../etc/passwd', 'sub/x', leading-dot) causes the next LINEIN / LINEOUT to raise an error and STREAM('S') to report 'ERROR'.

Operators

Listed lowest to highest precedence. All operators are left-associative except ** which is right-associative.

Precedence	Operators	Notes
1 (lowest)	\	Logical OR. Strict boolean: both sides must evaluate to 0 or 1. Short-circuit.
2	& / &&	Logical AND. Same strict-boolean rules. && is the strict-AND alias. Short-circuit.
3	= == \= \== <> >< < > <= >=	Comparison. The strict variants (==, \==) compare byte-for-byte; the plain forms compare as numbers when both sides parse as numeric (subject to NUMERIC FUZZ) and as trimmed strings otherwise. \= and <> and >< are all “not equal”.

Precedence	Operators	Notes
4	<code>\ \ </code> and juxtaposition	Concatenation. <code>A \ \ B</code> joins with no separator; <code>A B</code> (whitespace between operands) joins with a single space. There is no zero-separator juxtaposition — write <code>\ \ </code> explicitly.
5	<code>+</code> <code>-</code>	Addition / subtraction.
6	<code>*</code> <code>/</code> <code>%</code> <code>//</code>	Multiplication, real division, integer division (<code>%</code> — truncates toward zero), modulo (<code>//</code> — sign of the dividend).
7	unary <code>+</code> <code>-</code> <code>\</code>	Arithmetic sign and logical NOT.
8 (highest)	<code>**</code>	Exponentiation. Integer exponents are exact (via repeated squaring); fractional exponents use float64.

Comparison precedence note: REXX has no separate equality vs. ordering precedence, so `A < B = C` parses as `(A < B) = C` — comparing the boolean result of `A < B` against `C`.

Restrictions / unsupported

Feature	Status
TRACE (interpretive tracer)	Parsed and accepted; bricks does not emit trace output.
OPTIONS settings (NOVALUE, etc.)	Parsed and accepted; have no effect.
External function libraries / RXFUNCADD	No dynamic linking — every routine must be a PROCEDURE in the program or a built-in.
ADDRESS COMMAND / ADDRESS SYSTEM / ADDRESS TSO	Not wired — only ADDRESS CICS is recognised.
EXECIO and other host-environment I/O	Not provided — use the stream functions (next section) or EXEC CICS READQ TD / WRITEQ TD for file I/O.
PARSE PULL / terminal queue (PUSH / QUEUE)	Statements are parsed; the queue is always empty. Use EXEC CICS RECEIVE for terminal input.
NUMERIC FORM SCIENTIFIC / ENGINEERING	Statement is parsed; rendering is always plain decimal.
Strict IEEE arithmetic	Backed by float64 with NUMERIC DIGITS rounding, so very large factorials and the like will round before they overflow.

Part 2. The COBOL language

Part 2 begins here. Bricks ships a free-form COBOL interpreter (`cobol/`) that sits beside REXX as a second front end on the same EXEC CICS / EXEC SQL / EXEC CICS WEB surface. The chapters that follow describe COBOL source format, the DATA DIVISION, PROCEDURE DIVISION, the EIB binding, the COPY directive (including the standard DFHAID, DFHRESP, SQLCA, and DFHWB* copybooks), and the restrictions. Command-level behaviour is identical between COBOL and REXX — see **Parts 4, 6, and 7** for the verbs.

Chapter 20. COBOL source format

Bricks ships a free-form COBOL interpreter (`cobol/`) that sits beside REXX as a second front end on the same EXEC CICS surface. The language-specific layer is `cobol/frame.go`, which adapts COBOL's group-item world to the REXX-style STEM.TAIL lookup the CICS handlers use.

Source rules

- **Free-form.** No column 1-72 ruling; no Area A / Area B.
- **Hyphens** are allowed in identifiers (CUST-RECORD).
- **Comments** use modern `*>` anywhere on a line, or legacy `*` in column 1.
- **Strings** use `'...'` or `"..."` with quote-doubling for embedded quotes (`'don''t'`, `"say ""hi"""`).
- **Numeric literals** are decimal digit runs, optionally signed (+, -) when they appear in a context that accepts a signed number (VALUE, COMPUTE, arithmetic verbs). There is no COBOL-side decimal-point literal — fractional VALUES use the V in the PIC plus a digit-run interpreted against that scale.
- **Hex literals** like `X'F3'` and `X"7C"` decode to their byte value — used for `IF EIBAID = X'F3'` to check PF3 / PF12 / etc. without `C2X(EIBAID) = 'F3'` round-trips.
- **Identifiers** start with a letter or `_`, continue with letters, digits, `_`, and `-`. Names are case-insensitive (uppercased at parse time).
- **Periods** are scope terminators, not per-statement. The parser requires a period after each division header, section header, paragraph header, and top-level data item; periods inside an explicit `END-IF` / `END-EVALUATE` / etc. body are tolerated as inner punctuation, not as scope terminators.

Divisions

The four divisions parse in their canonical order:

```
IDENTIFICATION DIVISION.  
PROGRAM-ID. HELC.
```

```
ENVIRONMENT DIVISION.      *> optional, must be empty
```

```
DATA DIVISION.  
WORKING-STORAGE SECTION.  
01 SCR.  
05 INFOLINE PIC X(78).
```

```

05 GREETING PIC X(20).

PROCEDURE DIVISION.
MAIN.
    MOVE 'Hello' TO GREETING.
    EXEC CICS SEND MAP('HEL01') FROM(SCR) ERASE END-EXEC.
    EXEC CICS RETURN END-EXEC.
    STOP RUN.

```

LINKAGE SECTION is not yet parsed; DFHCOMMAREA is auto-injected as PIC X(2000) if the program doesn't declare it (see `cobol.ensureSystemItems`), so a sub-program can MOVE DFHCOMMAREA TO key immediately. The dispatcher strips trailing space when reading the COBOL frame's DFHCOMMAREA back out, so a fixed-width buffer round-trips cleanly across an inter-language EXEC CICS LINK.

Chapter 21. DATA DIVISION

The DATA DIVISION declares every data item the program touches. Bricks expects a single WORKING-STORAGE SECTION followed by an optional LINKAGE SECTION (DFHCOMMAREA is auto-injected when the program omits it).

Level numbers

Level	Meaning
01	Top-level data item. Owns a self-contained byte buffer.
02 – 49	Subordinate items inside a group. Children share their parent's buffer at a computed offset.
77	Standalone elementary item (no children). Functionally equivalent to a flat 01.
88	Condition-name attached to the preceding non-88 item (see below).

The level numbers 66 (RENAMES) and 78 (constant) are **not** parsed.

PIC clauses — unedited

Unedited PICs hold raw data: digits, letters, or padded bytes.

Form	Class	Notes
X(n) or XXXX...	Alphanumeric	n bytes; left-justified, space-padded on MOVE. Up to 1 MiB.
A(n) or AAA...	Alphabetic	Same storage as X(n); runtime class-check rejects non-alpha on INSPECT-style tests.

Form	Class	Notes
9(n) or 999...	Numeric (unsigned)	n decimal digits; right-justified, zero-padded. Capped at 18 digits (int64 limit).
S9(n)	Numeric (signed)	Byte 0 holds the sign character (' ' for positive, - for negative); bytes 1..n-1 hold the digits. Cap 18 digits.
9(n)V9(m)	Numeric with implicit decimal	The V is positional — no byte is consumed. Total n+m digits stored; arithmetic uses the fixed-point <code>decimal</code> type so scale survives <code>MOVE / COMPUTE / DISPLAY</code> .
S9(n)V9(m)	Signed scaled numeric	Same as above plus a sign byte.

(n) is a repetition count; 9(3) and 999 are equivalent.

PIC clauses — edited numeric

Edited PICs hold a formatted representation of a numeric value suitable for display. Five edit characters are recognised:

Edit char	Role
Z	Replace a leading zero with a space (zero suppression).
.	Insert a literal decimal point at that column.
,	Insert a thousands separator at that column.
\$	Insert a \$. A <i>single</i> \$ is fixed (always emitted, must be the leftmost character). Two or more in a row <i>float</i> — the leftmost is the symbol and slides right to the position immediately before the first significant digit.
*	Check protection. Replaces a leading zero with * instead of a space — used on cheques to prevent forgery.

Constraints:

- At most one suppression family per PIC: Z, floating \$, and * are mutually exclusive (a single fixed \$ may coexist with Z — e.g. \$ZZ9.99).
- Z and * are forbidden in the fractional half (after the . insertion).
- The literal width of the mask is the PIC length; total digit positions (Z + floating-\$ + * + 9) are capped at 18.

Worked examples:

Source	PIC mask	Stored bytes
42	ZZ9	" 42"
0	ZZ9	" 0"
1.5	ZZZ.99	" 1.50"
1234.56	Z,ZZ9.99	"1,234.56"
12345.67	\$\$\$,\$\$9.99	"\$12,345.67"
0.45	\$\$\$,\$\$\$9.99	" \$.45"
12.34	**,**9.99	"****12.34"
42	\$9999	"\$0042"

When a scaled numeric source is moved through an edited receiver the source's V-scale is honoured: MOVE N TO N-Z where N is PIC 9(5)V99 VALUE 12345.67 and N-Z is PIC ZZ,ZZ9.99 stores "12,345.67".

The remaining ANSI edit characters — +, -, CR, DB, B (blank), 0 (zero insertion), / (slash), and the BLANK WHEN ZERO clause — are not yet supported.

VALUE

VALUE declares the initial content of an elementary data item. Accepted forms:

Form	Example
String literal	VALUE 'ENTER ID'. or VALUE "OK".
Numeric literal	VALUE 42. or VALUE 0. (signed +/- allowed).
Figurative	VALUE SPACES., VALUE ZEROS., VALUE HIGH-VALUES., VALUE LOW-VALUES., VALUE QUOTES.

The figurative aliases SPACE / ZERO / ZEROES / HIGH-VALUE / LOW-VALUE / QUOTE are normalised to their plural form at parse time. VALUE on a group item is rejected — initialise the children individually.

Group items

A 01 (or any subordinate level >01) item with one or more children becomes a group. The group's storage is the concatenation of its children's storage; the group itself owns no extra bytes beyond what its children occupy.

01 PARENT.

05 CHILD PIC X(8).

05 OTHER PIC X(4).

MOVE source TO PARENT fans out to every elementary child; EXEC CICS SEND MAP FROM(PARENT) walks the children by name to look up map-field values.

OCCURS

OCCURS *n* [TIMES] declares an array of *n* copies of an item. The cap is 4096 occurrences per item. Reference one slot with a 1-based subscript expression: *K(I)*, *K(5)*, *K(I + 1)*. Subscript bounds are checked at runtime. **OCCURS** works on elementaries and on group items; nested **OCCURS** (an array of arrays) is **rejected at parse time**.

```
01 TABLE-AREA.  
    05 ROW OCCURS 15 TIMES.  
        10 K PIC X(8).  
        10 N PIC X(28).  
01 I PIC 9(4).  
...  
MOVE 'KEY-A' TO K(1).  
MOVE 'KEY-B' TO K(I).
```

FILLER

FILLER is a reserved name for anonymous storage. Multiple **FILLER** children under the same parent are allowed (they don't collide on the duplicate-name check) and they're never registered in the lookup tables — they can only be addressed through the parent.

USAGE — DISPLAY vs. COMP vs. COMP-3

Numeric items have three storage layouts:

USAGE	Synonyms	Storage
DISPLAY (default)	—	One byte per digit; signed PICs reserve byte 0 for the sign character (' ' / '-'). The historical bricks layout.
COMP	COMPUTATIONAL, COMP-4, BINARY	Big-endian two's-complement signed integer packed into 2, 4, or 8 bytes based on the total digit count.
COMP-3	COMPUTATIONAL-3, PACKED-DECIMAL	Packed decimal: two BCD digits per byte (high nibble first), with the last byte's low nibble carrying the sign.

The **USAGE** keyword is optional — **PIC S9(8) COMP.** and **PIC S9(8) USAGE IS COMP.** both work. **USAGE** may appear before or after the **PIC** clause within the same item.

COMP **byte sizing** follows IBM's halfword / fullword / doubleword convention:

Total digits (PIC 9(<i>n</i>) or S9(<i>n</i>)V9(<i>m</i>) → <i>n+m</i>)	Bytes
1 – 4	2 (halfword)
5 – 9	4 (fullword)

Total digits (PIC 9(n) or S9(n)V9(m) → n+m)	Bytes
10 – 18	8 (doubleword)

USAGE COMP requires a numeric PIC (9 or S9); declaring it on PIC X(n) is rejected at parse time. The signed/unsigned flag does *not* change the byte count — both PIC 9(4) COMP and PIC S9(4) COMP occupy two bytes. Bricks always stores the value as a two’s-complement signed int internally.

Scale survives the format transition: a value moved between a PIC 9(5)V99 DISPLAY field and a PIC S9(8)V99 COMP field preserves the implicit decimal point. Within the interpreter every numeric load goes through the fixed-point **decimal** engine; DISPLAY decodes digits, COMP decodes binary bytes, but both produce the same **decimal{val, scale}** tuple.

COMP-3 byte sizing uses the packed-decimal formula $\text{ceil}((\text{digits} + 1) / 2)$ — digits plus the trailing sign nibble, rounded up:

Total digits	Bytes
1	1
4	3
9	5
15	8
18	10

The sign nibble follows IBM Enterprise COBOL convention: 0xC for signed positive, 0xD for signed negative, 0xF for unsigned. On read, sign nibbles 0xB and 0xD are treated as negative; everything else is positive. Like COMP, COMP-3 requires a numeric PIC and the digit cap is 18 (the int64 fixed-point engine’s ceiling) — PIC S9(19) and wider are parser-rejected.

The canonical mainframe shape PIC S9(15) COMP-3 is the standard CICS EIBABTIME carrier:

```
01 SYS-DT PIC S9(15) COMP-3.
EXEC CICS ASKTIME ABTIME(SYS-DT) END-EXEC.
EXEC CICS FORMATTIME ABTIME(SYS-DT) YYYYMMDD(YYYYMMDD) END-EXEC.
```

ASKTIME writes the 15-digit ms-since-1900 decimal string into SYS-DT; bricks packs it into 8 BCD bytes transparently. FORMATTIME reads the packed bytes back into the same decimal string and parses it.

Other USAGE variants — COMP-1 (single-precision float) and COMP-2 (double-precision float) are explicitly rejected at parse time with a “not yet supported” message. REDEFINES, SYNC, INDEXED BY, and OCCURS DEPENDING ON are also unsupported.

Level 88 condition-names

A level-88 line declares a boolean condition tied to the **preceding non-88 data item**. The condition is true when the parent’s current value matches one of the listed values or falls within a THRU range. Reference as a bare name in any PROCEDURE DIVISION condition (see Chapter 22 — 88-level condition names for the runtime behaviour).


```

01 RESP-CODE PIC 9(3).
   88 RESP-OK          VALUE 0.
   88 RESP-MISSING     VALUE 13.
   88 RESP-RECOVER     VALUES 12, 26, 80.
   88 RESP-FATAL       VALUE 100 THRU 999.

```

VALUES (plural) and VALUE (singular) are interchangeable. Multiple values are separated by spaces or commas. THRU (synonym THROUGH) declares an inclusive range; ranges and single values can be mixed: 88 OK VALUES 0, 1 THRU 5, 9.

A condition-name lives in its own namespace, separate from data items. Declaring an 88 whose name collides with an existing data item is rejected at parse time.

LINKAGE SECTION

LINKAGE SECTION is parsed but minimal: bricks auto-injects DFHCOMMAREA PIC X(2000) when the program doesn't declare it, so a sub-program can MOVE DFHCOMMAREA TO key immediately. The dispatcher strips trailing space when reading the COBOL frame's DFHCOMMAREA back out, so a fixed-width buffer round-trips cleanly across an inter-language EXEC CICS LINK.

Cross-group name collisions — OF / IN qualification

A child name that appears under more than one group is fine; the parser accepts the declaration and tracks the collision in Program.AmbiguousNames. Unqualified access to such a name is a runtime error ("ambiguous reference") — disambiguate with OF (or its synonym IN):

```

01 SCR.
   05 CUSTNO PIC X(8).
01 DET.
   05 CUSTNO PIC X(8).
...
MOVE 'A1234567' TO CUSTNO OF SCR.
MOVE 'B7654321' TO CUSTNO OF DET.
DISPLAY CUSTNO OF SCR.

```

Single-step qualification (X OF Y) picks the matching child anywhere in Y's subtree; multi-step (X OF Y OF Z) chains through nested groups. Bare names that are unique across the program continue to work without qualification — the pre-7 runtime/cobol/gust.cob convention of prefixed child names (DCUSTNO, DNAME, DMSG) still compiles and runs, it just isn't required any more.

A sibling-duplicate (two children of the SAME parent sharing a name) is still a parse-time error — no qualifier can disambiguate between siblings of one group.

Type coercion on MOVE

Source class	Target class	Behaviour
Alphanumeric → Alphanumeric	X / A → X / A	Copy, right-pad with spaces if target is wider; right-truncate if source is wider.

Source class	Target class	Behaviour
Numeric → Numeric	9 / S9 → 9 / S9	Load through the fixed-point decimal engine, rescale to the target's V position, store. Truncates high-order if the integer part doesn't fit (silent unless ON SIZE ERROR is set on the surrounding verb).
Numeric → Edited	9 / S9 → Z / \$ / * / ...	Read the source as a decimal (honouring V-scale), render the value through the edited mask.
Edited → Numeric	Z / \$ / * / ... → 9 / S9	De-edit on read by stripping insertion characters (., ., \$, *, spaces) and re-parsing the digits.
Alphanumeric → Numeric	X → 9	Best-effort loadNumeric — succeeds if the byte content parses as digits after trimming. A letter raises a runtime data exception.
Numeric → Alphanumeric	9 / S9 → X	Source stringified, then alphanumeric-padded.
Anything → Group	* → group item	Fans out to every elementary child (recursively).

Figurative expansion

SPACES → ' ' repeated to fill the target; ZEROS → '0' for numeric targets, \x00 for alphanumeric; HIGH-VALUES → \xFF; LOW-VALUES → \x00; QUOTES → '\'. Group targets compute the fill length from the sum of the elementary children's storage.

Chapter 22. PROCEDURE DIVISION

The PROCEDURE DIVISION is a flat list of paragraphs. Each paragraph is a label (**PARAGRAPH-NAME.**) followed by one or more statements. Paragraphs are visible to **PERFORM** and **GO TO**.

Statement reference

Data movement

Verb	Syntax	Notes
MOVE	MOVE src T0 tgt1, tgt2, ...	One or more targets; fan-out to group children supported. Type coercion table in Chapter 21 — Type coercion.

Verb	Syntax	Notes
INITIALIZE	Not yet supported.	Initialise individual items with MOVE.

Arithmetic Every arithmetic verb accepts the optional clauses **ROUNDED**, **ON SIZE ERROR ... [END-VERB]**, and **GIVING** (where listed). See “**ROUNDED** and **ON SIZE ERROR**” below.

Verb	Syntax
COMPUTE	COMPUTE tgt [ROUNDED] = expr [ON SIZE ERROR ...] [END-COMPUTE]
ADD	ADD a TO b [GIVING c] [ROUNDED] [ON SIZE ERROR ...] [END-ADD]
SUBTRACT	SUBTRACT a FROM b [GIVING c] [ROUNDED] [ON SIZE ERROR ...] [END-SUBTRACT]
MULTIPLY	MULTIPLY a BY b [GIVING c] [ROUNDED] [ON SIZE ERROR ...] [END-MULTIPLY]
DIVIDE (INTO)	DIVIDE a INTO b [GIVING c] [ROUNDED] [ON SIZE ERROR ...] [END-DIVIDE]
DIVIDE (BY)	DIVIDE a BY b GIVING c [ROUNDED] [ON SIZE ERROR ...] [END-DIVIDE]

REMAINDER is not yet supported on **DIVIDE**. Arithmetic expressions support **+**, **-**, *****, **/**, ******, parentheses, and unary sign.

Control flow

Verb	Syntax	Notes
IF	IF cond [THEN] stmts [ELSE stmts] [END-IF]	The period after the last unscoped statement closes the IF when END-IF is omitted.
EVALUATE	EVALUATE subject WHEN val [WHEN val] ... [WHEN OTHER stmts] END-EVALUATE	Simple value form only — EVALUATE TRUE / FALSE are rejected at parse time.
PERFORM	PERFORM para PERFORM para UNTIL cond PERFORM para N TIMES PERFORM para VARYING idx FROM x BY y UNTIL cond	In-line PERFORM ... END-PERFORM (body inline, no paragraph) is not yet supported.
GO TO	GO TO para	Calculated GO TO ... DEPENDING ON not supported.
CONTINUE	CONTINUE	Explicit no-op (used in empty WHEN OTHER etc.).
STOP RUN	STOP RUN (or just STOP)	Terminate the task.
GOBACK	GOBACK	Synonym for EXIT PROGRAM .

Verb	Syntax	Notes
EXIT	EXIT [PROGRAM]	EXIT alone is a no-op; EXIT PROGRAM returns to the caller.

Strings

Verb	Syntax
STRING	STRING src1 [DELIMITED BY (SIZE \ 'lit')] src2 ... INTO tgt [ON OVERFLOW stmts] [END-STRING]
UNSTRING	UNSTRING src DELIMITED BY 'lit' INTO t1 t2 ... [ON OVERFLOW stmts] [END-UNSTRING]
INSPECT TALLYING	INSPECT subject TALLYING counter FOR (ALL \ LEADING \ CHARACTERS) [needle] [BEFORE/AFTER INITIAL delim] [, ...]
INSPECT REPLACING	INSPECT subject REPLACING (ALL \ LEADING \ FIRST \ CHARACTERS) [needle] BY replacement [BEFORE/AFTER INITIAL delim] [, ...]

INSPECT CONVERTING is not yet supported.

I/O and external

Verb	Syntax	Notes
DISPLAY	DISPLAY item1 item2 ...	Concatenates items with no separator; newline after the list.
ACCEPT	Not yet supported.	Use EXEC CICS RECEIVE for terminal input.
EXEC CICS	EXEC CICS verb operand-list END-EXEC	Same verb set as REXX. See Parts 4–5 and 7.
EXEC SQL	EXEC SQL stmt END-EXEC	See Part 6.
CALL, SEARCH, ALTER	Not yet supported.	

Every target name above (the second operand of MOVE, the target of COMPUTE / ADD / SUBTRACT / MULTIPLY / DIVIDE / GIVING / STRING INTO / UNSTRING INTO / INSPECT, and the subject of EVALUATE / IF) accepts qualification via OF / IN (see the Data Division section on globally non-unique names), subscripts via (idx) for OCCURS-resident items, and reference modification via (start:length).

ROUNDED and ON SIZE ERROR

ROUNDED applied to COMPUTE and the four arithmetic verbs rounds the final value half-away-from-zero to the destination's PIC scale instead of truncating toward zero (the unROUNDED default).

ON SIZE ERROR runs its statement list when the calculation overflows the int64 fixed-point engine, the result's integer-part digits don't fit in the target PIC, or a **DIVIDE** divisor is zero; when present, the destination is left untouched on a size-error firing. Without an **ON SIZE ERROR** branch the value is silently truncated/wrapped as before so existing programs aren't disrupted.

PERFORM N TIMES and PERFORM VARYING

```
PERFORM FILL-ROW 15 TIMES.
```

```
PERFORM FILL-ROW  
  VARYING I FROM 1 BY 1 UNTIL I > 15.
```

PERFORM ... N TIMES runs the named paragraph exactly N times; N can be a numeric literal or a numeric data item. **PERFORM ... VARYING idx FROM x BY y UNTIL cond** initialises idx to x, runs the body while cond is false, then increments idx by y between iterations.

OCCURS arrays

```
01 TABLE-AREA.  
  05 ROW OCCURS 15 TIMES.  
    10 K PIC X(8).  
    10 N PIC X(28).  
01 I PIC 9(4).  
...  
MOVE 'KEY-A' TO K(1).  
MOVE 'KEY-B' TO K(I).  
PERFORM SHOW-ROW VARYING I FROM 1 BY 1 UNTIL I > 15.
```

OCCURS n TIMES declares n copies of an item; references add a parenthesised 1-based subscript expression that picks one. The subscript can be any numeric expression — a literal, a data item, or arithmetic — and is bounds-checked at runtime. Subscripts work on group items (**ROW(5)**) and on elementaries inside the **OCCURS** group (**K(5)**) alike. The cap is 4096 occurrences per item; nested **OCCURS** (an **OCCURS** group inside another **OCCURS** group) is rejected at parse time.

Periods

Periods are scope-terminators, not per-statement: a statement inside an **IF ... END-IF** body has no period; only the unscoped statement at the paragraph level does.

Postfix NOT

IBM-style postfix **NOT** (**X NOT = Y**, **X NOT > Y**) is supported in **IF / EVALUATE / PERFORM UNTIL** conditions.

Relational operators

Both symbolic and English-spelled relational operators are accepted in **IF**, **EVALUATE WHEN**, and **PERFORM UNTIL** conditions. The two styles are interchangeable; legacy CICS applications typically use the English spelling.

The optional words **IS**, **THAN**, and **TO** are noise — present or omitted without changing meaning. **NOT** may prefix any operator and appears before or after **IS** (**IF X IS NOT EQUAL TO Y**).

Meaning	Symbolic	English-spelled forms (noise words bracketed)
equal	=	[IS] EQUAL [TO]
not equal	<>, NOT =	[IS] NOT EQUAL [TO]
greater	>	[IS] GREATER [THAN]
not greater ()	NOT >	[IS] NOT GREATER [THAN]
less	<	[IS] LESS [THAN]
not less ()	NOT <	[IS] NOT LESS [THAN]
greater or equal	>=	[IS] GREATER [THAN] OR EQUAL [TO]
less or equal	<=	[IS] LESS [THAN] OR EQUAL [TO]

The four lines below are equivalent — all four compile to the same comparison and run identically:

```
IF WK-MTH = 7          DISPLAY 'JULY'.
IF WK-MTH EQUAL 7      DISPLAY 'JULY'.
IF WK-MTH EQUAL TO 7   DISPLAY 'JULY'.
IF WK-MTH IS EQUAL TO 7 DISPLAY 'JULY'.
```

Class condition tests

X IS [NOT] {NUMERIC | ALPHABETIC | ALPHABETIC-UPPER | ALPHABETIC-LOWER} tests the current **byte contents** of a data item against a character class — not its static **PIC** class. A **PIC 9(4)** field holding **0000** is **NUMERIC**; the same field after a partial move that injects letters is not.

IS is optional noise; **NOT** negates as in any other condition.

```
IF WS-INPUT IS NUMERIC
    PERFORM CALC
ELSE
    DISPLAY 'NON-NUMERIC INPUT'.
IF WS-NAME IS ALPHABETIC
    PERFORM ACCEPT-NAME.
```

ALPHABETIC accepts uppercase letters, lowercase letters, and space. **ALPHABETIC-UPPER** and **ALPHABETIC-LOWER** restrict to their case (plus space). For **PIC S9** signed numerics the byte 0 sign position is allowed to hold ' ', '+', or '-'.

Sign condition tests

X IS [NOT] {POSITIVE | NEGATIVE | ZERO} tests the numeric sign of a numeric expression. The operand must be a numeric data item (**PIC 9...** or **PIC S9...**) or a numeric expression; a sign test on a non-numeric operand is a runtime error rather than a silent false.

```
IF WS-BALANCE IS NEGATIVE
    DISPLAY 'OVERDRAWN'.
```

```
IF AMOUNT IS ZERO
    PERFORM SKIP-ENTRY.
```

Signed zero is neither POSITIVE nor NEGATIVE; an unsigned PIC (PIC 9... without S) can never be NEGATIVE — that short-circuits to false without inspecting the value.

88-level condition names

A level-88 line in DATA DIVISION declares a boolean condition tied to the **preceding non-88 data item**. The condition is true when the parent's current value matches one of the listed values or falls within a THRU range. Reference as a bare name in any PROCEDURE DIVISION condition.

```
DATA DIVISION.
WORKING-STORAGE SECTION.
01 RESP-CODE PIC 9(3).
    88 RESP-OK          VALUE 0.
    88 RESP-MISSING     VALUE 13.
    88 RESP-RECOVER     VALUES 12, 26, 80.
    88 RESP-FATAL       VALUE 100 THRU 999.
PROCEDURE DIVISION.
MAIN.
    EXEC CICS READ FILE('CUSTOMER')
                INTO(REC) RIDFLD(KEY)
                RESP(RESP-CODE) END-EXEC.

    IF RESP-OK
        CONTINUE
    ELSE IF RESP-RECOVER
        PERFORM RETRY
    ELSE IF RESP-FATAL
        PERFORM ABEND.
```

VALUES (plural) and VALUE (singular) are interchangeable. Multiple values may be separated by commas or by spaces. THRU (synonym THROUGH) declares an inclusive range; ranges and single values can be mixed in one declaration: 88 OK VALUES 0, 1 THRU 5, 9.

Comparison style follows the parent's PIC class — numeric parents compare numerically, alphanumeric parents compare byte-by-byte (after rstripping trailing spaces). NOT and the AND/OR connectives compose normally: IF NOT RESP-OK OR RESP-FATAL.

A condition-name lives in its own namespace, separate from data items — 01 STATUS PIC X. and 88 STATUS-OK VALUE 'Y'. are two different lookups. Declaring an 88 whose name collides with an existing data item is rejected at parse time.

Intrinsic functions

The shipped intrinsic library is:

Function	Args	Result
FUNCTION UPPER-CASE(s)	1 alphanumeric	s uppercased.
FUNCTION LOWER-CASE(s)	1 alphanumeric	s lowercased.

Function	Args	Result
FUNCTION TRIM(s)	1 alphanumeric	s with leading and trailing spaces removed (both ends; one-arg form).
FUNCTION REVERSE(s)	1 alphanumeric	s reversed (rune-aware).
FUNCTION LENGTH(s)	1 alphanumeric	numeric byte length of the operand's storage (PIC X(8) → 8).
FUNCTION NUMVAL(s)	1 alphanumeric	numeric value of s after stripping leading / trailing whitespace; errors on non-numeric content (does not silently coerce to zero).
FUNCTION POS(needle, haystack)	2 alphanumeric	1-based byte position of needle in haystack, or 0 when not found. Mirrors REXX's POS(). Empty needle returns 0.

Numeric-returning intrinsics (LENGTH, NUMVAL, POS) participate in IF / EVALUATE / arithmetic comparisons as numeric operands, so IF FUNCTION POS(NEEDLE, HAY) > 0 works without an intermediate COMPUTE. The remaining intrinsics return alphanumeric and compare by bytes (rstripped).

FUNCTION POS is the substring-search hook used by runtime/cobol/gusl.cob to filter the customers file when the operator types a search term at GUST's S action.

INSPECT

```
INSPECT subject TALLYING counter FOR (ALL | LEADING | CHARACTERS)
    [needle] [BEFORE/AFTER INITIAL delim]
    [, ...]
INSPECT subject REPLACING (ALL | LEADING | FIRST | CHARACTERS)
    [needle] BY replacement
    [BEFORE/AFTER INITIAL delim]
    [, ...]
```

Quantifiers:

- **ALL** — every occurrence of *needle* (TALLYING) or every occurrence rewritten (REPLACING).
- **LEADING** — only successive occurrences at the start of the in-scope region.
- **FIRST** — only the first occurrence (REPLACING only).
- **CHARACTERS** — every character in the in-scope region; for TALLYING this just counts characters, for REPLACING it overwrites each one with the first byte of *replacement*.

Limiters:

- **BEFORE INITIAL delim** narrows the in-scope region to everything before the first occurrence of *delim*.
- **AFTER INITIAL delim** narrows to everything after the first occurrence of *delim*.

Both can be set on the same phrase; AFTER picks the start, BEFORE picks the end.

Multiple phrases can be combined per statement, optionally separated by commas:

```
INSPECT REC REPLACING ALL ',' BY '|' ALL ';' BY '/'.
INSPECT REC TALLYING C FOR LEADING '0', ALL 'X' AFTER INITIAL ' '.
```

Counter behaviour: TALLYING **accumulates** — successive INSPECT TALLYING statements add to the existing counter value rather than resetting, matching IBM semantics. REPLACING phrases run left-to-right in declared order, so later phrases see the output of earlier ones.

Subject, counter, and operands all accept qualified data-names (INSPECT REC OF DET ...). Trailing PIC X padding is rstripped from both the subject and the needle before matching, so a 30-byte filter containing F00 plus trailing spaces matches against the meaningful content of the subject rather than against the padding.

IBM's CONVERTING form is not parsed yet.

EVALUATE limitations

EVALUATE only supports the simple value form. EVALUATE TRUE / EVALUATE FALSE is rejected at parse time with a hint to rewrite as IF/ELSE — those forms (and WHEN ... THRU ... ranges, multi-subject ALSO clauses, condition-name arms) are deferred.

Chapter 23. The EIB block in COBOL

EIBRESP, EIBRESP2, EIBAID, EIBCP0SN, EIBCALEN, EIBTRMID, RC, and DFHCOMMAREA are auto-injected if not declared, so most programs need no boilerplate. After every EXEC CICS, EIBRESP and EIBRESP2 are populated by the handler the same way they are for REXX. The legacy “test EIBRESP after every verb” pattern is the most common idiom; for non-local error handling without a REXX-style SIGNAL ON ERROR, use EXEC CICS HANDLE CONDITION / EXEC CICS HANDLE ABEND (Chapter 10).

See Chapter 11 for the field meanings.

Chapter 24. EXEC CICS in COBOL

The COBOL parser collects every token between EXEC CICS and END-EXEC and reconstructs the body verbatim for the same cics.ParseCommand REXX uses. Three consequences:

- **Map-field names must match group-child names exactly.** When EXEC CICS SEND MAP('CUST1') FROM(SCR) fires, the CICS handler asks the frame for SCR.INFOLINE, SCR.MSG, etc. — so SCR must have children spelled exactly as the map declares fields. Real COBOL solves this with BMS-generated copybooks; bricks doesn't ship those, so the operator declares fields by hand.
- **DFHCOMMAREA is the COMMAREA marshalling slot.** Caller passes bytes in via the frame; sub-program reads with MOVE DFHCOMMAREA TO ..., mutates a working copy, and MOVE ... TO DFHCOMMAREA. Trailing space is stripped by the dispatcher on the way back out.

- **Command-line arguments arrive via EXEC CICS RECEIVE INTO(...).** When the operator types EXAM 1 2 3 at the blank prompt, the dispatcher hands the unedited line (TRANSID prefix included) to the first task in the chain. The program reads it with EXEC CICS RECEIVE INTO(WS-INPUT) LENGTH(WS-LEN) END-EXEC and parses with UNSTRING WS-INPUT DELIMITED BY ' ' INTO TRANSID A B C END-UNSTRING. Single-shot per task, then EOC. See runtime/cobol/exam.cob.

EXEC CICS SEND TEXT FROM(area) [ERASE] works the same way. Reconstruction routes through the shared `cics.Handler`, so any verb REXX can issue, COBOL can issue too. The body is laid out as a flat row-major buffer (every cols bytes = one row, default 80) — 3270 has no LF, so build area as a group of PIC X(80) children and fill them row by row, or compose a single PIC X(n*80) with a matching MOVE per row.

The full EXEC CICS verb set is documented in Part 2; behaviour is identical between REXX and COBOL.

Chapter 25. Copybooks

A copybook is a fragment of COBOL source — typically declarations for constants and groups — kept in a separate file and pulled into a program at parse time with a single line:

```
WORKING-STORAGE SECTION.
COPY DFHAID.
COPY DFHRESP.
01 OWN-COUNTER PIC 9(4).
```

bricks expands every COPY name. directive at the source-text level before the lexer runs, so the rest of the parser sees one unbroken stream of COBOL.

The COPY directive

Syntax:

```
COPY <name>.
```

- The directive must be on its own line. Anything before or after the COPY keyword on the same line is a parse error.
- <name> is the basename of a copybook file; leave the extension off. The lookup tries <name>.cpy, then <name>.cbl, then the exact <name>, all case-insensitive.
- The trailing period is required.
- Leading whitespace is tolerated, and an *> inline comment on the same line is fine: COPY DFHAID. *> 3270 AID bytes.
- The directive is case-insensitive (COPY, Copy, copy all work), and so is the name (COPY DFHAID. and copy dfhaid. resolve to the same file).

Where bricks searches

The search directory is configured by `copybook_dir=` in `bricks.cnf`. The default is `runtime/cobolcopy/` (auto-derived from `runtime_dir` when that line is set). The standard CICS copybooks ship under this directory, so a fresh install already has `DFHAID.cpy` and `DFHRESP.cpy` available.

Copybook names are restricted to a flat alphanumeric namespace: A-Z, a-z, 0-9, -, _, and .. Names containing a path separator, a .. sequence, or a leading dot are rejected before any filesystem access. A copybook itself may contain COPY directives; nesting is capped at depth 5 and cycles are detected explicitly.

Delivered copybook: DFHAID

COPY DFHAID. brings in the standard 3270 attention-identifier (AID) byte constants. Every AID is declared **twice**, once under the friendly bricks mnemonic and once under the IBM-traditional DFH alias — both names resolve to the same byte, and a program may use either style or mix them.

bricks mnemonic	IBM alias	AID byte	Key
ENTER	DFHENTER	X'7D'	Enter
CLEAR	DFHCLEAR	X'6D'	Clear
PA1	DFHPA1	X'6C'	PA1
PA2	DFHPA2	X'6E'	PA2
PA3	DFHPA3	X'6B'	PA3
PF01	DFHPPF1	X'F1'	PF1
PF02	DFHPPF2	X'F2'	PF2
PF03	DFHPPF3	X'F3'	PF3
PF04	DFHPPF4	X'F4'	PF4
PF05	DFHPPF5	X'F5'	PF5
PF06	DFHPPF6	X'F6'	PF6
PF07	DFHPPF7	X'F7'	PF7
PF08	DFHPPF8	X'F8'	PF8
PF09	DFHPPF9	X'F9'	PF9
PF10	DFHPPF10	X'7A'	PF10
PF11	DFHPPF11	X'7B'	PF11
PF12	DFHPPF12	X'7C'	PF12
PF13	DFHPPF13	X'C1'	PF13
PF14	DFHPPF14	X'C2'	PF14
PF15	DFHPPF15	X'C3'	PF15
PF16	DFHPPF16	X'C4'	PF16
PF17	DFHPPF17	X'C5'	PF17
PF18	DFHPPF18	X'C6'	PF18
PF19	DFHPPF19	X'C7'	PF19
PF20	DFHPPF20	X'C8'	PF20
PF21	DFHPPF21	X'C9'	PF21
PF22	DFHPPF22	X'4A'	PF22
PF23	DFHPPF23	X'4B'	PF23
PF24	DFHPPF24	X'4C'	PF24

The bricks mnemonic uses uniform-width PF01..PF24; the IBM alias keeps the no-leading-zero form (DFHPPF1..DFHPPF24) everyone recognises from real CICS code.

Delivered copybook: DFHRESP

COPY DFHRESP. brings in the EXEC CICS condition-code constants for EIBRESP. Each code is declared twice as well — RESP-X mnemonic and DFHRESP-X traditional alias. Numeric values match the runtime’s emitter table (cics/resp.go), so any code bricks returns from a verb has a named constant here.

The most useful entries:

bricks mnemonic	IBM alias	EIBRESP	Condition
RESP-NORMAL	DFHRESP-NORMAL	0	success
RESP-ERROR	DFHRESP-ERROR	1	generic error
RESP-EOC	DFHRESP-EOC	6	end-of-chain / nothing to RECEIVE
RESP-NOTFND	DFHRESP-NOTFND	13	record / item / file not found
RESP-DUPREC	DFHRESP-DUPREC	14	duplicate record on WRITE
RESP-DUPKEY	DFHRESP-DUPKEY	15	duplicate key in browse
RESP-INVREQ	DFHRESP-INVREQ	16	invalid request (bad args, sandbox violation)
RESP-IOERR	DFHRESP-IOERR	17	underlying I/O failure
RESP-NOSPACE	DFHRESP-NOSPACE	18	TS / TD queue full
RESP-NOTOPEN	DFHRESP-NOTOPEN	19	file not open / dataset not enabled
RESP-ENDFILE	DFHRESP-ENDFILE	20	end of browse
RESP-LENGERR	DFHRESP-LENGERR	22	length mismatch
RESP-QZERO	DFHRESP-QZERO	23	TS / TD queue empty (READQ TD returns this)
RESP-ITEMERR	DFHRESP-ITEMERR	26	TS item number out of range
RESP-PGMIDERR	DFHRESP-PGMIDERR	27	LINK / XCTL program not found
RESP-MAPFAIL	DFHRESP-MAPFAIL	36	RECEIVE MAP with no input
RESP-INVMPSZ	DFHRESP-INVMPSZ	38	map size mismatch
RESP-QIDERR	DFHRESP-QIDERR	44	TS queue id unknown
RESP-ROLLEDBACK	DFHRESP-ROLLEDBACK	82	unit-of-work rolled back

The full list (every code bricks emits) is in runtime/cobolcopy/DFHRESP.cpy. Open the file to see the codes not summarised above (RESP-ILLOGIC, RESP-SIGNAL, RESP-QBUSY, the various RESP-INV... codes, RESP-SYSIDERR, etc.).

Delivered copybook: SQLCA

COPY SQLCA. brings in named constants for SQLCODE and the most common SQLSTATE values, so embedded-SQL programs can write:

```
COPY SQLCA.
...
EXEC SQL SELECT name INTO :NM
        FROM customers WHERE id = :CUSTID END-EXEC.
EVALUATE SQLCODE
    WHEN SQL-OK                PERFORM SHOW-ROW
    WHEN SQL-NODATA            PERFORM SHOW-NOT-FOUND
    WHEN SQL-MULTIPLEROWS      PERFORM SHOW-AMBIGUOUS
    WHEN OTHER                  PERFORM SHOW-SQL-ERROR
END-EVALUATE.
```

The SQLCA fields themselves (SQLCODE, SQLSTATE, SQLERRMC) are auto-injected by the bricks COBOL parser — no COPY needed for the data items. SQLCA only adds the constants programs compare against.

bricks mnemonic	IBM alias	SQLCODE	Meaning
SQL-OK	DB2-SUCCESS	0	success
SQL-NODATA	DB2-NOTFOUND	+100	no row / end-of-cursor
SQL-NOTFND	DB2-NOTFOUND	+100	alias
SQL-NOCONFIG	—	-1	SQL not configured in bricks.cnf
SQL-DDLREJECTED	—	-2	DDL rejected (CEDA-only)
SQL-GENERIC	—	-100	generic PG error (see SQLERRMC + log)
SQL-MULTIPLEROWS	DB2-DUPLICATE	-811	SELECT INTO returned >1 row

SQLSTATE constants are PIC X(5) and compare directly:

```
IF SQLSTATE = SQLSTATE-DUP-KEY THEN
    MOVE 'Already exists.' TO MSG
END-IF.
```

Constant	SQLSTATE	Class
SQLSTATE-OK	00000	success
SQLSTATE-NODATA	02000	no-data
SQLSTATE-WARNING	01000	warning
SQLSTATE-NOT-NULL	23502	NOT NULL constraint violated
SQLSTATE-FK-VIOL	23503	foreign-key constraint violated
SQLSTATE-UQ-VIOL	23505	UNIQUE constraint violated

Constant	SQLSTATE	Class
SQLSTATE-DUP-KEY	23505	alias
SQLSTATE-CHK-VIOL	23514	CHECK constraint violated
SQLSTATE-CONN-FAIL	08001	unable to connect to PG
SQLSTATE-CONN-LOST	08006	connection dropped mid-flight
SQLSTATE-INV-AUTH	28000	PG authentication failed
SQLSTATE-INV-PWD	28P01	wrong password
SQLSTATE-SYNTAX	42601	SQL syntax error
SQLSTATE-UNDEF-TBL	42P01	table doesn't exist
SQLSTATE-UNDEF-COL	42703	column doesn't exist
SQLSTATE-UNDEF-FN	42883	function doesn't exist
SQLSTATE-AMBIG-COL	42702	ambiguous column reference
SQLSTATE-INSUF-PRIV	42501	permission denied

The bricks SQL executor also logs every non-zero `SQLCODE`'s full PG error text via `brickslog.Sys` (one line on the operator console + the per-run log file). Programs render only the short mnemonic-driven message on the 3270 screen; the diagnostic detail lives in the log so PG hostnames / port numbers / wrapper text don't leak to terminal users.

Idiom: replacing raw literals

Before — opaque hex and bare integers, easy to mis-key:

```

IF EIBAID = X'F3' THEN
    MOVE 'Y' TO EXIT-FLAG
END-IF.
IF EIBRESP = 13 THEN
    MOVE 'Customer not found.' TO MSG
END-IF.
IF EIBRESP = 23 THEN          *> TD queue empty
    MOVE 'Y' TO DONE-FLAG
END-IF.
```

After — names a reader can scan:

```

WORKING-STORAGE SECTION.
COPY DFHAID.
COPY DFHRESP.
...
IF EIBAID = PF03 THEN          *> or DFHPF3 if you prefer
    MOVE 'Y' TO EXIT-FLAG
END-IF.
IF EIBRESP = RESP-NOTFND THEN
    MOVE 'Customer not found.' TO MSG
END-IF.
IF EIBRESP = RESP-QZERO THEN
    MOVE 'Y' TO DONE-FLAG
END-IF.
```

Live examples: `qagc.cob` (PF03 cancel branch), `gusl.cob` (EVALUATE EIBRID with PF03 / PF07 / PF08 / PF12), `ordr.cob` (RESP-QZERO and RESP-DUPREC against READQ TD / WRITE FILE), `exam.cob` (RESP-EOC after RECEIVE). All under `runtime/cobol/`.

Restrictions

The bricks COPY preprocessor is deliberately minimal. The following real-CICS extensions are **not** supported:

- `COPY ... REPLACING ==X== BY ==Y==`. — token substitution. Rare in modern code; deferred until a concrete use case appears.
- `COPY xyz OF DFHCOB.` / `COPY xyz IN libname.` — library qualifier. bricks has one search root per process.
- `COPY xyz SUPPRESS.` — suppress copybook listing. bricks produces no listing to suppress.
- Embedded directives: multiple statements on one line including a `COPY` are rejected. The line must be `COPY <name>.` and nothing else.

If a copybook is edited, programs that include it will pick up the new contents only after the program's own source mtime changes (the program cache keys by the program path + mtime, not the copybook's). A trivial way to force a reload is to touch the COBOL file:

```
touch runtime/cobol/qagc.cob
```

Or use `CEMT P C` from a TSO/3270 session to flush the entire program cache.

Part 6. EXEC SQL command reference

Part 6 begins here. The chapter that follows is the complete EXEC SQL surface — identical for COBOL and REXX. It covers every embedded-SQL verb Bricks supports, the SQLCA layout and SQLCODE catalogue, the `WHenever` declarative error handler, cursor lifecycle, per-task `CONNECT TO`, null indicators, and SYNCPOINT integration with the bricks unit of work.

Chapter 26. Embedded SQL (COBOL and REXX)

Programming Note. Although this chapter was originally written against COBOL, every EXEC SQL shape and behaviour described here applies identically to REXX. The REXX-specific notes appear later in the chapter under *REXX equivalent*. Programs in either language read and write the SQLCA fields (`SQLCODE`, `SQLSTATE`, `SQLERRMC`) the same way the COBOL samples do.

bricks accepts the classic IBM-style `EXEC SQL ... END-EXEC` embedded-SQL surface and runs it against the Postgres connection configured via `db_*` in `bricks.cnf` (see SQL support — Phase 1). Programs read and write rows through prepared statements; the SQLCA fields `SQLCODE`, `SQLSTATE`, `SQLERRMC` are auto-injected into every program's DATA DIVISION so the program can test them without declaring them.

Scope: single-row CRUD (SELECT INTO, INSERT, UPDATE, DELETE, COMMIT, ROLLBACK), the four-verb cursor lifecycle (DECLARE / OPEN / FETCH / CLOSE), per-task CONNECT TO 'name' database switching, null indicators (:hostvar :indicator), and the WHENEVER declarative error-handling directive. The same surface works identically in COBOL and REXX.

Format

```
EXEC SQL <statement> END-EXEC.
```

Where <statement> is any of the supported verbs (below). Host variables are written with a leading colon: :CUSTID, :NM, :BALANCE. The variable name must already exist in DATA DIVISION — the SQL executor reads its current value (for INPUT bindings) and writes back (for SELECT INTO targets).

Supported verbs

Verb	Notes
SELECT cols INTO :v, :v FROM ...	Single-row read. Returns +100 (no-data) when 0 rows, -811 when >1 row.
INSERT INTO t (...) VALUES (...)	Host-variable references bound as \$N placeholders.
UPDATE t SET ... WHERE ...	Returns RESP-NORMAL even when 0 rows match (matches DB2).
DELETE FROM t WHERE ...	Same row-count semantics as UPDATE.
COMMIT WORK	Commits the per-task PG transaction. Also fired by EXEC CICS SYNCPOINT.
ROLLBACK WORK	Rolls back. Also fired by EXEC CICS SYNCPOINT ROLLBACK.

DDL (CREATE DATABASE, DROP DATABASE, CREATE USER, DROP USER, ALTER ROLE, CREATE TABLESPACE, ...) is **rejected** with SQLCODE = -2 and SQLSTATE = 42501. CEDA DATABASE owns those operations and audits each one through brickslog.Audit. CREATE TABLE / ALTER TABLE / similar non-privileged DDL is not blocked but is also unsupported in the executor's statement classification — porters should run schema migrations with `psql`.

SQLCODE catalog

bricks adopts the IBM DB2 convention adapted to Postgres. The catalog below covers what programs typically test:

SQLCODE	SQLSTATE	Meaning
0	00000	Success.
+100	02000	No data: SELECT INTO returned 0 rows, or an update/delete affected 0 rows.

SQLCODE	SQLSTATE	Meaning
-1	empty	No database configured (no <code>db_*</code> lines in <code>bricks.cnf</code>).
-2	42501	DDL rejected (CEDA owns CREATE/DROP DATABASE
-100	varies	Generic SQL failure — PG returned an error not specifically mapped.
-811	21000	SELECT INTO returned more than one row.

`SQLERRMC` always carries the human-readable message from Postgres (truncated to 70 chars to match the PIC X(70) auto-injection). When the error path includes a specific `SQLSTATE`, that 5-char code lands in `SQLSTATE`; otherwise `SQLSTATE` is the relevant standard code (`00000` on success, `02000` on no-data, `HV000` when the failure didn't carry a state).

SYNCPOINT integration

Each task lazily begins a Postgres transaction on its first `EXEC SQL` that touches data. The transaction commits when:

- `EXEC CICS SYNCPOINT` runs.
- `EXEC SQL COMMIT WORK` runs.
- The task ends cleanly (`RETURN`, `GOBACK`, `STOP RUN`).

And rolls back when:

- `EXEC CICS SYNCPOINT ROLLBACK` runs.
- `EXEC SQL ROLLBACK WORK` runs.
- The task abends via `EXEC CICS ABEND`.

Postgres-side commit fires *before* the bricks-side bbolt journal commit. That ordering avoids a hung PG commit leaving the bricks journal already finalised; failures during the SQL commit surface as `ROLLEDBACK` from `SYNCPOINT`, and the bricks journal then rolls back too.

Idiomatic test

```
EXEC SQL
  SELECT name INTO :NM
  FROM customers_sql
  WHERE id = :CUSTID
END-EXEC.
```

```
EVALUATE SQLCODE
  WHEN RESP-NORMAL   PERFORM SHOW-NAME      *> +0
  WHEN RESP-NOTFND   PERFORM SHOW-NOTFOUND  *> wait -- that's an EIBRESP code; SQLCODE uses RE
  WHEN OTHER         PERFORM SHOW-SQL-ERR
END-EVALUATE.
```

The `RESP-NORMAL` constant (from `COPY DFHRESP`) is value `0`, the same as the `SQLCODE-OK` value, so it conveniently doubles as the success test for both `EIBRESP` (`EXEC CICS`) and `SQLCODE` (`EXEC SQL`). For the no-data case, test `SQLCODE = +100` directly — there's no `DFHSQLCODE` copybook yet.

Worked example: SQLD

`runtime/cobol/sql.d.cob` (`TRANSID SQLD`) is the bundled demonstration. It reads a single `customers_sql` row by id and renders the name + `SQLCA` verdict on screen `SQLD1`. The expected schema is documented in the program header. The operator runs `psql` once to seed the table:

```
CREATE TABLE customers_sql (  
    id    text PRIMARY KEY,  
    name text NOT NULL  
);  
INSERT INTO customers_sql VALUES  
    ('K001', 'Alice'),  
    ('K002', 'Bob'),  
    ('K003', 'Carol');
```

Then `bricks` → sign on → `SQLD` → type `K001` → ENTER. The operator sees `Alice` in the `NM` cell with `SQLCODE = 0` and `SQLSTATE = 00000`.

REXX equivalent

REXX programs use the same `EXEC SQL` shape as COBOL, with `:name` binding REXX variables. There's no `DATA DIVISION` analog – REXX is dynamically typed – so `SQLCODE`, `SQLSTATE`, `SQLERRMC` arrive as ordinary REXX variables the program reads after each statement:

```
ADDRESS CICS
```

```
CUSTID = 'K001'
```

```
EXEC SQL
```

```
    SELECT name INTO :NM
```

```
    FROM customers_sql
```

```
    WHERE id = :CUSTID
```

```
END-EXEC
```

```
IF SQLCODE = 0 THEN
```

```
    SAY 'Customer:' NM
```

```
ELSE IF SQLCODE = 100 THEN
```

```
    SAY 'Not found'
```

```
ELSE
```

```
    SAY 'SQL error' SQLCODE ':' SQLERRMC
```

Behind the scenes the REXX preprocessor (`rexx.PreprocessExecCICS`) rewrites the `EXEC SQL` block into a sentinel bare-string that `runCommand` recognises and routes to the same per-task SQL executor COBOL uses. The two language surfaces are parity-clean: the executor doesn't know or care which language drove it. Live sample: `runtime/rexx/sqlr.rexx`, `TRANSID SQLR`.

REXX stem-tail gotcha. Classic REXX resolves `STEM.NAME` by looking up the simple symbol

NAME and using its value as the tail. If a program does `EXEC SQL ... INTO :NM` then writes `SCR.NM = NM`, the LHS resolves to `SCR.<value-of-NM> = SCR.Alice` — the value lands in the wrong slot. Workaround: pick a host-var name that is NOT also a stem tail. The bundled SQLR uses `CUSTNM` instead of `NM` for exactly this reason.

WHENEVER (declarative error handling)

Instead of testing `SQLCODE` after every statement, a program can declare a standing rule with `EXEC SQL WHENEVER`:

```
EXEC SQL WHENEVER SQLERROR  GO TO SQL-ERROR-EXIT END-EXEC.
EXEC SQL WHENEVER NOT FOUND GO TO NO-MORE-ROWS  END-EXEC.
```

```
EXEC SQL SELECT name INTO :NM FROM customers_sql
        WHERE id = :CUSTID END-EXEC.
```

**> no SQLCODE test here -- WHENEVER handles it*

Three conditions, checked after every subsequent `EXEC SQL`:

Condition	Fires when
SQLERROR	SQLCODE is negative
NOT FOUND	SQLCODE = +100
SQLWARNING	SQLWARN0 = 'W' (bricks rarely sets this — PG errors rather than warns)

Each condition takes one of two actions:

- `CONTINUE` — ignore the condition, fall through (the default for any condition never declared).
- `GO TO label / GOTO label` — branch to the paragraph (COBOL) or label (REXX). The branch unwinds any enclosing `PERFORM` exactly as an explicit `GO TO` would.

A later `WHENEVER` for the same condition replaces the earlier one (e.g. declare `GO TO` for a block of statements, then `CONTINUE` to resume manual handling).

REXX uses the identical syntax; the branch is a `SIGNAL` to the named label:

```
EXEC SQL WHENEVER SQLERROR GOTO SQLERR END-EXEC
EXEC SQL SELECT name INTO :CUSTNM FROM customers_sql
        WHERE id = :CUSTID END-EXEC
SAY 'customer:' CUSTNM
EXIT
SQLERR:
SAY 'SQL failed, SQLCODE' SQLCODE
EXIT
```

Scoping caveat. bricks's `WHENEVER` is *execution-order* scoped: the directive set by the most recently **executed** `EXEC SQL WHENEVER` governs later statements. Real DB2 scopes *lexically* (by source position) because its precompiler inlines a `SQLCODE` test after every statement. For the dominant pattern — one `WHENEVER` near the top of the program — the two are identical. Programs that re-declare `WHENEVER` inside conditional branches see execution-order semantics; keep `WHENEVER` declarations at the top of a paragraph to avoid surprises.

Cursors (DECLARE / OPEN / FETCH / CLOSE)

Multi-row result sets use the four-verb cursor lifecycle. The executor parks one `*sql.Rows` per cursor per task; cursors are closed automatically on `COMMIT`, `ROLLBACK`, or task end so a program that forgets `CLOSE` doesn't pin Postgres MVCC.

```
EXEC SQL DECLARE C1 CURSOR FOR
    SELECT id, name FROM customers_sql ORDER BY id
END-EXEC.
```

```
EXEC SQL OPEN C1 END-EXEC.
```

```
PERFORM UNTIL SQLCODE = 100
    EXEC SQL FETCH C1 INTO :ID, :NM END-EXEC
    IF SQLCODE = 0 THEN
        DISPLAY 'row: ' ID ' ' NM
    END-IF
END-PERFORM.
```

```
EXEC SQL CLOSE C1 END-EXEC.
```

REXX uses the same four verbs (the preprocessor routes them all through the same executor):

```
EXEC SQL DECLARE C1 CURSOR FOR SELECT id, name FROM customers_sql END-EXEC
EXEC SQL OPEN C1 END-EXEC
DO FOREVER
    EXEC SQL FETCH C1 INTO :ID, :CUSTNM END-EXEC
    IF SQLCODE = 100 THEN LEAVE
    SAY 'row:' ID CUSTNM
END
EXEC SQL CLOSE C1 END-EXEC
```

End-of-data is `SQLCODE = +100`. A closed cursor stays in the registry, so `OPEN c1` can rewind it without `re-DECLARE`.

Database binding (transactions.conf 5th field)

Every transaction starts on exactly one database. Which one is fixed by the 5th colon-separated field of its row in `runtime/transactions.conf`:

```
BANK:cobol:bank.cob:public,users,admin:bank
```

The grammar of the file is therefore:

```
transid:lang:program[:groups[:database]]
```

- **database** – name of a row in `runtime/databases.conf`. The value is case-sensitive and must match the row's first column exactly. Whitespace around the name is trimmed.
- **Absent 5th field** – the transaction binds to the **first** row of `databases.conf`. The first row is the deployment-wide default; CEDA DATABASE labels it (`def`) and refuses `Delete` on it.
- **Unknown name** – the dispatcher logs the misbinding at task start and the first `EXEC SQL` returns `SQLCODE = -1` with an empty `SQLSTATE`.

A transaction whose program never runs EXEC SQL may leave the field empty – the implicit default never gets touched. A transaction whose program **does** run EXEC SQL against a non-default database **must** list the binding explicitly: omitting it sends every statement to the default pool, which typically does not host the requested table. The operator-visible symptom is a flood of paired errors:

```
SQL error SQLCODE=-204 SQLSTATE=42P01: relation "accounts" does not exist
```

```
SQL error SQLCODE=-100 SQLSTATE=25P02: current transaction is aborted, commands ignored until end of transaction
```

The -204 is the first statement landing on the wrong pool; the -100 / 25P02 cascade is Postgres marking the per-task PG transaction aborted, so every follow-up statement under a **WHENEVER SQLERROR CONTINUE** declaration logs the same pair until SYNCPOINT runs at task end. Fix: add the correct 5th field and restart the task.

CEDA TRANSACTION (A / U actions) and operator-vi edits of transactions.conf both honour the field; the file is hot-reloaded on mtime change.

CONNECT TO 'name' (per-task database switch)

A program that needs to touch a second database during a task issues EXEC SQL CONNECT TO 'name'. The named database must exist in databases.conf (the CEDA DATABASE catalogue); an unknown name returns SQLCODE = -1. An in-flight PG transaction on the previous database is committed implicitly (matches DB2's behaviour) – programs that want to discard the previous work should EXEC SQL ROLLBACK before connecting.

```
EXEC SQL CONNECT TO 'orders' END-EXEC.
```

```
EXEC SQL INSERT INTO order_lines VALUES (:ID, :QTY) END-EXEC.
```

```
EXEC SQL COMMIT END-EXEC.
```

```
EXEC SQL CONNECT TO 'customers' END-EXEC.
```

```
EXEC SQL SELECT email INTO :EM FROM customers WHERE id = :CID END-EXEC.
```

Implicit binding still applies to the first statement of every task – the transaction's Database column in transactions.conf picks the starting pool. CONNECT TO is only needed when the program needs to move between databases inside a single task.

Null indicators (:hostvar :indicator)

DB2 programs use a second host var beside the value to signal NULL. Bricks supports the same pair syntax in both COBOL and REXX:

```
EXEC SQL
```

```
    SELECT name INTO :NM :NIND
```

```
    FROM customers_sql WHERE id = :CUSTID
```

```
END-EXEC.
```

```
EVALUATE NIND
```

```
    WHEN 0    DISPLAY 'Name: ' NM
```

```
    WHEN -1   DISPLAY 'Name is NULL'
```

```
END-EVALUATE.
```

Output side (SELECT INTO / FETCH INTO):

- Column was non-NULL → indicator = 0, value holds the result.
- Column was NULL → indicator = -1, value is cleared to the empty string so a program that ignored its indicator can't read a stale value from a previous statement.

Input side (WHERE / VALUES / SET clauses):

```
MOVE -1 TO NIND.
EXEC SQL
    INSERT INTO customers_sql (id, name) VALUES (:K, :NM :NIND)
END-EXEC.
```

- Indicator = -1 at bind time → bricks passes NULL to PG, regardless of what NM currently holds.
- Indicator = 0 (or unset) → the value var's frame contents bind normally.

Pair detection: a value/indicator pair is two :name tokens separated only by whitespace. :NM, :NIND (with a comma) means two independent host vars, not a pair. The INDICATOR keyword form (:NM INDICATOR :NIND) is not currently supported – use the juxtaposed form.

Column-value coercion

PG's wire format doesn't always match what COBOL or REXX want to read. Bricks normalises three cases before the value reaches the program's host variable:

- **bool columns** – PG returns "t" / "f" (or "true" / "false"). Bricks converts to "1" / "0" so COBOL PIC X(1) flags and REXX IF VAR = 1 idioms work the way they do in DB2.
- **NUMERIC(p, s) columns** – PG drops trailing fractional zeros for computed expressions (SELECT 1.5 returns "1.5" even though the result type has scale 2). Bricks pads the value to the column's declared scale, so a COBOL PIC 9(3)V99 target receives "1.50" → digits "150" → stored as 001.50, not 000.15.
- **CHAR(N) columns** – PG returns space-padded values. Bricks trims trailing spaces so a REXX literal compare (IF VAR = 'X') doesn't fail because the CHAR(5) value was "X ". VARCHAR is left alone (PG never pads it; its trailing spaces are real).

Other types (INT, BIGINT, TEXT, DATE, TIMESTAMP, JSON, ...) flow through verbatim. COBOL's MOVE semantics handle integer sign and zero-pad automatically; REXX is dynamically typed so the string form works for both display and arithmetic.

Restrictions

Out of scope	Why
Stored procedure calls (EXEC SQL CALL)	Out of scope — porters use plain SELECT-from-function.
Multi-row INSERT VALUES (...), (...) CREATE / DROP DATABASE USER	Out of scope; rewrite as one statement per row. CEDA DATABASE only (C / X actions, audit-logged).
Long-running cursors across EXEC CICS RETURN chains	Out of scope; each task gets a fresh PG connection.

Chapter 27. Restrictions and deferred features

This chapter is the canonical list of language features bricks does **not** accept. Anything not listed here can be assumed implemented; refer back to the relevant Chapter 14–26 section for the supported syntax.

COBOL — disallowed at parse time

- **Calculated GOTOs.** `GO TO ... DEPENDING ON expr` is rejected. Every task has its own heap and stack with no static control-block aliasing; supporting calculated GOTOs would require a per-program label-table the parser deliberately doesn't build.
- **Nested OCCURS.** An array of arrays is rejected at parse time. One level of `OCCURS` is fine.
- **EVALUATE TRUE / EVALUATE FALSE.** The conditional / computed form is rejected with a hint to rewrite as `IF/ELSE`. The simple-value form is supported.

COBOL — deferred (would be implementable, not yet wired)

Area	Specifics
DATA DIVISION	REDEFINES, USAGE COMP-1 (single float), USAGE COMP-2 (double float), SYNC, INDEXED BY, OCCURS DEPENDING ON, 66 / 78 levels, BLANK WHEN ZERO. USAGE COMP / COMP-4 / COMPUTATIONAL / BINARY and USAGE COMP-3 / COMPUTATIONAL-3 / PACKED-DECIMAL ARE supported — see Chapter 21.
Edited PIC	Edit characters +, -, CR, DB, B, 0, /. (Z, ., ,, \$, * are supported.)
PROCEDURE DIVISION	CALL (external program), SEARCH / SEARCH ALL, ALTER, INITIALIZE, ACCEPT, in-line PERFORM ... END-PERFORM (only paragraph-targeted PERFORM works), INSPECT CONVERTING, DIVIDE ... REMAINDER.
Intrinsic functions	Only UPPER-CASE, LOWER-CASE, LENGTH, NUMVAL, TRIM, REVERSE, POS are implemented. The statistical / date / time families are not. FUNCTION TRIM does NOT accept the LEADING / TRAILING modifier.
Copybooks	COPY ... REPLACING ==X== BY ==Y==., library qualifiers (COPY name OF lib), SUPPRESS.
EXEC SQL	Multi-row INSERT VALUES (...), (...). Stored-procedure calls (EXEC SQL CALL). DDL inside the executor — use <code>psql</code> or <code>CEDA DATABASE</code> .
EXEC CICS	SCREENHT-based map family suffix (e.g. CUST1L on a mod-4 screen) is REXX-only; COBOL twins always render the unsuffixed mod-2 map.

REXX — deferred / accepted but inert

Area	Specifics
Tracing Settings	TRACE is parsed but emits no trace output. OPTIONS ... parsed and accepted; recognised options are none. NUMERIC FORM SCIENTIFIC / ENGINEERING accepted syntactically, rendering is always plain decimal.
External routines	No dynamic linking — every callable is either a PROCEDURE in the program or a built-in.
Address environments	Only ADDRESS CICS is wired. ADDRESS COMMAND / ADDRESS SYSTEM / ADDRESS TSO are not recognised.
Terminal queue	PUSH, QUEUE, PULL, PARSE PULL are parsed; the queue is always empty. Use EXEC CICS RECEIVE for terminal input.
PARSE	PARSE SOURCE, PARSE VERSION, PARSE NUMERIC are not wired. PARSE LOWER is not parsed (only UPPER).
Conditions	FAILURE, NOTREADY, LOSTDIGITS are not implemented. HALT parses but never fires (bricks has no external-halt fan-in).
Numeric	Arithmetic is float64 internally, then rounded to NUMERIC DIGITS significant figures — strict IEEE behaviour is not preserved.
Built-ins	ERRORTEXT(code) echoes the code rather than returning the standard REXX error-text string.

Part 8. Sample programs

Part 8 begins here. Bricks ships a curated set of REXX and COBOL programs that exercise every command family in the earlier parts. Each transaction has been built to be operator- reachable from the blank prompt and self-explaining when run — reading the source is the fastest way to internalise the bricks programming model.

Chapter 28. Pre-installed sample transactions

COBOL

Transid	File	Notes
HELC	runtime/cobol/hello.cob	Hello-world; SEND MAP('HEL01') FROM(SCR), RETURN. Smallest end-to-end demo.

Transid	File	Notes
QAGC	runtime/cobol/qagc.cob	COBOL twin of QAGR (REXX). Validates the QAGE1 birthdate, computes age in years and approximate days, sends QAGR1. Pseudo-conversational redisplay of QAGE1 on validation errors.
GUST	runtime/cobol/gust.cob	COBOL CUST. A=Add, Q=Query, U=Update, D=Delete, L=List, S=Search; the S action LINKs to GUSL with the search term in COMMAREA and renders the match count returned.
GUSV	runtime/cobol/gusv.cob	COBOL twin of CUSV. Validates the customer-number COMMAREA (LINK target).
GUSL	runtime/cobol/gusl.cob	COBOL twin of CUSL. Renders the customers file via STARTBR / READNEXT / ENDBR. Blank inbound COMMAREA = paginated all-records list (PF7/PF8). Non-blank inbound COMMAREA = filtered single-page browse: every record is FUNCTION POS-tested against the upper-cased filter, the first 15 matches populate ROW1..ROW15, and the total match count flows back through DFHCOMMAREA so the caller (GUST) can render a summary.
EXAM	runtime/cobol/exam.cob	Worked example of reading the operator's command-line arguments. Type EXAM 1 2 3 at the blank prompt.

Transid	File	Notes
ORDR	runtime/cobol/ordr.cob	Conversational import: reads run-time/tmp/orders.sample.txt via READQ TD, parses pipe-delimited rows, and WRITE FILE('ORDERS') keyed on customer-id. Tolerates duplicates (EIBRESP = RESP-DUPREC). Summary screen shows counts. See worked example E.
TIMC	runtime/cobol/timc.cob	Timed-reminder demo: exercises CONVERSE, START (with INTERVAL + FROM), and RETRIEVE end-to-end. Shares tim1.map + tim2.map with TIMR.
SQLD	runtime/cobol/sqld.cob	Embedded-SQL demo: EXEC SQL SELECT name INTO :NM FROM customers_sql WHERE id = :CUSTID. Renders the row + SQLCODE / SQLSTATE / SQLERRMC. Requires Postgres seeded with the schema shown in Chapter 26.
SQLR	runtime/rexx/sqlr.rexx	REXX twin of SQLD. Shares the customers_sql table; demonstrates how REXX accesses the same EXEC SQL surface (SQLCODE / SQLSTATE / SQLERRMC arrive as plain REXX variables).

All five non-trivial COBOL samples (QAGC, GUST, GUSL, ORDR, EXAM) COPY DFHAID and/or COPY DFHRESP instead of hard-coding hex AID bytes or numeric EIBRESP codes. Skim any of them as living examples of the named-constant idiom from Chapter 25.

REXX (selected)

Transid	File	Notes
HELO	runtime/rexx/hello.rexx	Hello-world with system-info pane on mod 4.

Transid	File	Notes
CUST	runtime/rexx/cust.rexx	Full customer maintenance — Add / Query / Update / Delete / List / Search; LINKs to CUSV for validation.
CUSV	runtime/rexx/cusv.rexx	Validation sub-program; called by CUST via EXEC CICS LINK.
CUSL	runtime/rexx/cusl.rexx	Paginated customer list using STARTBR / READNEXT / ENDBR; adapts paging to mod 2 vs mod 4.
QAGE / QAGR	runtime/rexx/qage.rexx / qagr.rexx	Pseudo-conversational chain; QAGE prompts for a birthdate and chains to QAGR to render the result.
PROD / CONS	runtime/rexx/prod.rexx / cons.rexx	TS queue producer / consumer pair. Conversational; PF3 to exit.
GETC	runtime/rexx/getc.rexx	RECEIVE of command-line + READ FILE + SEND TEXT (no map).
TIMR	runtime/rexx/timr.rexx	REXX twin of TIMC: START schedules a reminder; RETRIEVE discriminates cold vs scheduled entry. Shares tim1.map + tim2.map with TIMC.

Transid	File	Notes
CHAT	runtime/rexx/chat.rexx	Real-time multi-user chat. Self-refreshes every 2 seconds via EXEC CICS START TRANSID('CHAT') INTERVAL(000002); the tick handler does SEND MAP ... DATAONLY so the operator's in-progress typing at the bottom of the screen is not clobbered by the refresh. Messages persist in the auto-created KSDS file CHATLOG (one record per message, key shape YYYYMMDDHHMMSS-NNNN-TTTT for lexicographic / chronological sort). Adapts between Model 2 (16 history rows, chatm2.map) and Model 4 (35 history rows, chatm4.map) via EXEC CICS ASSIGN SCREENHT. F3 exits cleanly — no further tick is scheduled, the self-refresh chain dies on its own. Colours mirror the original tsu/chat.go palette: BLUE/BRIGHT title, TURQUOISE clock + footer, YELLOW topic, RED status line, GREEN history rows + prompt, WHITE underscored input.

Run any TRANSID by typing it at the blank prompt after CSSN sign-on. Refer to runtime/transactions.conf for the full list and ACL configuration.

Bank-domain bindings. BANK, BALC, BALR, and BRDS carry an explicit :bank 5th field in transactions.conf so their EXEC SQL statements land on the retail-banking pool rather than the default bricks database. BALC / BALR / BRDS do not currently issue SQL, but they share the bank-domain map set and are pinned to the same pool for forward compatibility — adding an EXEC SQL later will Just Work without re-binding. See Chapter 26 — Database binding.

Built-in transactions (no entry in `transactions.conf`)

The bricks core dispatches a handful of TRANSIDs directly, without consulting `transactions.conf`. They take precedence over a same-name entry in the table.

Transid	Purpose	Notes
CSSN	Sign-on	Default authentication flow. Configurable via <code>secure_login_transacton</code> in <code>bricks.cnf</code> .
CSSF	Sign-off	CSSF LOGOFF clears the session's identity; bare CSSF is a no-op.
CEMT	Master-operator	INQUIRE / MONITOR / PERFORM trees; CONTROLBLOCKS sub-tree and PERFORM gated on the <code>admin</code> group.
CEDA	Resource definitions	TRANSACTION / PROGRAM / USER screens; admin-only.

Transid	Purpose	Notes
CECI	Command-level interpreter	Type one EXEC CICS / EXEC SQL / EXEC WEB command (up to 5 input rows), press PF5, see RESPONSE / RESP2 / LEN / ELAPSED plus any variables the handler set or INTO buffers it filled. The TxCB and cics.Handler are session-scoped — built once on CECI entry, torn down on PF3 / CLEAR / PA1 / disconnect. Each PF5 press is a per-PF5 commit boundary via h.CommitImplicit() on success / h.RollbackImplicit() on failure; SYNCPOINT in bricks does not touch cursors, so the per-task TS read cursor, all open browses, TD handles, WEB sessions, and DOCUMENT tokens survive across PF5 presses (a STARTBR on PF5 #1 drives READNEXT on PF5 #2). Variable frame is also session-scoped. READ UPDATE + REWRITE still cannot span PF5 — the per-PF5 implicit SYNCPOINT releases non-HOLD record locks; see the REWRITE deviation note for the recommended workaround. Verbs that would unwind the CECI task itself (RETURN, XCTL, LINK, ABEND), repaint the screen (SEND MAP, SEND TEXT, RECEIVE MAP, CONVERSE), schedule deferred work (START, RETRIEVE, CANCEL, DELAY), or set per-task trap tables (HANDLE, IGNORE, WHENEVER) are refused with a short red-status message. Every command is wrapped in a 7-second wall-clock cap; a cap-hit <i>poisons</i> the session (every subsequent PF5 short-circuits with TIMEOUT, and the session-end defer skips the handler-owned closers to avoid racing the abandoned goroutine — PF3 is the only exit). Gated on the dev group and the same gate applies to

Transid	Purpose	Notes
ISPF	Source editor	Browse and edit the REXX, COBOL, and BMS-map source trees. Gated on the dev group. Operator manual: <code>ISPF_editor.md</code> — covers every PF key, every command-line word, every line-prefix command (D / I / C / M / R / U / L /) / (/ X / O / A / B plus the doubled block forms), the file browser, the warn-then-save flow, multi-file editing, and edit locks.

Every built-in TRANSID above can be reached via `EXEC CICS LINK PROGRAM(name)` and `EXEC CICS XCTL PROGRAM(name)` as well as by typing it at the blank prompt. The two paths share one dispatch helper, so the per-built-in auth gate (dev for ISPF/CECI, admin for CEDA/IDCA) fires uniformly regardless of how the operator reached it. CEMT carries a mixed model — the outer menu is open to any signed-on user (INQUIRE TRANSACTION, MONITOR), and the `CONTROLBLOCKS / PERFORM` sub-trees gate on admin inside `walk()`. The ACL check at the LINK / XCTL boundary refuses a denied caller before the built-in's Run is reached: the LINK returns the access-denied error to `programControl` which surfaces `PGMIDERR` via `EIBRESP`, and the XCTL renders the familiar 403 task-error screen — no internal refusal paint hijacks the caller's session. Built-ins do not have a COMMAREA-style interface; `LINK PROGRAM('CEMT') COMMAREA(VAR)` returns an empty COMMAREA (VAR is set to "" on return) and the caller's `sess.Commarea` is restored unchanged. `CSSN` is the one exception: it refuses LINK/XCTL with a short message because the auth-prompt loop cannot be wedged into a caller's task stack sanely; operators sign on at a blank prompt.

Chapter 29. Worked examples

A. Producer / consumer over a TS queue

PROD writes one item per ENTER from an interactive map; CONS reads with the implicit per-task cursor. PF4 in CONS deletes the queue; PF5 rewinds the cursor by chaining back to itself (`RETURN TRANSID('CONS')`), so the dispatcher's task-end hook clears the cursor.

```
/* PROD: write one item per ENTER */
ADDRESS CICS
DO FOREVER
  EXEC CICS SEND      MAP('PROD1') FROM(SCR.) ERASE END-EXEC
  EXEC CICS RECEIVE MAP('PROD1')                END-EXEC
  IF C2X(EIBAID) = 'F3' THEN EXEC CICS RETURN END-EXEC
  EXEC CICS WRITEQ TS QUEUE(MAP.QNAME) FROM(MAP.PAYLOAD) END-EXEC
END
```

```

/* CONS: cursor-advancing read */
ADDRESS CICS
DO FOREVER
  EXEC CICS SEND      MAP('CONS1') FROM(SCR.) ERASE END-EXEC
  EXEC CICS RECEIVE MAP('CONS1')                END-EXEC
  IF C2X(EIBRID) = 'F3' THEN EXEC CICS RETURN END-EXEC
  EXEC CICS READQ TS QUEUE(QNM) INTO(REC) ITEM(GOTI) END-EXEC
  IF EIBRESP = 26 THEN /* ITEMERR - end of queue */ NOP
END

```

B. SEND TEXT + RECEIVE — no BMS map

GETC takes a customer number from the operator's command line (GETC 100), reads the customers KSDS, and paints the record as free-form text. Because **3270 has no LF**, the body is built as a flat row-major buffer — each logical line padded to the 80-column screen width with LEFT(s,80).

```

/* GETC: lookup-and-display, no map */
ADDRESS CICS
EXEC CICS RECEIVE INTO(BUF) END-EXEC          /* "GETC 100" */
PARSE VAR BUF TID CKEY .
EXEC CICS READ FILE('customers') INTO(REC) RIDFLD(CKEY) END-EXEC
PARSE VAR REC NM '|' AD '|' CY '|' PH
TXT = LEFT('Customer #' || CKEY, 80)          /* row 0 */
TXT = TXT || LEFT(' ', 80)                    /* row 1 (blank) */
TXT = TXT || LEFT('Name:   ' || NM, 80)       /* row 2 */
TXT = TXT || LEFT('Address: ' || AD, 80)      /* row 3 */
EXEC CICS SEND TEXT FROM(TXT) ERASE END-EXEC
EXEC CICS RETURN END-EXEC

```

The COBOL companion runtime/cobol/exam.cob shows the same RECEIVE INTO pattern with UNSTRING ... DELIMITED BY ' ' doing the tokenisation:

```

EXEC CICS RECEIVE INTO(WS-INPUT) LENGTH(WS-LEN) END-EXEC
UNSTRING WS-INPUT DELIMITED BY ' '
  INTO WS-TID WS-A WS-B WS-C
END-UNSTRING.

```

C. Browse with GENERIC prefix

A paginated customer list filtered by a 3-character key prefix:

```

EXEC CICS STARTBR FILE('CUSTOMERS')
          RIDFLD('NY-')
          GENERIC KEYLENGTH(3)
END-EXEC
DO FOREVER
  EXEC CICS READNEXT FILE('CUSTOMERS') INTO(REC) RIDFLD(K) END-EXEC
  IF EIBRESP = 20 THEN LEAVE          /* ENDFILE */
  CALL ADD_ROW K, REC
END

```



```
EXEC CICS ENDBR FILE('CUSTOMERS') END-EXEC
```

D. Synchronous LINK with COMMAREA

CUST LINKs to CUSV to validate a customer number:

```
SAV = CKEY                                /* in: number to validate */
EXEC CICS LINK PROGRAM('CUSV') COMMAREA(SAV) END-EXEC
PARSE VAR SAV STATUS '|' MSG              /* out: status, message */
IF STATUS <> 'OK' THEN ...
```

CUSV reads DFHCOMMAREA, runs its check, and writes the result back into DFHCOMMAREA before RETURN. The dispatcher hands the final value back to the caller's SAV.

E. Sequential import via READQ TD + WRITE FILE

The ORDR transaction (runtime/cobol/ordr.cob) demonstrates the “text file → VSAM” pattern that the tmp_dir sandbox is built for. The sample data ships in runtime/tmp/orders.sample.txt:

```
C0000001|WIDGET-A|10|19.95
C0000002|WIDGET-B|2|249.00
...
```

The loop body is just two EXEC CICS calls and an UNSTRING:

```
IMPORT-ONE.
  MOVE SPACES TO REC.
  EXEC CICS READQ TD QUEUE('orders.sample.txt') INTO(REC) END-EXEC.
  IF EIBRESP = 12 THEN
    MOVE 'Y' TO DONE-FLAG      *> QZERO -- end of file
  END-IF.
  IF EIBRESP = 0 THEN
    COMPUTE N-READ = N-READ + 1
    MOVE SPACES TO CUST-ID PRODUCT QTY PRICE
    UNSTRING REC DELIMITED BY '|'
      INTO CUST-ID PRODUCT QTY PRICE
    END-UNSTRING
    PERFORM WRITE-ORDER
  END-IF.

WRITE-ORDER.
  MOVE SPACES TO OREC.
  STRING PRODUCT DELIMITED BY SIZE
    '|' DELIMITED BY SIZE
    QTY DELIMITED BY SIZE
    '|' DELIMITED BY SIZE
    PRICE DELIMITED BY SIZE
  INTO OREC
  END-STRING.
  EXEC CICS WRITE FILE('ORDERS') FROM(OREC) RIDFLD(CUST-ID) END-EXEC.
```

```

IF EIBRESP = 0 THEN COMPUTE N-WRITE = N-WRITE + 1 END-IF.
IF EIBRESP = 14 THEN COMPUTE N-DUP = N-DUP + 1 END-IF.

```

Notes:

- The READQ loop tracks EOF with `EIBRESP = 12 (QZERO)` — the CICS-canonical “queue empty / no more records” signal. The handle is closed automatically at task end.
- `EIBRESP = 14 (DUPREC)` is treated as a counted no-op, not an error: `WRITE FILE` against an existing key fails atomically without overwriting, so a re-run of `ORDR` on the same file is idempotent on the customer-id set.
- `ORDR` ships `public` in `runtime/transactions.conf` so the sample runs out of the box. In a production deployment that imports real data, restrict it to a privileged group (`ORDR:cobol:ordr.cob:admin`) so a casual operator can’t replay the import.
- A REXX equivalent reads the same file via `LINEIN` and writes via `EXEC CICS WRITE`. Because both languages share the `tmp_dir` backend, the sample file works untouched from either side.

Appendix A. Adapting to terminal size (mod 2 vs mod 4)

A 3270 connection negotiates one of several screen models — typically mod 2 (24 - 80) or mod 4 (43 - 80). Bricks captures the size from the telnet/3270 handshake into `session.TCB.Rows/Cols`, and exposes it to programs via `EXEC CICS ASSIGN`:

```

EXEC CICS ASSIGN SCREENHT(SCRH) SCREENWD(SCRW)
                ALTSCRNHT(AH) ALTSCRNWD(AW) END-EXEC

```

`SEND MAP` always passes the negotiated `DevInfo` through to `go3270.ScreenOpts.AltScreen`, so the underlying datastream uses Erase/Write Alternate (`0x7e`) and the terminal clears its full buffer — but a 24-row map painted on a 43-row screen leaves rows 24-42 blank. To use the extra real estate the program has to dispatch to a sized map variant.

Convention

Author one map per model. The mod-2 map keeps its bare name; bigger models add a single-letter suffix:

Model	Suffix	Example map names
mod 2	(none)	HEL01, CUST1, CUST2, CUSTL
mod 3	M	HEL01M, CUST1M, ...
mod 4	L	HEL01L, CUST1L, CUST2L, CUSTLL
mod 5	W	HEL01W, ...

The REXX program builds the suffixed name once and dispatches:

```

EXEC CICS ASSIGN SCREENHT(SCRH) END-EXEC
SUFFIX = ''
IF SCRH >= 43 THEN SUFFIX = 'L'
ELSE IF SCRH >= 32 THEN SUFFIX = 'M'

```

```
EXEC CICS SEND MAP('HEL01' || SUFFIX) FROM(SCR.) ERASE END-EXEC
IF EIBRESP = 36 THEN DO                /* MAPFAIL - sized variant missing */
    EXEC CICS SEND MAP('HEL01') FROM(SCR.) ERASE END-EXEC
END
```

Three properties make this work without any DSL changes:

- 1. **Same field names across the family.** `helo1.map` and `helo1l.map` both declare `INFOLINE`, `GREETING`, `FOOTER`. The same `SCR.` stem feeds either one. Bonus tails (e.g. `INF01` / `INF02` / `ACT1` ...) are silently ignored on the smaller map (the renderer only writes values for fields that the map declares).
- 2. **MAPFAIL fallback.** If the suffixed map isn't on disk, the `SEND` returns `EIBRESP = 36` and the program retries with the bare name — so an operator who deletes `helo1l.map` doesn't break mod-4 connections; they just see the 24-80 layout.
- 3. **Paging arithmetic adapts at runtime.** `custl.rexx` reads `SCREENHT` and uses `ROWS_PER_PAGE = 35` on mod 4 vs `15` on mod 2, then picks `CUSTLL` vs `CUSTL` accordingly.

Bricks ships sized variants for every map the demo transactions use:

Mod-2 map (24-80)	Mod-4 sibling (43-80)
<code>runtime/map/helo1.map</code>	<code>runtime/map/helo1l.map</code> — adds system-information + recent-activity panes
<code>runtime/map/cust1.map</code> (menu)	<code>runtime/map/cust1l.map</code> — adds recent-activity history
<code>runtime/map/cust2.map</code> (detail)	<code>runtime/map/cust2l.map</code> — adds an audit-log pane
<code>runtime/map/custl.map</code> (15-row list)	<code>runtime/map/custll.map</code> (35-row list)

To see the difference: connect with `c3270 -model 2 localhost 2300` vs. `c3270 -model 4 localhost 2300`, sign on, and run `HELO` or `CUST`. The mod-4 view fills the bottom three-quarters of the screen with extra panels.

Appendix B. Pitfalls and idioms

REXX compound-symbol pitfall

`STEM.tail` with `tail` an *unset* symbol resolves to `STEM.<TAIL>` (a literal tail). With `tail` a *set* symbol it resolves to `STEM.<value-of-tail>` — so reusing a map field name (`OUT.BIRTH = ...` when `BIRTH` is also a local variable) silently writes the wrong tail.

The convention used by `runtime/rexx/cust.rexx` is to give locals distinct names from map fields (`AKT` vs `ACTION`, `CKEY` vs `CUSTNO`, `BSTR` / `NDAYS` vs `BIRTH` / `DAYS`).

COBOL data names across groups

A child name reused across groups is allowed; bare references that match more than one declaration must be qualified with `OF / IN` (`MOVE x OF DET TO y OF SCR`). Bare names that are unique across the program continue to work without qualification. Sibling duplicates within a single group are still rejected at parse time because no qualifier can disambiguate them.

3270 has no LF...obviously.

`SEND TEXT` lays its body out as a flat row-major buffer, every `cols` bytes wrapping to the next row. Programs that expect newline semantics must pad each logical line to the column width (`LEFT(s,80)` in REXX, `PIC X(80)` group children in COBOL) and concatenate.

Kinda obvious, but just making sure it's clear to folks who are new to 3270.

Reset stems before paginated reads

A REXX `STEM.` accumulator that's reused across pages will leak values from the previous page if not dropped. Use `DROP RECS.` (with the trailing dot) at the top of each pagination loop.

EIBAID comparison

In REXX, compare `C2X(EIBAID)` to a hex string: `IF C2X(EIBAID) = 'F3' THEN ...` (PF3). In COBOL, compare `EIBAID` directly to a hex literal: `IF EIBAID = X'F3' ....`

Appendix C. Quick command reference card

A one-page cheat-sheet of every command family. Use the chapter reference for full syntax, options, and condition codes.

EXEC CICS — terminal I/O (Chapter 4)

Command	Purpose
<code>SEND MAP(name) [FROM(stem.)] [ERASE]</code> <code>[CURSOR(p)]</code>	Paint a BMS map; wait for an AID.
<code>RECEIVE MAP(name) [INTO(stem.)]</code>	Pull the operator's response.
<code>CONVERSE MAP(name) FROM(s) INTO(s) [ERASE]</code>	Fused <code>SEND MAP+RECEIVE MAP</code> .
<code>SEND TEXT FROM(buf) [LENGTH(n)] [ERASE]</code>	Free-form row-major output.
<code>RECEIVE INTO(buf) [LENGTH(v)]</code>	Read the operator's command-line tail.

EXEC CICS — program control (Chapter 5)

Command	Purpose
<code>RETURN [TRANSID(id)] [COMMAREA(d)]</code>	End task; optional chain.
<code>XCTL PROGRAM(name) [COMMAREA(d)]</code>	Transfer control.
<code>LINK PROGRAM(name) [COMMAREA(v)]</code>	Synchronous sub-program call.
<code>ABEND [ABCODE(c)]</code>	Abnormal task termination.
<code>START TRANSID(id) [INTERVAL/TIME] [FROM(d)]</code> <code>[TERMID(t)]</code>	Schedule a deferred fire.
<code>RETRIEVE INTO(v) [LENGTH(v)]</code>	Pull the <code>START</code> payload.

EXEC CICS — system services (Chapter 6)

Command	Purpose
ASSIGN <FIELD>(v) ...	Read EIB / environment fields.
ASKTIME [ABSTIME(v)]	Refresh EIBDATE / EIBTIME (+ optional ABSTIME).
FORMATTIME ABSTIME(s) [DATE / TIME / YYYY... / DAYOFWEEK ...]	Decode an ABSTIME.

EXEC CICS — KSDS files (Chapter 7-8)

Command	Purpose
READ FILE(f) INTO(v) RIDFLD(k) [UPDATE]	Exact-key lookup.
WRITE FILE(f) FROM(d) RIDFLD(k)	Insert record.
REWRITE FILE(f) FROM(d)	Replace the READ ... UPDATE record.
DELETE FILE(f) [RIDFLD(k)]	Drop record.
STARTBR FILE(f) [RIDFLD(k)] [GTEQ/EQUAL] [GENERIC KEYLENGTH(n)]	Open browse.
READNEXT FILE(f) INTO(v) [RIDFLD(v)]	Forward one record.
READPREV FILE(f) INTO(v) [RIDFLD(v)]	Backward one record.
RESETBR FILE(f) RIDFLD(k)	Re-position within an open browse.
ENDBR FILE(f)	Close browse.

EXEC CICS — temporary storage / transient data (Chapter 9)

Command	Purpose
READQ TS QUEUE(q) INTO(v) [ITEM(n) NEXT]	Read one TS item.
WRITEQ TS QUEUE(q) FROM(d) [ITEM(n) REWRITE]	Append or rewrite TS item.
DELETEQ TS QUEUE(q)	Drop a TS queue.
READQ TD QUEUE(q) INTO(v)	Read next line from tmp_dir/q.
WRITEQ TD QUEUE(q) FROM(d)	Append line to tmp_dir/q.
DELETEQ TD QUEUE(q)	Delete a TD file.

EXEC CICS — recovery + condition (Chapter 10)

Command	Purpose
SYNCPPOINT	Commit pending unit of work.
SYNCPPOINT ROLLBACK	Undo pending unit of work.
HANDLE CONDITION cond(label) ...	Arm condition trap.
IGNORE CONDITION cond ...	Suppress condition trap.
HANDLE AID key(label) ...	Arm AID-key trap.
HANDLE ABEND LABEL(l) PROGRAM(p) CANCEL RESET	Arm abend exit.

EXEC SQL (Chapter 26)

Command	Purpose
EXEC SQL SELECT cols INTO :v, :v FROM ... END-EXEC	Single-row read; +100 = no-data, -811 = >1 row.
EXEC SQL INSERT INTO t (...) VALUES (...) END-EXEC	Insert.
EXEC SQL UPDATE t SET ... WHERE ... END-EXEC	Update; 0 rows is still NORMAL.
EXEC SQL DELETE FROM t WHERE ... END-EXEC	Delete.
EXEC SQL DECLARE c CURSOR FOR <select> END-EXEC	Declare cursor.
EXEC SQL OPEN c END-EXEC	Open cursor.
EXEC SQL FETCH c INTO :v, :v END-EXEC	Next row; +100 at end.
EXEC SQL CLOSE c END-EXEC	Close cursor.
EXEC SQL COMMIT WORK END-EXEC / ROLLBACK WORK	Transaction control.
EXEC SQL CONNECT TO 'db' END-EXEC	Per-task database switch.
EXEC SQL WHENEVER {SQLERROR NOT FOUND SQLWARNING} {CONTINUE GO TO label} END-EXEC	Declarative error handler.

EXEC CICS WEB — server side (Phase 1)

Command	Purpose
WEB EXTRACT METHOD(v) PATH(v) HOST(v) PORT(v) SCHEME(v) QUERYSTRING(v) ...	Inbound request meta.
WEB EXTRACT CERTIFICATE COMMONNAME(v) ORGANISATION(v) COUNTRY(v) SERIALNUM(v) ISSUER(v)	mTLS peer-cert fields.
WEB EXTRACT TCIPSERVICE(v) PORTNUMBER(v) IPADDRESS(v) CLIENT(v) AUTHENTICATE(v)	Listener inspection.
WEB READ HTTPHEADER(name) VALUE(v) [LENGTH(v)]	Inbound header by name.
WEB STARTBROWSE / READNEXT / ENDBROWSE HTTPHEADER	Walk inbound headers.
WEB READ QUERYPARM(name) VALUE(v)	Query parameter / path capture.
WEB STARTBROWSE / READNEXT / ENDBROWSE QUERYPARM	Walk parameters.
WEB READ FORMFIELD(name) VALUE(v)	Form-encoded field.
WEB STARTBROWSE / READNEXT / ENDBROWSE FORMFIELD	Walk form fields.
WEB RECEIVE INTO(v) [MAXLENGTH(n)] [LENGTH(v)] [MEDIATYPE(v)] [TYPE(v)]	Read raw request body.
WEB WRITE HTTPHEADER(name) VALUE(v)	Set outbound response header.
WEB SEND FROM(buf) [MEDIATYPE(s)] [STATUSCODE(n)]	Emit response.

Command	Purpose
WEB PARSE URL URL(s) SCHEMENAME(v) HOST(v) PORT(v) PATH(v) QUERYSTRING(v)	URL splitter.
WEB CONVERTTIME DATESTRING(s) ABSTIME(v)	RFC 1123 / 850 / asctime → ABSTIME.
WEB RETRIEVE DOCTOKEN(v)	Inbound body as DOCUMENT.

EXEC CICS WEB — client side (Phase 2)

Command	Purpose
WEB OPEN HOST(s) [PORT(n)] [SCHEME(s)] SESSTOKEN(v)	Open client session.
WEB OPEN URIMAP(name) SESSTOKEN(v)	Open via URIMAP.
WEB CONVERSE SESSTOKEN(t) METHOD(m) PATH(p) [FROM(b)] INTO(v) [STATUSCODE(v)] CONVERSE WEB ...	One-shot send + receive. Alias for WEB CONVERSE.
WEB SEND SESSTOKEN(t) METHOD(m) PATH(p) [FROM(b)] [QUERYSTRING(s)] [MEDIATYPE(s)]	Stage request.
WEB SEND URIMAP(name) METHOD(m) PATH(p) ... INTO(v)	One-shot via URIMAP.
WEB RECEIVE SESSTOKEN(t) INTO(v) [STATUSCODE(v)] [MEDIATYPE(v)] [MAXLENGTH(n)]	Fire staged request.
WEB CLOSE SESSTOKEN(t)	Release session.
WEB WRITE HTTPHEADER(name) VALUE(v) SESSTOKEN(t)	Set outbound <i>request</i> header.
WEB READ HTTPHEADER(name) VALUE(v) [LENGTH(v)] SESSTOKEN(t)	Read response header.
WEB STARTBROWSE / READNEXT / ENDBROWSE HTTPHEADER SESSTOKEN(t)	Walk response headers.
WEB EXTRACT SESSTOKEN(t) [SCHEME(v)] [HOST(v)] [PORT(v)] [PATH(v)]	Inspect session endpoint.
WEB EXTRACT URIMAP(name) [SCHEME(v)] [HOST(v)] [PORT(v)] [PATH(v)]	Inspect URIMAP entry.

EXEC CICS DOCUMENT

Command	Purpose
DOCUMENT CREATE DOCTOKEN(v) [SYMBOLLIST(s)] [LISTLENGTH(n)] [DELIMITER(s)]	New empty document.
DOCUMENT INSERT DOCTOKEN(t) FROM(b) [LENGTH(n)] [AT(pos)]	Append / splice bytes.
DOCUMENT INSERT DOCTOKEN(t) SYMBOL(name)	Substitute from symbol list.
DOCUMENT INSERT DOCTOKEN(t) DOCUMENT(other-tok) [AT(pos)]	Splice another document.

Command	Purpose
DOCUMENT SET DOCTOKEN(t) SYMBOLLIST(s) [DELIMITER(d)]	Re-bind symbol list.
DOCUMENT RETRIEVE DOCTOKEN(t) INTO(b) LENGTH(v)	Read assembled body.
DOCUMENT DELETE DOCTOKEN(t)	Release document.
WEB SEND DOCTOKEN(t) [STATUSCODE(n)] [MEDIATYPE(s)]	Emit document as response.

Standard copybooks

Copybook	Brings in
DFHAID	AID byte constants (ENTER, PF03, PA1, ...) and their DFH... aliases.
DFHRESP	EIBRESP condition constants (RESP-NORMAL / DFHRESP-NORMAL, etc.).
SQLCA	SQLCODE + SQLSTATE constants (SQL-OK, SQLSTATE-DUP-KEY, ...).
DFHWBSC	HTTP status codes (DFHRESP-WB-OK, ...).
DFHWBUH	Common header-name literals (WB-CONTENT-TYPE, ...).
DFHWBMT	MIME-type literals (WB-MT-JSON, ...).
DFHWBMETH	HTTP method literals (WB-GET, WB-POST, ...).
DFHWBSI	Session-token + body buffer templates.
DFHWBUM	URIMAP record fields.
DFHWBHB	Header-browse helpers (DFH-WB-HDR-NAME, ...).
DFHWBCC	TLS / mTLS cert-extract pack.
DFHDCDOC	DOCUMENT token + delimiter constants.

EIB fields (Chapter 11)

Field	Set by	Meaning
EIBAID	SEND MAP / RECEIVE MAP	AID byte.
EIBCPOSN	SEND MAP / RECEIVE MAP	1-based cursor position.
EIBCALEN	dispatcher entry	DFHCOMMAREA length.
EIBTRMID	dispatcher	Terminal ID.
EIBRESP / EIBRESP2	every EXEC CICS	Response code + secondary.
EIBDATE / EIBTIME	ASKTIME	0CYYDDD / HHMMSS.
EIBABCODE	abend trap	4-char abend code.
RC	every EXEC CICS	Mirror of EIBRESP.

SQLCA fields (Chapter 26)

Field	Type	Meaning
SQLCODE	PIC S9(9)	0 = OK, +100 = no-data, -N = error.
SQLSTATE	PIC X(5)	5-character ISO state code.
SQLERRMC	PIC X(70)	Human-readable PG error text.
SQLERRD3	PIC S9(9)	Rows-affected count after INSERT / UPDATE / DELETE.
SQLWARN0	PIC X(1)	'W' if any warning fired (Bricks rarely sets).

End of publication.